



Εισαγωγή στην Παράλληλη Επεξεργασία

Αθανάσιος Ι. Μάργαρης

**ΚΕΦΑΛΑΙΟ 6 Προγραμματισμός Μαζικά
Παράλληλων Επεξεργασιών**

Copyright © Εκδόσεις TZIOΛA, 2024. Με επιφύλαξη παντός νόμιμου δικαιώματος
ISBN 978-618-221-094-9



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Εισαγωγικές παρατηρήσεις

Στις μοντέρνες αρχιτεκτονικές υπολογιστών, ο υπολογιστικός φόρτος που σχετίζεται με την εκτέλεση μιας εφαρμογής, δεν ανατίθεται πλέον μόνο στην ΚΜΕ, αλλά και στην συσκευή GPU, η οποία πραγματοποιεί υπολογισμούς πάσης φύσεως και δεν περιορίζεται μόνο στην υποστήριξη της εκτέλεσης των εφαρμογών γραφικών.

Αυτή η συνεργασία ανάμεσα στην ΚΜΕ και στην GPU υποστηρίζεται από εξειδικευμένα εργαλεία ανάπτυξης κώδικα όπως είναι το περιβάλλον της CUDA (Compute Unified Device Architecture – Nov 2006) που απαιτεί την χρήση κάρτας γραφικών της NVIDIA

Οι GPUs ως αμιγώς παράλληλες διατάξεις, δεν μπορούν να εκτελέσουν σειριακό κώδικα με τον τρόπο που το κάνει η ΚΜΕ ενώ η μετάβαση από τα λίγες δεκάδες ή εκατοντάδες νήματα της ΚΜΕ στις δεκάδες χιλιάδες νήματα της GPU, απαιτεί ριζική μεταβολή του προγραμματιστικού μοντέλου καθώς και του τρόπου επιμερισμού του υπολογιστικού φόρτου στα διαθέσιμα νήματα.

Για τον λόγο αυτό, οι εφαρμογές CUDA διαθέτουν και σειριακή συνιστώσα και κατά συνέπεια, αποτελούνται, από δύο τμήματα κώδικα, εκ των οποίων το ένα εκτελείται στην ΚΜΕ, ενώ το άλλο εκτελείται στην GPU.

Εκτός από την CUDA υπάρχουν και άλλα συναφή περιβάλλοντα ανάπτυξης εφαρμογών όπως είναι η OpenCL, η OpenACC, η βιβλιοθήκη Thrust της C++ καθώς και η βιβλιοθήκη ArrayFire, ενώ σε επίπεδο υλικού ευρεία χρήση παρουσιάζουν οι διατάξεις FPGA (Field Programmable Gate Arrays).



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Η αρχιτεκτονική της CUDA

Το περιβάλλον της CUDA διαθέτει χαρακτηριστικά παρόμοια με το MPI και με το OpenMP και επιτρέπει την ανάπτυξη εφαρμογών που χρησιμοποιούν τον τύπο επιμερισμού GSLP κυρίως σε γλώσσες Python και C / C++.

Το υπολογιστικό σύστημα, αποτελείται από μία ΚΜΕ και από μία ομάδα συσκευών, οι οποίες χαρακτηρίζονται από την ύπαρξη ενός μεγάλου πλήθους ALUs που χρησιμοποιούνται ως μαζικά παράλληλοι επεξεργαστές.

Το περιβάλλον της CUDA είναι ιδανικό για εφαρμογές που χρησιμοποιούν το μοντέλο παραλληλίας δεδομένων, στο οποίο λαμβάνει χώρα ταυτόχρονη εφαρμογή της ίδιας αριθμητικής πράξης ή διαδικασίας, πάνω σε διαφορετικά σύνολα δεδομένων.

Η ανάπτυξη της εφαρμογής απαιτεί τον διαχωρισμό της σε υπολογισμούς, οι οποίοι θα ανατεθούν για εκτέλεση σε μία ομάδα νημάτων που πρόκειται να εκτελεστούν παράλληλα, αν και αυτό θα συμβεί με ασύγχρονο τρόπο, υπό την έννοια πως δεν έχουμε τον παραμικρό έλεγχο όσον αφορά στο πότε θα ξεκινήσει και θα τερματιστεί η λειτουργία αυτών των νημάτων

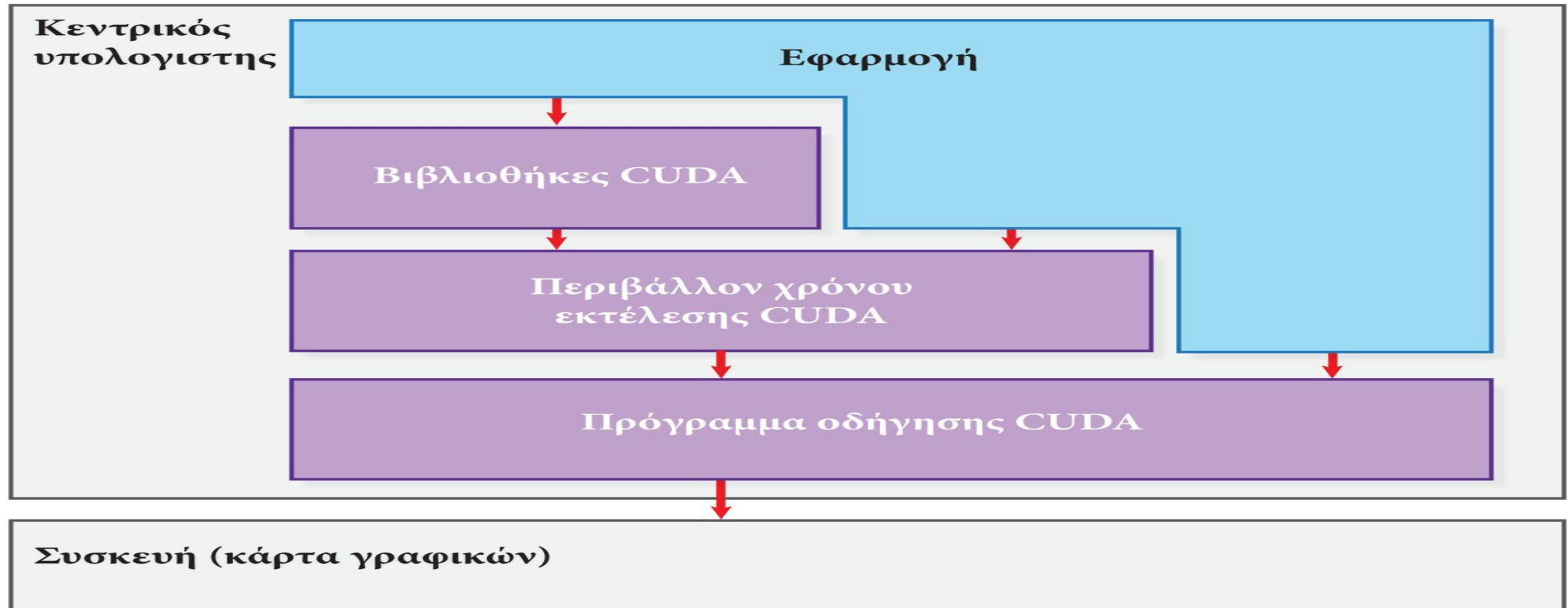
Ο προγραμματιστής θα πρέπει να λάβει αποφάσεις σχετικά με τον τρόπο εκτέλεσης ενός πολύ μεγάλου πλήθους νημάτων που μπορεί να περιλαμβάνουν εκατοντάδες ή ακόμα και χιλιάδες νήματα, καθώς και τον τρόπο με τον οποίο το καθένα από αυτά τα νήματα θα πρέπει να αρχικοποιηθεί, έτσι ώστε να αναλάβει την εκτέλεση διαφορετικού υπολογιστικού έργου.

Ο τρόπος εκτέλεσης των νημάτων στηρίζεται στην ταυτόχρονη εκκίνηση της λειτουργίας των νημάτων ως μία ομάδα, ενώ η ταυτοποίηση ενός νήματος στηρίζεται στην χρήση του κατάλληλου αναγνωριστικού



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Η πολυεπίπεδη αρχιτεκτονική του περιβάλλοντος της CUDA



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Νήματα και συναρτήσεις πυρήνα

Μία τυπική εφαρμογή **CUDA**, αποτελείται από ένα **σύνολο τμημάτων κώδικα** τα οποία χαρακτηρίζονται ως **φάσεις** και σχετίζονται είτε με **παράλληλους** είτε με **σειριακούς** υπολογισμούς.

Υφίστανται τρία επίπεδα αφαίρεσης:

- οι **ομάδες νημάτων**
- οι **κοινόχρηστες μνήμες**
- οι **μηχανισμοί συγχρονισμού** τύπου **χρονικής φραγής**

Οι **σειριακοί υπολογισμοί** μαζί με τους υπολογισμούς που χαρακτηρίζονται από **μικρό βαθμό παραλληλισμού** εκτελούνται στην **KME**, ενώ οι **υπόλοιποι υπολογισμοί** που χαρακτηρίζονται από **μεγάλο βαθμό παραλληλισμού**, εκτελούνται στην **GPU**

Αν και ο κώδικας της εφαρμογής είναι **ενιαίος** και περιλαμβάνει τόσο τον κώδικα που εκτελείται στην **KME** όσο και τον κώδικα που εκτελείται στην **GPU**, ωστόσο, ο **μεταγλωττιστής της NVIDIA** διαχωρίζει αυτούς τους δύο κώδικες.

Ο κώδικας της **KME** είναι ένας **συνήθης κώδικας της γλώσσας C**, ενώ αντίθετα, ο **κώδικας της GPU** είναι και αυτός **υλοποιημένος σε C**, αλλά περιλαμβάνει **πρόσθετες επεκτάσεις** με την μορφή **λέξεων κλειδιών** για τον προσδιορισμό των συναρτήσεων εκείνων που σχετίζονται με **παράλληλη εκτέλεση** και των εμπλεκόμενων **δομών δεδομένων**.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Νήματα και συναρτήσεις πυρήνα

Αυτές οι συναρτήσεις ονομάζονται **πυρήνες (kernels)**, ορίζονται μέσω της **δεσμευμένης λέξης `__global__`** και κατά την εκτέλεσή τους, δημιουργούν ένα **υπολογιστικό πλέγμα** που αποτελείται από ένα **μεγάλο πλήθος νημάτων**, οργανωμένων σε **ομάδες** για την **αξιοποίηση** της **παράλληλης δεδομένων**, οι οποίες δρομολογούνται για εκτέλεση στους **πολυεπεξεργαστές** της **GPU**

Επομένως, ένας **πυρήνας** περιέχει **κώδικα** που θα εκτελεστεί **από όλα** τα νήματα κατά τη διάρκεια μιας **φάσης παράλληλης**.

Επειδή **όλα τα νήματα** εκτελούν τον ίδιο κώδικα, ο **προγραμματισμός** σε **CUDA** ακολουθεί το μοντέλο προγραμματισμού **SPMD**

Οι **ομάδες νημάτων** του πλέγματος ανατίθενται για εκτέλεση στους **πολυεπεξεργαστές** της **GPU**, ενώ κάθε φορά που ένας πολυεπεξεργαστής **ολοκληρώνει** την εκτέλεση μιας **ομάδας νημάτων**, αναζητά για εκτέλεση την **επόμενη διαθέσιμη** ομάδα

Αυτή η διαδικασία πραγματοποιείται **ταυτόχρονα** από **όλους** τους **πολυεπεξεργαστές** μέχρι να **ολοκληρωθεί** η εκτέλεση **όλων των ομάδων νημάτων** γεγονός που σηματοδοτεί και την **ολοκλήρωση της εκτέλεσης** της παράλληλης εφαρμογής.

Από την στιγμή που μία ομάδα νημάτων **ανατεθεί** σε κάποιον πολυεπεξεργαστή για εκτέλεση, παραμένει σε αυτόν **για όλη την διάρκεια της ζωής της**, χωρίς να είναι δυνατή η μετακίνησή της σε **κάποιον άλλο** πολυεπεξεργαστή



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Τα στημόνια νημάτων και η χρονοδρομολόγησή τους

Σε μία **GPU**, ο κάθε πολυεπεξεργαστής περιέχει **καταχωρητές των 32 bits** που αναλαμβάνουν την **εκτέλεση** των νημάτων, ενώ τα νήματα που εκτελούνται στον **κάθε πολυεπεξεργαστή**, συνιστούν ομάδες **32 νημάτων**, οι οποίες ονομάζονται **στημόνια** (warps)

Το **πλήθος των στημονιών** ταυτίζεται με το **πλήθος των πυρήνων** του **πολυεπεξεργαστή**, με τα νήματα ενός στημονιού να χαρακτηρίζονται από **συνεχόμενες αυξανόμενες τιμές TIDs** και με το **πρώτο στημόνι** να περιέχει **πάντοτε** το νήμα με **TID = 0**.

Ο κάθε πολυεπεξεργαστής περιέχει και μία **παράλληλη μνήμη δεδομένων που διαμοιράζεται** ανάμεσα στις ομάδες νημάτων. Κατά την διάρκεια της **λειτουργίας** της συσκευής, λαμβάνει χώρα **παράλληλη εκτέλεση** τόσο των νημάτων που ανήκουν **στην ίδια ομάδα**, όσο και **αυτών των ίδιων** των ομάδων νημάτων.

Εάν κατά την **εκτέλεσή** τους, τα νήματα κάποιας ομάδας ζητήσουν **δεδομένα** από την **(αργή) καθολική μνήμη** της συσκευής, το σύστημα της **χρονοδρομολόγησης** μεταφέρει μία **δεύτερη ομάδα νημάτων** σε αυτόν τον πολυεπεξεργαστή για εκτέλεση.

Επομένως, ένας **πολυεπεξεργαστής** που μπορεί να εκτελεί **εκατοντάδες νήματα ταυτόχρονα**, δεν παραμένει ποτέ **ανενεργός**, ενώ το **πλήθος** των ομάδων νημάτων που μπορούν να ανατεθούν **ταυτόχρονα** σε έναν **πολυεπεξεργαστή**, εξαρτάται από το **πλήθος των καταχωρητών** και από την **διαθέσιμη ποσότητα** της **κοινόχρηστης μνήμης**.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Τα στημόνια νημάτων και η χρονοδρομολόγησή τους

Όλα τα νήματα ενός στημονιού, ξεκινούν από την ίδια διεύθυνση του προγράμματος, αν και το καθένα από αυτά διαθέτει τον δικό του μετρητή διεύθυνσης εντολών (instruction address counter) και χρησιμοποιεί τους δικούς του καταχωρητές.

Με τον τρόπο αυτό εκτελούνται ταυτόχρονα και ανεξάρτητα το ένα από το άλλο, ακολουθώντας ακόμη και διαφορετικές διαδρομές εκτέλεσης αν και επειδή όλα τα νήματα εκτελούν την ίδια εντολή σε κάθε χρονική στιγμή, η μέγιστη απόδοση επιτυγχάνεται όταν όλα τα νήματα ενός στημονιού ακολουθούν την ίδια διαδρομή εκτέλεσης.

Η προσωρινή απενεργοποίηση ενός νήματος είτε επειδή δεν ακολουθεί την τρέχουσα διαδρομή εκτέλεσης είτε επειδή έχει τερματίσει την λειτουργία του πιο νωρίς από τα υπόλοιπα είτε επειδή δεν χρησιμοποιείται, οδηγεί σε μείωση απόδοσης.

Ο κάθε πολυεπεξεργαστής σε μία σύγχρονη GPU, διαθέτει αρκετούς χρονοδρομολογητές που συντονίζουν την λειτουργία των στημονιών υλοποιώντας την παραλληλία σε επίπεδο εντολής (Instruction Level Parallelism, ILP).

Ο συντονισμός της λειτουργίας των πυρήνων ενός πολυεπεξεργαστή, πραγματοποιείται από μία ειδική ομάδα ελέγχου, η οποία διασφαλίζει πως η ίδια εντολή θα εκτελείται ταυτόχρονα από τα νήματα αυτών των ομάδων.

Με την ολοκλήρωση της εκτέλεσης όλων των νημάτων μιας ομάδας, το σύστημα αναθέτει στον πολυεπεξεργαστή για εκτέλεση, την λογικά επόμενη ομάδα, ο οποίος εκτελεί τα νήματα των παραπάνω ομάδων εκ περιτροπής, έτσι ώστε να μην γίνονται αντιληπτές οι χρονικές υστερήσεις που σχετίζονται με τις διαδικασίες προσπέλασης στη μνήμη.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Το μοντέλο προγραμματισμού SIMT – η επέκταση της ταξινόμιας του Flynn

Το παραπάνω μοντέλο προγραμματισμού μίας GPU είναι γνωστό ως μοντέλο SIMT (Single Instruction Multiple Thread) και αν και μοιάζει με τον τρόπο λειτουργίας των υπολογιστών SIMD, ωστόσο, δεν ταυτίζεται μαζί του.

Ειδικότερα, ενώ στην αρχιτεκτονική SIMD η κάθε εντολή εφαρμόζει παράλληλα την ίδια διαδικασία σε πολλά διαφορετικά στοιχεία, στο μοντέλο SIMT αντί να χρησιμοποιηθεί ένα απλό νήμα το οποίο εφαρμόζει διανυσματικές εντολές σε διανύσματα δεδομένων, υπάρχουν πολλά νήματα τα οποία εφαρμόζουν τις ίδιες εντολές σε αυθαίρετα δεδομένα

Οι εντολές εκτελούνται μέσω διοχέτευσης αξιοποιώντας, με τον τρόπο αυτό, την παραλληλία επιπέδου εντολής σε ένα απλό νήμα, καθώς και τον παραλληλισμό σε επίπεδο νήματος δια της ταυτόχρονης εκτέλεσης πολλών νημάτων.

Χρησιμοποιώντας αυτό το μοντέλο, οι προγραμματιστές έχουν την δυνατότητα να αναπτύξουν εφαρμογές σε επίπεδο νήματος, τόσο για νήματα που λειτουργούν ανεξάρτητα το ένα από το άλλο, όσο και για νήματα τα οποία συνεργάζονται μεταξύ τους.

Αυτή η προσέγγιση υποστηρίζει την ιδιότητα της κλιμάκωσης, αφού η συνολική απόδοση των υπολογισμών μπορεί να αυξηθεί απλά και εύκολα, διαμοιράζοντας τον υπολογιστικό φόρτο σε περισσότερους πολυεπεξεργαστές



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Οι τύποι συναρτήσεων της CUDA

Η λέξη `__global__` προστά από μία συνάρτηση, προσδιορίζει **κώδικα πυρήνα**, ο οποίος μπορεί να κληθεί **μόνο από συνάρτηση** που εκτελείται στην **KME**, προκειμένου να δημιουργήσει ένα **πλέγμα νημάτων** για εκτέλεση στην μονάδα **GPU**

Με άλλα λόγια, μία **συνάρτηση πυρήνα** καλείται από την **KME** αλλά η **εκτέλεσή της** πραγματοποιείται στη **μονάδα GPU** και κατά συνέπεια, **δεν είναι δυνατό** να επιστρέψει τιμή, αφού **εκτελείται σε άλλη διάταξη**.

```
/* Function Execution Space Specifiers */

__global__      /* A function is a kernel, callable from host, executed on device */
__device__     /* A function callable from device only, executed on device */
__host__       /* Callable from the host only, executed on the host */
__noinline__   /* A hint for the compiler not to inline the function if possible */
__forceinline__ /* Can be used to force the compiler to inline the function */
__inline_hint__ /* Enables more aggressive inlining in the compiler */

/* Variable Memory Space Specifiers */

__device__      /* Declares a variable that resides on the device's global memory */
__constant__   /* Declares a variable that resides in constant memory space */
__shared__     /* Declares a variable that resides in the shared memory space of a thread block */
__grid_constant__ /* Declared a const-qualified __global__ function parameter of non-reference type */
__managed__    /* Declares a variable that can be referenced from both device and host code */
__restrict__   /* Declares a restricted pointers used to to alleviate the aliasing problem */
```

Οι **βασικοί προσδιοριστές** του περιβάλλοντος της **CUDA** για τους **χώρους εκτέλεσης** και **περιοχής μνήμης**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η οργάνωση των νημάτων στην CUDA

Κατά την εκτέλεση ενός πυρήνα της CUDA, δημιουργείται ένα πλέγμα από νήματα που περιλαμβάνει χιλιάδες ή περισσότερα νήματα που εκτελούν όλα τον κώδικα του πυρήνα. Αυτό το πλέγμα συνιστά μία διευθέτηση νημάτων οργανωμένων συνήθως στις δύο ή στις τρεις διαστάσεις και με τα νήματα του πλέγματος να είναι οργανωμένα σε μία ιεραρχία δύο επιπέδων.

Το κάθε πλέγμα αποτελείται από μία ή περισσότερες ομάδες νημάτων που περιέχουν όλες το ίδιο πλήθος νημάτων και έχουν τις ίδιες διαστάσεις, γεγονός που υποδηλώνει πως τα νήματα σε αυτές τις ομάδες, είναι οργανωμένα με τον ίδιο τρόπο.

Η κάθε ομάδα νημάτων έχει τη μορφή ενός τρισδιάστατου πίνακα νημάτων με συνολικό μέγεθος έως και 1024 νήματα αν και δεν είναι υποχρεωτικό να χρησιμοποιηθούν και οι τρεις αυτές διαστάσεις.

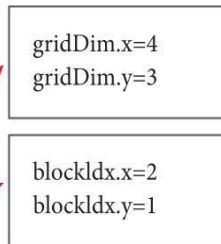
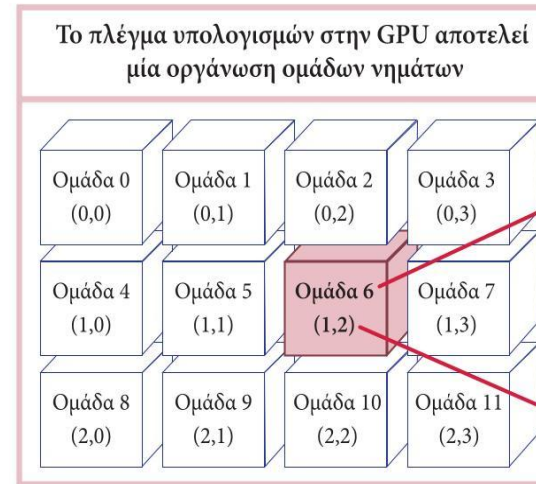
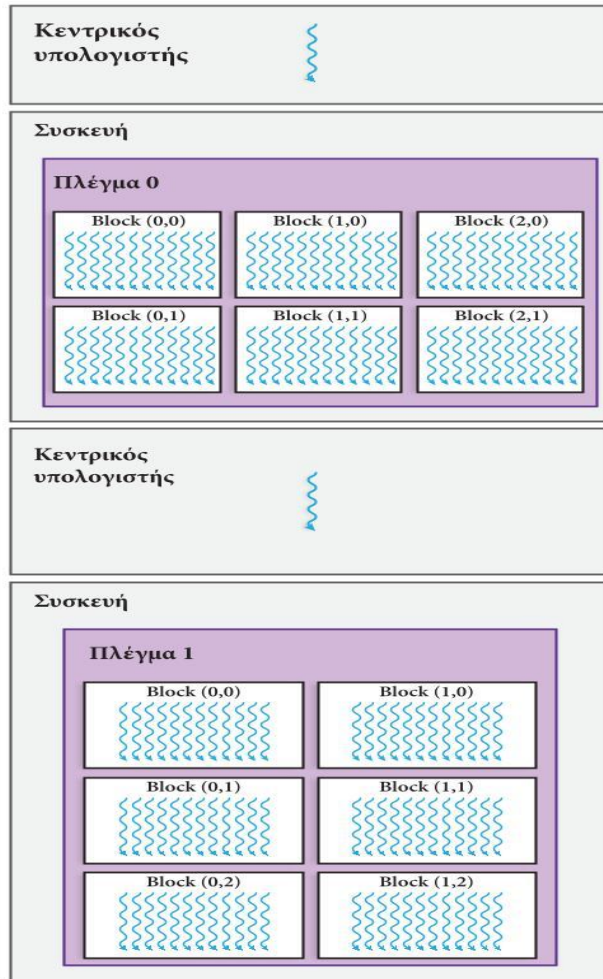
Αυτά τα νήματα της CUDA χαρακτηρίζονται από πολύ μικρότερο βάρος σε σχέση με τα νήματα του OpenMP, αφού λόγω της κατάλληλης υποστήριξης από το υλικό, απαιτούν λίγους μόνο κύκλους ρολογιού για να δημιουργηθούν και να χρονοδρομολογηθούν, σε αντίθεση με τα νήματα της CPU, τα οποία απαιτούν, αντίστοιχα, χιλιάδες κύκλους του ρολογιού.

Μετά την ολοκλήρωση της εκτέλεσης όλων των νημάτων ενός πυρήνα, η λειτουργία του πλέγματος τερματίζεται και η εκτέλεση της παράλληλης εφαρμογής συνεχίζεται πλέον μόνο στην ΚΜΕ, μέχρι να ανακύψει ένα άλλο τμήμα παράλληλου κώδικα το οποίο οδηγεί στην δημιουργία ενός νέου πυρήνα που συμπεριφέρεται με τον ίδιο ακριβώς τρόπο.

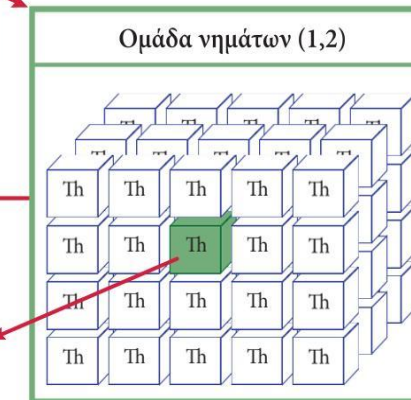
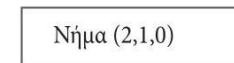
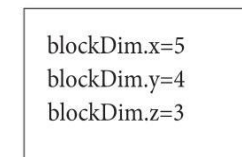


Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η οργάνωση των νημάτων στην CUDA



```
struct uint3 {  
    unsigned x, y, z; };  
  
typedef uint3 dim3  
  
extern "C" { extern const uint3 threadIdx; }  
extern "C" { extern const uint3 blockIdx; }  
extern "C" { extern const dim3 blockDim; }  
extern "C" { extern const dim3 gridDim; }  
extern "C" { extern const int warpSize; }
```



Η οργάνωση δύο επιπέδων των νημάτων στο περιβάλλον της CUDA

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η οργάνωση των νημάτων στην CUDA

Σε συσκευές με υπολογιστική ικανότητα 9.0 ή και μεγαλύτερη, ορίζεται ένα επιπλέον επίπεδο ιεραρχίας (Thread Block Clusters), που επιτρέπει την ομαδοποίηση των ομάδων νημάτων σε ακόμη μεγαλύτερες ομάδες ή συστοιχίες, τα νήματα των οποίων είναι εγγυημένο πως θα εκτελεστούν στην ίδια μονάδα GPC (GPU Processing Cluster)

Οι συντεταγμένες ενός νήματος στην δομή του πλέγματος Το κάθε νήμα σε ένα πλέγμα CUDA, ταυτοποιείται και περιγράφεται με την βοήθεια των κατάλληλων συντεταγμένων που το προσδιορίζουν με μοναδικό τρόπο μέσα στην εφαρμογή

Για κάθε νήμα διατηρούνται δύο δείκτες, ο δείκτης `blockIdx` που προσδιορίζει την ομάδα στην οποία ανήκει ένα νήμα και ο δείκτης `threadIdx` που προσδιορίζει το νήμα μέσα στην ομάδα του με τις τιμές αυτών των δεικτών, να αρχικοποιούνται από το περιβάλλον εκτέλεσης της CUDA και να μπορούν να ανακτηθούν χρησιμοποιώντας τις κατάλληλες συναρτήσεις.

Άλλες δύο μεταβλητές είναι οι `gridDim` και `blockDim`, εκ των οποίων η πρώτη επιστρέφει την διάσταση του συνολικού πλέγματος που δημιουργείται κατά την εκτέλεση πυρήνα, ως μία τριάδα (G_x, G_y, G_z) , ενώ η δεύτερη περιέχει την διάσταση της ομάδας στην οποία ανήκει το νήμα, εκπεφρασμένη ως μία τριάδα (G_x, G_y, G_z) , που είναι η ίδια για όλες τις ομάδες.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η διαδικασία δημιουργίας ενός πλέγματος νημάτων

Η ιεραρχική δομή των νημάτων στην CUDA ορίζεται κατά την εκτέλεση της συνάρτησης ενός πυρήνα με κλήση της μορφής

```
kernelName <<< gridInfo, blockInfo, [sharedMem], [stream] >>> ([kernel arguments]); όπου
```

- **kernelName** είναι το όνομα της συνάρτησης του πυρήνα
- **blockInfo** είναι το συνολικό πλήθος των ομάδων σε ένα πλέγμα (υποχρεωτικό όρισμα)
- **gridInfo** είναι το συνολικό πλήθος των νημάτων σε μία ομάδα (υποχρεωτικό όρισμα)
- **sharedMem** είναι το πλήθος των bytes της κοινόχρηστης μνήμης (προαιρετικό όρισμα)
- **stream** είναι το ρεύμα εντολών που σχετίζεται με την κλήση πυρήνα (προαιρετικό όρισμα)

Η κοινόχρηστη μνήμη που προσδιορίζεται από το τρίτο όρισμα, δεσμεύεται δυναμικά, με την ποσότητα που προσδιορίζεται από αυτό το όρισμα, να ορίζει την κοινόχρηστη μνήμη ανά ομάδα νημάτων

Εάν υποθέσουμε πως η δομή πλέγματος του σχήματος υλοποιείται κατά την κλήση μιας συνάρτησης πυρήνα με όνομα **myKernel**, ο κώδικας που δημιουργεί αυτή τη δομή, θα περιέχει μεταξύ άλλων και τις επόμενες τρεις γραμμές.

```
dim3 blockInfo (5,4,3);  
dim3 gridInfo (4,3);  
myKernel <<< gridInfo, blockInfo >>> ();
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η διαδικασία δημιουργίας ενός πλέγματος νημάτων

Το **μέγιστο πλήθος** των νημάτων σε μία ομάδα είναι **1024 νήματα**, ενώ εάν απαιτούνται **περισσότερα** νήματα, θα πρέπει να χρησιμοποιηθούν **περισσότερες ομάδες νημάτων** με **ίδιο** μέγεθος που όλες εκτελούν **την ίδια** συνάρτηση πυρήνα.

Αυτό οφείλεται στο γεγονός πως **όλα τα νήματα μιας ομάδας** εκτελούνται **στον ίδιο πολυεπεξεργαστή** και διαμοιράζονται την **κοινόχρηστη** μνήμη αυτού του επεξεργαστή, το μέγεθος της οποίας είναι γενικά **περιορισμένο**.

Το **μέγεθος** των **ομάδων** και του **πλέγματος** θα πρέπει να **ταιριάζει** στα δεδομένα της εφαρμογής και να **μεγιστοποιεί** το πλήθος των **ενεργών νημάτων** σε κάθε χρονική στιγμή, το οποίο με τη σειρά του, επηρεάζεται από το **ποσοστό χρήσης** της **κοινόχρηστης μνήμης**, των **καταχωρητών** και το **μέγεθος** της **ομάδας νημάτων**. Οι παραπάνω παράμετροι εξαρτώνται από

- το **μοντέλο** της **κάρτας** και την **υπολογιστική της ικανότητα**
- την **ποσότητα** της **κοινόχρηστης μνήμης**

Επειδή ο κάθε **πολυεπεξεργαστής**, περιέχει **32 πυρήνες** και αποστέλλει οδηγίες εκτέλεσης σε **ομάδες 32 νημάτων**, το μέγεθος των ομάδων πρέπει να είναι **ακέραιο πολλαπλάσιο του 32** έτσι ώστε να **μην οδηγούμαστε** σε **σπατάλη** υπολογιστικών πόρων.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η διαδικασία δημιουργίας ενός πλέγματος νημάτων

Οι διαστάσεις των δομών του πλέγματος, εξαρτώνται από τις απαιτήσεις και επομένως, μία ομάδα νημάτων, μπορεί να είναι μονοδιάστατη (1D), δισδιάστατη (2D) ή τρισδιάστατη (3D).

Οι τιμές των δεικτών `threadIdx`, `blockIdx`, `gridDim` και `blockDim`, επιτρέπουν την ταυτοποίηση του νήματος και την ανάθεση σε αυτό τμήματος υπολογιστικού έργου, κάτι που απαιτεί τον προσδιορισμό των δεδομένων εισόδου και εξόδου του νήματος

Αυτές οι δομές αντικαθιστούν τις δομές των επαναληπτικών βρόχων της γλώσσας C, οι οποίοι δεν είναι πλέον αναγκαίοι

Η οργάνωση των νημάτων του σχήματος απεικονίζει την γενική περίπτωση, ενώ εάν δεν επιθυμούμε να χρησιμοποιήσουμε όλες τις διαστάσεις, μπορούμε να θέσουμε τις μεταβλητές για τις διαστάσεις που δεν χρειαζόμαστε, σε τιμή ίση με τη μονάδα

Το μέγιστο πλήθος των νημάτων ανά ομάδα και η μέγιστη διαστατικότητα του πλέγματος, εξαρτώνται από την υπολογιστική ικανότητα x.y της συσκευής. Για την **NVIDIA GTX 680**, το μέγιστο πλήθος νημάτων για τις διαστάσεις x, y και z είναι αντίστοιχα **1024**, **1024** και **64**. Θα είναι, λοιπόν,

$$\text{blockDim.x} \times \text{blockDim.y} \times \text{blockDim.z} = 1024$$

ενώ επιπλέον, θα ισχύει ότι

$$1 \leq \text{blockDim.x} \leq 1024 \quad 1 \leq \text{blockDim.y} \leq 1024 \quad 1 \leq \text{blockDim.z} \leq 64$$



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η διαδικασία δημιουργίας ενός πλέγματος νημάτων

Ο τρόπος με τον οποίο χρησιμοποιούνται οι δείκτες των νημάτων και των ομάδων τους για την ταυτοποίηση του κάθε νήματος, εξαρτάται από την τοπολογία του εκάστοτε προβλήματος και ειδικότερα, από την διαστατικότητα της κάθε ομάδας νημάτων

Στην περίπτωση της μονοδιάστατης ομάδας, το κάθε νήμα προσδιορίζεται από μία τιμή έστω `threadIdx` που υπολογίζεται ως

$$\text{threadIdx} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

Στην περίπτωση των δύο διαστάσεων, το κάθε νήμα προσδιορίζεται από το ζεύγος (`threadIdx`, `threadIdx.y`) όπου

$$\text{threadIdx} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

$$\text{threadIdx.y} = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}$$

Στην περίπτωση των τριών διαστάσεων το κάθε νήμα ταυτοποιείται από την τριάδα (`threadIdx`, `threadIdx.y`, `threadIdx.z`) όπου

$$\text{threadIdx} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

$$\text{threadIdx.y} = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}$$

$$\text{threadIdx.z} = \text{blockIdx.z} \times \text{blockDim.z} + \text{threadIdx.z}$$



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η διαδικασία δημιουργίας ενός πλέγματος νημάτων

Το **μέγιστο πλήθος** των ομάδων νημάτων σε **κάθε διάσταση** του πλέγματος, περιγράφεται από τις τιμές των παραμέτρων **gridDim.x**, **gridDim.y** και **gridDim.z** οι οποίες ικανοποιούν **εξ ορισμού** τις συνθήκες

$$\begin{array}{lll} 0 \leq \text{blockDim.x} < \text{gridDim.x} & 0 \leq \text{blockDim.y} < \text{gridDim.y} & 0 \leq \text{blockDim.z} < \text{gridDim.z} \\ 0 \leq \text{threadIdx.x} < \text{blockDim.x} & 0 \leq \text{threadIdx.y} < \text{blockDim.y} & 0 \leq \text{threadIdx.z} < \text{blockDim.z} \end{array}$$

Αν και η τιμή της παραμέτρου **threadIdx** είναι μοναδική στο εσωτερικό της ομάδας του νήματος, ωστόσο, η ύπαρξη δύο νημάτων διαφορετικών ομάδων που χαρακτηρίζονται **από την ίδια** τιμή **threadIdx**, είναι **επιτρεπτή**.

Προκειμένου να κατασκευάσουμε ένα **αναγνωριστικό** για το νήμα που θα είναι **μοναδικό** σε **καθολικό επίπεδο**, θα πρέπει να θεωρήσουμε το **κάθε νήμα**, ως ένα **στοιχείο** ενός πίνακα που ορίζεται σε έναν χώρο **έξι διαστάσεων** της μορφής¹

Thread t [gridDim.z] [gridDim.y] [gridDim.x] [blockDim.z] [blockDim.y] [blockDim.x]

Υπολογίζοντας την **απόσταση** του νήματος από την **αρχή** αυτού του πίνακα των **έξι διαστάσεων**, μπορούμε να υπολογίσουμε ένα **αναγνωριστικό** η τιμή του οποίου να είναι **μοναδική σε καθολικό επίπεδο**, από την εξίσωση

```
int myID1 = (blockIdx.z * blockDim.x * blockDim.y + blockIdx.y * blockDim.x + blockIdx.x) *
            blockDim.x * blockDim.y * blockDim.z +
            threadIdx.z * blockDim.x * blockDim.y + threadIdx.y * blockDim.x + threadIdx.x
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η διαδικασία δημιουργίας ενός πλέγματος νημάτων

Εναλλακτικά μπορούμε να θεωρήσουμε το **κάθε νήμα** ως στοιχείο ενός **πίνακα τριών διαστάσεων** της μορφής

```
Thread t [gridDim.z*blockDim.z] [gridDim.y*blockDim.y] [gridDim.x*blockDim.x]
```

Στην περίπτωση αυτή οι **συντεταγμένες** του **αναγνωριστικού** του νήματος που είναι **μοναδικό** σε **καθολικό επίπεδο** είναι οι

```
int x = blockIdx.x * blockDim.x + threadIdx.x  
int y = blockIdx.y * blockDim.y + threadIdx.y  
int z = blockIdx.z * blockDim.z + threadIdx.z
```

Σε αυτή την **νέα περιγραφή**, το **μοναδικό** σε **καθολικό επίπεδο** αναγνωριστικό του κάθε νήματος υπολογίζεται από την εξίσωση

```
int myID2 = threadIdx.x + blockIdx.x * blockDim.x + (blockIdx.y * blockDim.y + threadIdx.y) *  
            blockDim.x + (blockIdx.z * blockDim.z + threadIdx.z) *  
            blockDim.x * blockDim.y
```

και φυσικά έχει **την ίδια τιμή** με αυτή που **υπολογίζεται** και **καταχωρείται** στο αναγνωριστικό **myID1**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Ιδιωτικές και κοινόχρηστες μεταβλητές

Η **εμβέλεια** μιας **μεταβλητής** προσδιορίζει το **σύνολο των νημάτων** τα οποία μπορούν να **προσπελάσουν μία μεταβλητή**, το οποίο μπορεί να περιλαμβάνει **μόνο ένα** νήμα, **όλα** τα νήματα μιας **ομάδας**, ή **όλα** τα νήματα όλων των **πλεγμάτων**.

Εάν η μεταβλητή είναι **ιδιωτική** και η εμβέλειά της περιορίζεται **σε ένα και μοναδικό** νήμα, λαμβάνει χώρα δημιουργία στην **στοίβα** του νήματος, ενός **ιδιωτικού αντιγράφου** αυτής της μεταβλητής, που προσπελάσσεται **μόνο από αυτό** το νήμα.

Όσον αφορά στον **χρόνο ζωής** της μεταβλητής, υπάρχουν **δύο** ενδεχόμενα:

- είτε η μεταβλητή είναι **διαθέσιμη** μέσα στην **κλήση** ενός πυρήνα
- είτε η μεταβλητή είναι **διαθέσιμη** κατά την διάρκεια της εκτέλεσης **ολόκληρης** της εφαρμογής.

Στην πρώτη περίπτωση, η μεταβλητή θα πρέπει να δηλωθεί **μέσα στον κώδικα του πυρήνα** αποτελώντας με τον τρόπο αυτό **τοπική μεταβλητή** αυτής της **συνάρτησης πυρήνα**, ενώ στην **δεύτερη περίπτωση**, κατά την οποία η μεταβλητή είναι **καθολική**, θα πρέπει να δηλωθεί **έξω από το σώμα** όλων των συναρτήσεων του πηγαίου κώδικα.

Στην περίπτωση αυτή, το **περιεχόμενο** της μεταβλητής διατηρείται **σε όλη τη διάρκεια** της εκτέλεσης και είναι διαθέσιμο σε **όλους** τους πυρήνες της εφαρμογής



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Ασύγχρονη ταυτόχρονη εκτέλεση

Η **CUDA** προσφέρει την δυνατότητα της **ταυτόχρονης εκτέλεσης** ενός **πλήθους** από **σημαντικές διαδικασίες**, όπως είναι

- η πραγματοποίηση **πάσης φύσεως υπολογισμών** στον **κεντρικό υπολογιστή** και στην συσκευή **GPU**
- η **μεταφορά δεδομένων** από την **μνήμη του κεντρικού υπολογιστή** στη **μνήμη της συσκευής GPU** και αντίστροφα
- η **μεταφορά δεδομένων** ανάμεσα σε **διευθύνσεις μνήμης** που ανήκουν στην **ίδια** ή σε **διαφορετικές** συσκευές

Το επίπεδο του **ταυτοχρονισμού** που μπορεί να επιτευχθεί, εξαρτάται από τα **χαρακτηριστικά** και την **υπολογιστική ικανότητα** της συσκευής και υποστηρίζεται από **ειδικές συναρτήσεις**, που **επιστρέφουν τον έλεγχο** στο νήμα που τις έχει καλέσει **πριν** από την **ολοκλήρωση** της θεωρούμενης διαδικασίας στην συσκευή, η οποία μπορεί να εκτελεστεί **ταυτόχρονα** με άλλες διαδικασίες

Η **ασύγχρονη ταυτόχρονη εκτέλεση** πυρήνων, υποστηρίζεται από συσκευές με υπολογιστική ικανότητα **2.x** ή μεγαλύτερη, η τιμή της οποίας καθορίζει και το **μέγιστο πλήθος** των **ταυτόχρονων εκτελέσεων** συνάρτησης πυρήνα.

Η **υποστήριξη** αυτού του χαρακτηριστικού εξαρτάται από την τιμή του πεδίου **concurrentKernels** της δομής **cudaDeviceProp** που επιστρέφεται από την συνάρτηση **cudaGetDeviceProperties** και το οποίο έχει τιμή ίση με **1 (YES)**, ή **0 (NO)**.

Η **χρονική επικάλυψη** ανάμεσα στις **διαδικασίες εκτέλεσης πυρήνα** και στις **διαδικασίες ασύγχρονης μεταφοράς δεδομένων**, είναι δυνατή εφόσον η τιμή του πεδίου **asyncEngineCount** της δομής δεδομένων **cudaDeviceProp** που επιστρέφεται από τη συνάρτηση **cudaGetDeviceProperties**, είναι **μεγαλύτερη** του μηδενός.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Οι διατάξεις GPU αποτελούν ξεχωριστές και αυτόνομες κάρτες επέκτασης, που διαθέτουν την δική τους μνήμη, η οποία φυσικά είναι διαφορετική και ανεξάρτητη από την κεντρική μνήμη RAM που χρησιμοποιεί η κεντρική μονάδα επεξεργασίας.

Επομένως η ΚΜΕ και η GPU έχουν η καθεμία τον δικό της ξεχωριστό χώρο μνήμης και ως εκ τούτου, οι υπολογισμοί που πραγματοποιούνται σε αυτές τις δύο διατάξεις, δεν ανταγωνίζονται για την δέσμευση της αναγκαίας ποσότητας μνήμης.

Ως συνέπεια, η μεταβίβαση δεδομένων στον πυρήνα με τη μορφή ενός δείκτη προς έναν πίνακα που είναι αποθηκευμένος στην μνήμη της ΚΜΕ, δεν είναι δυνατή (αν και υφίστανται κάποιες δυνατότητες).

Επιπλέον θα πρέπει να υφίσταται η δυνατότητα ανταλλαγής δεδομένων εισόδου και εξόδου των πυρήνων ανάμεσα στις μνήμες της ΚΜΕ και της GPU, καθώς και αποδέσμευσης της μνήμης της GPU που είχε δεσμευτεί για αυτό το σκοπό.

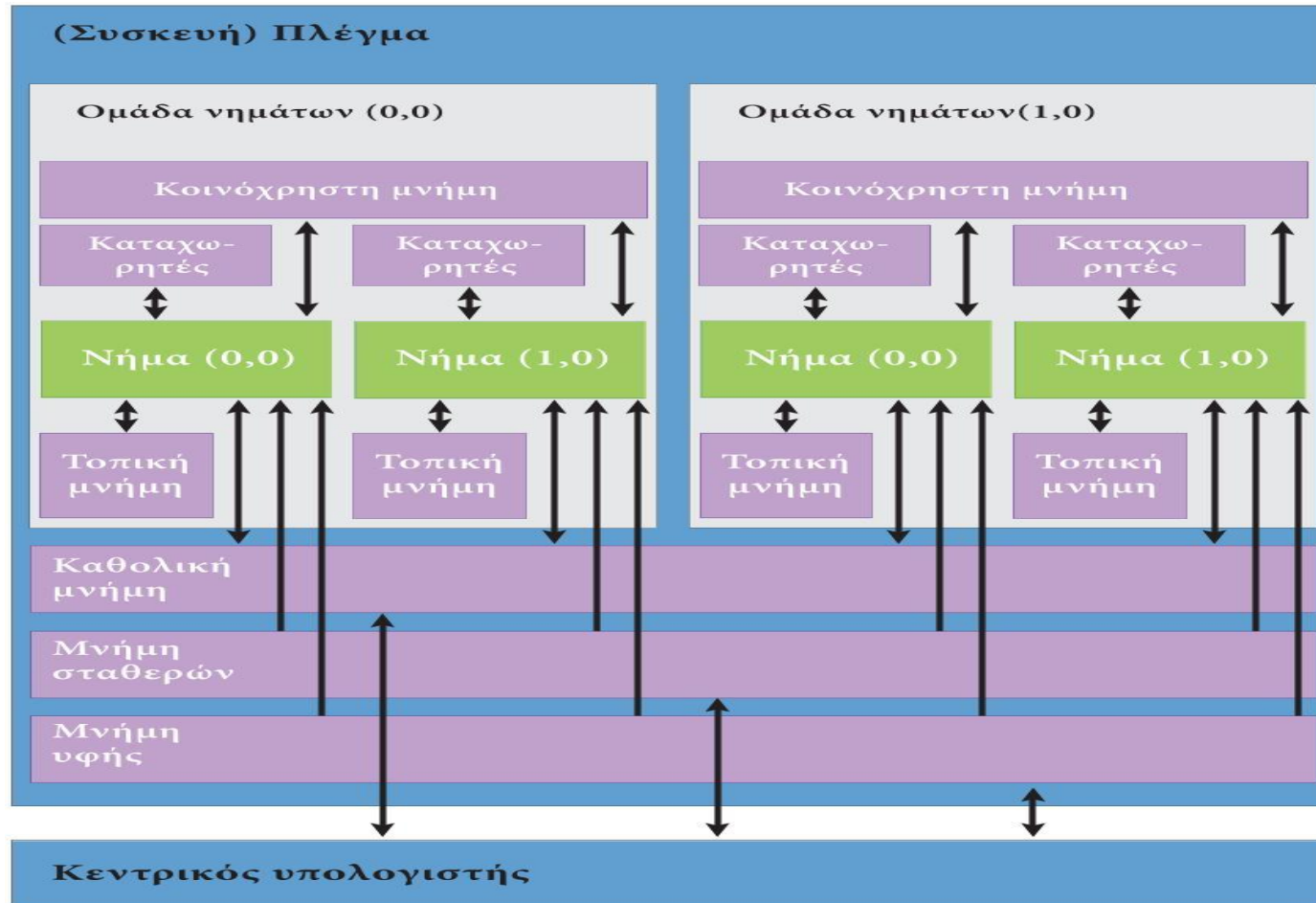
Συνήθως αυτή η μνήμη δεσμεύεται ως γραμμική μνήμη και περιγράφεται ως ένας γραμμικός χώρος διευθύνσεων, με τις διαφορετικές περιοχές μνήμης που δεσμεύονται σε αυτόν τον χώρο να προσπελούνται χρησιμοποιώντας δείκτες. Το μέγεθος αυτού του χώρου, εξαρτάται από το σύστημα του κεντρικού υπολογιστή και από την υπολογιστική ικανότητα της συσκευής.

Η δέσμευση και η αποδέσμευση αυτής της γραμμικής μνήμης, πραγματοποιούνται με τις συναρτήσεις `cudaMalloc` και `cudaFree`



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA



Μία **συσκευή GPU** περιλαμβάνει τις επόμενες **περιοχές μνήμης**

1. Καταχωρητές
2. Τοπική μνήμη
3. Κοινόχρηστη μνήμη
4. Κρυφή μνήμη L1
5. Κρυφή μνήμη L2
6. Καθολική μνήμη
7. Μνήμη σταθερών
8. Μνήμη υφής

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Καταχωρητές και τοπική μνήμη Οι καταχωρητές αποτελούν μικρές μνήμες ενσωματωμένες στο ολοκληρωμένο κύκλωμα.

Ο κάθε πολυεπεξεργαστής διαθέτει το δικό του σύνολο καταχωρητών, που προσφέρεται στους πυρήνες CUDA για την αποθήκευση των μεταβλητών που χρησιμοποιούνται από τα νήματα που εκτελούνται σε αυτούς.

Οι καταχωρητές χαρακτηρίζονται από πολύ μεγάλη ταχύτητα προσπέλασης και η χρήση τους οδηγεί σε πολύ μικρότερους χρόνους εκτέλεσης, σε σχέση με αυτούς που θα προέκυπταν εάν οι μεταβλητές ήταν αποθηκευμένες σε πιο αργές μνήμες

Οι καταχωρητές κατανέμονται σε μεμονωμένα νήματα, με το κάθε νήμα να προσπελαύνει μόνο τους δικούς του καταχωρητές στους οποίους αποθηκεύει συνήθως τις ιδιωτικές του μεταβλητές οι οποίες προσπελούνται με μεγάλη συχνότητα.

Εάν το νήμα έχει χρησιμοποιήσει όλους τους καταχωρητές του και εξακολουθούν να υφίστανται πρόσθετες απαιτήσεις αποθήκευσης, χρησιμοποιείται για τον σκοπό αυτό, κάποιο τμήμα της τοπικής μνήμης του νήματος.

Το πλήθος των καταχωρητών που χρησιμοποιούνται για το κάθε νήμα, επηρεάζει και το μέγιστο πλήθος των νημάτων που μπορούν να είναι ταυτόχρονα ενεργά σε έναν πολυεπεξεργαστή (resident threads), ενώ η χρήση ενός αρκετά μεγάλου πλήθους καταχωρητών για το κάθε νήμα αυξάνει την απόδοση, αφού δεν χρειάζεται προσπέλαση στην πιο αργή κοινόχρηστη μνήμη.

Ο λόγος των ενεργών στημονιών νημάτων προς το μέγιστο πλήθος των ενεργών στημονιών νημάτων είναι γνωστός ως πληρότητα και μία επιθυμητή τιμή για αυτό το μέγεθος είναι μία τιμή κοντά στη μονάδα



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Κοινόχρηστη μνήμη και κρυφή μνήμη L1 Η **κοινόχρηστη μνήμη** αποτελεί μία **μνήμη RAM**, **ενσωματωμένη** στο κύκλωμα που χαρακτηρίζεται από **πολύ μεγαλύτερη** ταχύτητα σε σχέση με την **ταχύτητα πρόσβασης** της **καθολικής μνήμης** και διαμοιράζεται σε μία ομάδα νημάτων, για διαδικασίες **ανάγνωσης** και **εγγραφής**.

Με άλλα λόγια, αυτή η μνήμη **διαμοιράζεται** ανάμεσα στους **πυρήνες CUDA** που ανήκουν **στον ίδιο πολυεπεξεργαστή** και **εκτελούν** αυτά τα νήματα, οδηγώντας σε **αποδοτικούς τύπους επικοινωνίας δεδομένων** ανάμεσα σε αυτούς τους πυρήνες.

Ο **χρόνος χρήσης** της **κοινόχρηστης μνήμης**, **ταυτίζεται** με τον **χρόνο ζωής της ομάδας νημάτων** που εκτελείται στον **πολυεπεξεργαστή** που την περιέχει, σε αντίθεση με τον χρόνο χρήσης των **υπόλοιπων** τύπων μνήμης, ο οποίος υπαγορεύεται από τον **χρόνο εκτέλεσης της εφαρμογής**, ακόμη και εάν αυτή εκτελεί πολλούς και διαφορετικούς πυρήνες.

Η **δέσμευση κοινόχρηστης μνήμης** μπορεί να πραγματοποιηθεί τόσο με **στατικό** τρόπο, όταν το **απαιτούμενο** μέγεθος της **κοινόχρηστης μνήμης** είναι **γνωστό** κατά την διάρκεια της **μεταγλώττισης**, όσο και με **δυναμικό** τρόπο, όταν οι απαιτήσεις σε κοινόχρηστη μνήμη γίνονται γνωστές **μόνο κατά την διάρκεια του χρόνου εκτέλεσης**.

- Στην **πρώτη περίπτωση**, οι μεταβλητές που επιθυμούμε να χαρακτηριστούν ως **κοινόχρηστες** περιλαμβάνουν στη δήλωσή τους τη δεσμευμένη λέξη **__shared__**
- Στην **δεύτερη περίπτωση**, χρησιμοποιούνται δείκτες με έναν **συγκεκριμένο τρόπο** και απαιτείται η χρήση ενός **τρίτου ορίσματος** στην συνάρτηση του πυρήνα, που προσδιορίζει το μέγεθος της κοινόχρηστης μνήμης σε **bytes**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Η **κοινόχρηστη μνήμη** της συσκευής είναι **άρρηκτα συνδεδεμένη** με την **κρυφή μνήμη**, αφού **αμφότερες** η **κρυφή μνήμη L1** και η **κοινόχρηστη μνήμη**, αποτελούν τμήματα του **ιδίου ολοκληρωμένου κυκλώματος του πολυεπεξεργαστή**, το οποίο διαμερίζεται σε **μικρότερα τμήματα** που χρησιμοποιούνται είτε ως **κρυφή** είτε ως **κοινόχρηστη μνήμη**

Οι εμπλεκόμενες συναρτήσεις είναι οι **cudaDeviceSetCacheConfig** και **cudaFuncSetCacheConfig**

Η **κοινόχρηστη μνήμη** είναι **άρρηκτα** συνδεδεμένη με την χρήση **ατομικών λειτουργιών**, ενώ επειδή τα νήματα εκτελούνται σε **ομάδες νημάτων**, η καθεμία των οποίων εκτελείται σε **επίπεδο στημονιού**, απαιτείται η εφαρμογή **διαδικασιών συγχρονισμού** ανάμεσα στις διάφορες **φάσεις εκτέλεσης** του πυρήνα, έτσι ώστε να αποφευχθεί η εμφάνιση **συνθηκών ανταγωνισμού**

```
/* Synchronization Functions */
```

```
__syncthreads          /* coordinates communication between the threads of the same block          */  
__syncthreads_count    /* as __syncthreads() plus it returns the # of threads with !=0 predicate val    */  
__syncthreads_and      /* as __syncthreads plus plus it returns TRUE if all threads have !=0 pred val    */  
__syncthreads_or       /* as __syncthreads plus plus it returns TRUE if any threads has a !=0 pred val    */  
__syncwarp             /* coordinates communication between the threads of the same warp              */
```

Οι **εσωτερικές συναρτήσεις συγχρονισμού** του **περιβάλλοντος** της **CUDA**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Το μοντέλο μνήμης της CUDA

Η κοινόχρηστη μνήμη διαχωρίζεται σε ζώνες ίσου μεγέθους που επιτρέπουν ταυτόχρονες διαδικασίες προσπέλασης υπό υψηλό εύρος ζώνης αφού οι διαδικασίες ανάγνωσης ή εγγραφής N διευθύνσεων μνήμης που εκτείνονται σε b διακριτές ζώνες, πραγματοποιούνται ταυτόχρονα, οδηγώντας σε ενεργό εύρος ζώνης με τιμή b φορές μεγαλύτερη από εκείνη μιας απλής ζώνης.

Σε περιπτώσεις κατά τις οποίες δύο ή περισσότερα νήματα προσπαθούν να προσπελάσουν δύο διαφορετικές διευθύνσεις που ανήκουν στην ίδια ζώνη της κοινόχρηστης μνήμης, λαμβάνει χώρα η εμφάνιση σύγκρουσης, η επίλυση της οποίας μπορεί να πραγματοποιηθεί μόνο μέσω της ακολουθιακής εκτέλεσης των νημάτων που συγκρούονται

Ενδεικτικοί κανόνες χρήσης της κοινόχρηστης μνήμης (για περισσότερους κανόνες δείτε τη σελίδα 805):

- Εάν νήματα που ανήκουν στο ίδιο στημόνι προσπελαίνουν δεδομένα που ανήκουν σε διαφορετικές ζώνες της κοινόχρηστης μνήμης, αυτή η διαδικασία προσπέλασης είναι στιγμιαία, δηλαδή πραγματοποιείται σε έναν κύκλο του ρολογιού.
- Εάν δύο ή περισσότερα νήματα της εφαρμογής, προσπελαίνουν διαφορετικά στοιχεία που ανήκουν στην ίδια ζώνη, λαμβάνει χώρα η εμφάνιση σύγκρουσης, που οδηγεί στην πραγματοποίηση της προσπέλασης με σειριακό τρόπο.
- Εάν δύο ή περισσότερα νήματα της εφαρμογής, πραγματοποιούν διαδικασία ανάγνωσης από θέσεις της κοινόχρηστης μνήμης που βρίσκονται μέσα στην ίδια λέξη μήκους 32 bits, τότε δεν εμφανίζεται σύγκρουση.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Η χρήση της **κρυφής μνήμης** στις συσκευές με **υπολογιστική ικανότητα** 3.x ή μεγαλύτερη, πραγματοποιείται ως εξής:

Εάν τα δεδομένα που έχουν ζητηθεί από το νήμα ενός στημονιού είναι **αποθηκευμένα** στην **κρυφή μνήμη L1**, λαμβάνει χώρα **μία και μοναδική προσπέλαση** ενός τμήματος μνήμης μεγέθους **128 bytes**, αφού **αυτό ακριβώς** είναι το μέγεθος των περιοχών στις οποίες διαμερίζεται η **κρυφή μνήμη L1**, προκειμένου, στη συνέχεια, αυτές να απεικονιστούν στην κεντρική μνήμη.

Στην **αντίθετη περίπτωση**, κατά την οποία τα δεδομένα που έχουν ζητηθεί **δεν βρίσκονται** στην **κρυφή μνήμη L1**, αυτά ανακτώνται από την **κρυφή μνήμη L2**

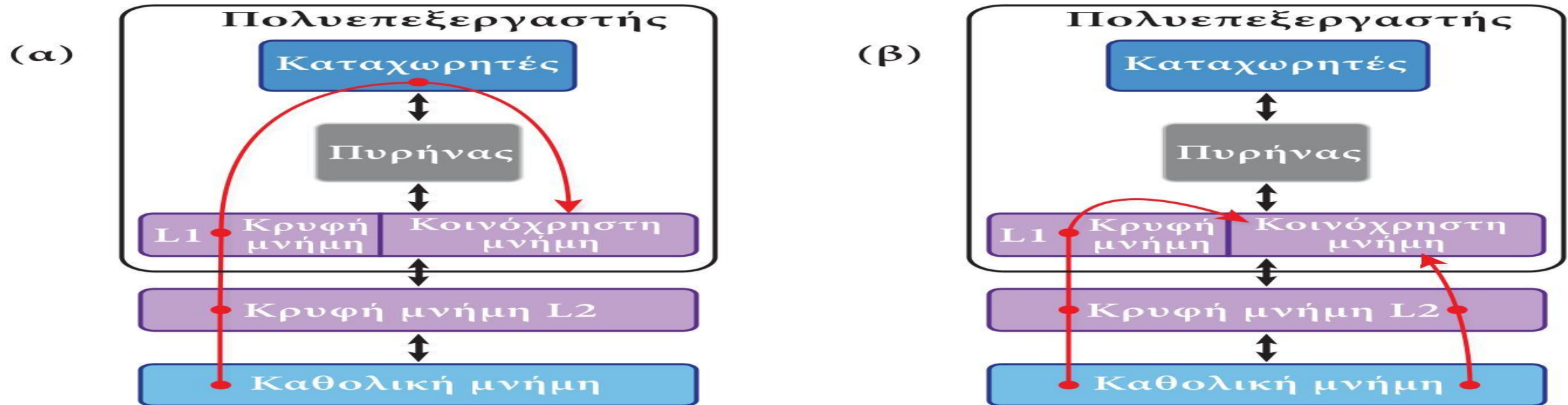
Η **μεταφορά των δεδομένων** από την **κρυφή μνήμη L2** στην **κρυφή μνήμη L1** πραγματοποιείται σε **ομάδες δεδομένων** μεγέθους **32 bytes**, προκειμένου να **αποφευχθεί** η μεταφορά **μεγάλου όγκου δεδομένων** ανάμεσα σε αυτές τις δύο μνήμες.

Η **επιλογή των δεδομένων** που θα μεταφερθούν στην **κρυφή μνήμη L1**, πραγματοποιείται **αυτόματα** από τον **μεταγλωττιστή**, αν και υφίσταται η **δυνατότητα** για τον **προγραμματιστή** να δηλώσει **και αυτός** τις δικές του προτιμήσεις, χρησιμοποιώντας στην δήλωση μιας μεταβλητής την δεσμευμένη λέξη **const** και στην δήλωση ενός δείκτη τη δεσμευμένη λέξη **__restricted__**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA



Η **συνήθης μεταφορά** δεδομένων από την **καθολική** στην **κοινόχρηστη** μνήμη [εικόνα (α)] και η **ασύγχρονη μεταφορά** δεδομένων που στηρίζεται στην χρήση της συνάρτησης **memcpy_async** [εικόνα (β)].

Αυτή η μεταφορά μπορεί να πραγματοποιηθεί **με δύο τρόπους** (με κριτήριο το εάν τα δεδομένα μετά από την **κρυφή μνήμη L2**, **διέρχονται** από την **κρυφή μνήμη L1** ή **αποστέλλονται απευθείας** στην **κοινόχρηστη** μνήμη)

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Κρυφή μνήμη L2 Κατά την εκτέλεση των **παράλληλων εφαρμογών** στο περιβάλλον της **CUDA**, οι προσπελάσεις που λαμβάνουν χώρα στην **καθολική μνήμη**, μπορεί να είναι είτε **διαρκείς**, είτε **περιστασιακές**

Λόγω της **σημαντικότητας επίδρασης** που έχουν οι **διαρκείς προσπελάσεις** στην **καθολική μνήμη** στην **απόδοση** του συστήματος, το περιβάλλον της **CUDA** από την **Έκδοση 11.0** και μετά, προσφέρει **ειδικές συναρτήσεις** που επιτρέπουν την **διαχείριση** της **κρυφής μνήμης L2** και οι οποίες υποστηρίζονται από συσκευές με **υπολογιστική ικανότητα 8.0 ή μεγαλύτερη**

Τα πεδία της δομής **cudaDeviceProp** που σχετίζονται με την χρήση της **κρυφής μνήμης L2** είναι τα

- **persistingL2CacheMaxSize** που ορίζει το **μέγιστο μέγεθος** της **κρυφής μνήμης L2** σε bytes
- **l2CacheSize** που περιέχει το **συνολικό μέγεθος** της **κρυφής μνήμης L2** σε bytes

Για να ορίσουμε ένα **άλλο μέγεθος** για αυτή την περιοχή, μικρότερο από **persistingL2CacheMaxSize**, καλούμε τη συνάρτηση

```
cudaDeviceSetLimit (cudaLimitPersistingL2CacheSize, newSize);
```

Μία **εναλλακτική τεχνική διαμόρφωσης** της **κρυφής μνήμης L2**, στην περίπτωση κατά την οποία χρησιμοποιούμε δομές γράφων, στηρίζεται στην χρήση της **δομής δεδομένων** που φέρει το όνομα **accessPolicyWindow** και η οποία αποτελεί πεδίο της δομής **cudaKernelNodeAttribute** (ανατρέξτε στα αρχεία τεκμηρίωσης του CUDA Toolkit)



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Καθολική μνήμη Αποτελεί μία **περιοχή μνήμης**, στην οποία έχουν πρόσβαση **όλα τα νήματα του πλέγματος**, βρίσκεται στα **χαμηλότερα επίπεδα** της **ιεραρχίας μνήμης** της συσκευής και χρησιμοποιείται για την αποθήκευση δεδομένων τα οποία **ανταλλάσσονται** ανάμεσα στην **KME** και στην **GPU**

Η **διαχείριση** αυτής της μνήμης πραγματοποιείται από την **KME**, η οποία **δεσμεύει** και **αποδεσμεύει** τις **απαραίτητες** περιοχές **καθολικής μνήμης**. Αυτή η μνήμη αποτελεί και την **μοναδική μνήμη** που **προσπελάσιμη** από συναρτήσεις της CUDA που εκτελούνται στην **KME**, ενώ **όλες οι υπόλοιπες** μνήμες είναι **προσπελάσιμες** από συναρτήσεις που εκτελούνται **μόνο** στην GPU

Η **μεταφορά** των δεδομένων πραγματοποιείται σε **ομάδες** των **32**, **64** ή **128** bytes κάθε φορά, ενώ δύο **συνιστώμενες** τεχνικές διαχείρισης οι οποίες μπορούν να θεωρηθούν ως **προκλήσεις** που θα πρέπει να ικανοποιούνται είναι οι εξής:

- τα νήματα που ανήκουν στο **ίδιο στημόνι** είναι προτιμότερο να προσπελαίνουν στοιχεία **της ίδιας ομάδας δεδομένων**
- τα δεδομένα θα πρέπει να είναι **ευθυγραμμισμένα**, έτσι ώστε οι διαδικασίες προσπέλασης **να μην ξεπερνούν** τα όρια που διαχωρίζουν δύο **διαδοχικές** ομάδες δεδομένων (κάτι που γίνεται χρησιμοποιώντας την δεσμευμένη λέξη **`__align__`**)

Η **δέσμευση** και η **αποδέσμευση** της καθολικής μνήμης πραγματοποιείται με τις συναρτήσεις

```
__host__ __device__ cudaError_t cudaMalloc (void** devPtr, size_t size);  
__host__ __device__ cudaError_t cudaFree (void* devPtr);
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Σε αντίθεση με την συνάρτηση `malloc` της `C`, η οποία δεσμεύει μία περιοχή μνήμης το μέγεθος της οποίας μπορεί να μεταβληθεί με την συνάρτηση `realloc`, η περιοχή μνήμης στην συσκευή που δεσμεύεται από την `cudaMalloc` δεν διαθέτει αυτό το χαρακτηριστικό και κατά συνέπεια, εάν ο χρήστης επιθυμεί να προβεί σε μία τέτοια διαδικασία, θα πρέπει

- να δεσμεύσει μία μεγαλύτερη περιοχή μνήμης με το κατάλληλο μέγεθος
- να αντιγράψει τα δεδομένα από την αρχική σε αυτή τη νέα περιοχή
- να απελευθερώσει την αρχική περιοχή που είχε δεσμεύσει
- να χρησιμοποιήσει από το σημείο αυτό και μετά, τις διευθύνσεις που σχετίζονται με την νέα περιοχή

Αυτό είναι κάτι που οδηγεί σε μείωση της απόδοσης και σε μεγαλύτερο βαθμό χρήσης της μνήμης κάτι που αντιμετωπίζεται καταφεύγοντας στην χρήση των συναρτήσεων διαχείρισης της εικονικής μνήμης

Τέλος, η μεταφορά δεδομένων από την μνήμη της ΚΜΕ στην μνήμη της GPU, πραγματοποιείται από την συνάρτηση `cudaMemcpy` η οποία λειτουργεί όπως η συνάρτηση `memcpy` της `C`.

Η καθολική μνήμη που βρίσκεται εκτός του ολοκληρωμένου κυκλώματος της συσκευής, χαρακτηρίζεται από μέγεθος της τάξεως του Gbyte, λανθάνοντα χρόνο της τάξεως των εκατοντάδων κύκλων του ρολογιού και σχετικά περιορισμένο εύρος ζώνης

Εάν δεν επιβληθεί από τον προγραμματιστή καθεστώς αμοιβαίου αποκλεισμού με την χρήση των κατάλληλων συναρτήσεων, είναι πιθανή η εμφάνιση συνθηκών ανταγωνισμού και η ασυνέπεια δεδομένων.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το μοντέλο μνήμης της CUDA

Μνήμη σταθερών Αποτελεί μία **μνήμη** με **μέγιστο μέγεθος** 64 KiB που βρίσκεται **εκτός του ολοκληρωμένου κυκλώματος**, στην οποία οι εφαρμογές αποθηκεύουν **σύμβολα σταθερών** που έχουν **σχετικά μικρή** τιμή.

Είναι μία κρυφή μνήμη **μόνο ανάγνωσης**, που επιτρέπει την **εκπομπή** μιας τιμής που βρίσκεται αποθηκευμένη σε αυτή, σε **όλα τα νήματα ενός στημονιού**, χαρακτηριστικά που καθιστούν αυτή τη μνήμη **ιδιαίτερα ελκυστική** για την **αποθήκευση δεδομένων** που προσπελαύνονται με **μεγάλη συχνότητα** και για τα οποία είναι γνωστό πως **δεν πρόκειται να τροποποιηθούν**.

Εάν απαιτείται **εγγραφή** σε αυτή την μνήμη, αυτή είναι δυνατή **μόνο από την ΚΜΕ** διά της χρήσεως της συνάρτησης **cudaMemcpyToSymbol**. Προκειμένου να υποδηλώσουμε πως μία μεταβλητή θα πρέπει να **αποθηκευτεί** στην **μνήμη σταθερών**, τοποθετούμε μπροστά από την **δήλωσή** της τη δεσμευμένη λέξη **__constant__**.

Μνήμη υφής Σε αυτή την περιοχή μνήμης, αποθηκεύονται **πληροφορίες** σχετικά με το **χαρακτηριστικό της υφής**, το οποίο σχετίζεται με τα **τρισδιάστατα γραφικά**. Αν και σχεδιάστηκε για χρήση με τα περιβάλλοντα της **OpenGL** και του **DirectX**, ωστόσο, μπορεί να χρησιμοποιηθεί και στην περίπτωση της **CUDA**, για την **επιτάχυνση της εκτέλεσης** της παράλληλης εφαρμογής.

Αυτή η μνήμη επιτρέπει την πραγματοποίηση διαδικασιών **μόνο ανάγνωσης**, ενώ **δεν επιτρέπονται** διαδικασίες **εγγραφής**. Η **εμβέλεια** του περιεχομένου της είναι **όλο το πλέγμα**, ενώ ο **χρόνος ζωής** του περιεχομένου της ελέγχεται από την **ΚΜΕ**.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι συναρτήσεις διαχείρισης μνήμης της CUDA

cudaArrayGetInfo	/* Gets info about the specified cudaArray	*/
cudaArrayGetMemoryRequirements	/* Returns the memory requirements of a CUDA array	*/
cudaArrayGetPlane	/* Gets a CUDA array plane from a CUDA array	*/
cudaArrayGetSparseProperties	/* Returns the layout props of a sparse CUDA array	*/
cudaFree	/* Frees memory on the device	*/
cudaFreeArray	/* Frees an array on the device	*/
cudaFreeHost	/* Frees page-locked memory	*/
cudaFreeMipmappedArray	/* Frees a mipmapped array on the device	*/
cudaGetMipmappedArrayLevel	/* Gets a mipmap level of a CUDA mipmapped array	*/
cudaGetSymbolAddress	/* Finds the address associated with a CUDA symbol	*/
cudaGetSymbolSize	/* Finds object size associated with a CUDA symbol	*/
cudaHostAlloc	/* Allocates page-locked memory on the host	*/
cudaHostGetDevicePointer	/* Passes back device pointer of mapped host memory	*/
cudaHostGetFlags	/* Passes back flags for allocate pinned host memory	*/
cudaHostRegister	/* Registers an existing host memory range	*/
cudaHostUnregister	/* Unregisters a memory range	*/
cudaMalloc	/* Allocate memory on the device	*/
cudaMalloc3D	/* Allocates logical 1D, 2D, or 3D memory objects	*/
cudaMalloc3DArray	/* Allocate an array on the device	*/
cudaMallocArray	/* Allocate an array on the device	*/
cudaMallocHost	/* Allocates page-locked memory on the host	*/
cudaMallocManaged	/* Allocates memory that will be managed by the UMS	*/
cudaMallocMipmappedArray	/* Allocate a mipmapped array on the device	*/
cudaMallocPitch	/* Allocates pitched memory on the device	*/



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι συναρτήσεις διαχείρισης μνήμης της CUDA

cudaMemAdvise	/* Advise about the usage of a given memory range	*/
cudaMemAdvise_v2	/* Advise about the usage of a given memory range	*/
cudaMemGetInfo	/* Gets free and total device memory	*/
cudaMemPrefetchAsync	/* Prefetches memory to the specified dest device	*/
cudaMemRangeGetAttribute	/* Query an attribute of a given memory range	*/
cudaMemRangeGetAttributes	/* Query attributes of a given memory range	*/
cudaMemcpy	/* Copies data between host and device	*/
cudaMemcpy2D	/* Copies data between host and device	*/
cudaMemcpy2DArrayToArray	/* Copies data between host and device	*/
cudaMemcpy2DAsync	/* Copies data between host and device	*/
cudaMemcpy2DFromArray	/* Copies data between host and device	*/
cudaMemcpy2DFromArrayAsync	/* Copies data between host and device	*/
cudaMemcpy2DToArray	/* Copies data between host and device	*/
cudaMemcpy2DToArrayAsync	/* Copies data between host and device	*/
cudaMemcpy3D	/* Copies data between 3D objects	*/
cudaMemcpy3DAsync	/* Copies data between 3D objects	*/
cudaMemcpy3DPeer	/* Copies memory between devices	*/
cudaMemcpy3DPeerAsync	/* Copies memory between devices asynchronously	*/
cudaMemcpyAsync	/* Copies data between host and device	*/
cudaMemcpyFromSymbol	/* Copies data from the given symbol on device	*/
cudaMemcpyFromSymbolAsync	/* Copies data from the given symbol on device	*/
cudaMemcpyPeer	/* Copies memory between two devices	*/
cudaMemcpyPeerAsync	/* Copies memory between two devices asynchronously	*/
cudaMemcpyToSymbol	/* Copies data to the given symbol on the device	*/



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι συναρτήσεις διαχείρισης μνήμης της CUDA

```
cudaMemcpyToSymbolAsync    /* Copies data to the given symbol on the device */
cudaMemset                 /* Initializes or sets device memory to a value */
cudaMemset2D               /* Initializes or sets device memory to a value */
cudaMemset2DAsync          /* Initializes or sets device memory to a value */
cudaMemset3D               /* Initializes or sets device memory to a value */
cudaMemset3DAsync          /* Initializes or sets device memory to a value */
cudaMemsetAsync            /* Initializes or sets device memory to a value */

/* Returns the memory requirements of a CUDA mipmapped array */
cudaMipmappedArrayGetMemoryRequirements

/* Returns the layout properties of a sparse CUDA mipmapped array */
cudaMipmappedArrayGetSparseProperties

make_cudaExtent            /* Returns a cudaExtent based on input params */
make_cudaPitchedPtr        /* Returns a cudaPitchedPtr based on input params */
make_cudaPos               /* Returns a cudaPos based on input params */
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Προηγμένες τεχνικές διαχείρισης μνήμης

Η **CUDA** προσφέρει την δυνατότητα να **κλειδώσουμε** μία **περιοχή μνήμης** στην μνήμη της **KME** έτσι ώστε αυτή να **παραμένει μόνιμα** στην κεντρική μνήμη, **χωρίς να υφίσταται η δυνατότητα** μετακίνησής της από και προς την εικονική μνήμη.

Τα εμπλεκόμενα τμήματα μνήμης συνήθως περιέχουν **κρίσιμο κώδικα** για την **λειτουργία** της εφαρμογής, ενώ η χρήση αυτού του μηχανισμού, που πραγματοποιείται από το **πρόγραμμα οδήγησης** της **NVIDIA** χωρίς την **παρέμβαση** του προγραμματιστή, είναι **απολύτως αναγκαία** και συνιστά **προϋπόθεση** για την υλοποίηση διαδικασιών **άμεσης πρόσβασης** στη μνήμη (**DMA**).

Ο μηχανισμός κλειδώματος υλοποιείται είτε μέσω της κλήσης **mlock** μετά από την δέσμευση της μνήμης με την **malloc** είτε μέσω της **κλήσης** της συνάρτησης

```
__host__ cudaError_t cudaMallocHost (void** ptr, size_t size);
```

ενώ η **απελευθέρωση** αυτής της μνήμης πραγματοποιείται από την συνάρτηση **cudaFreeHost**

Η **υπερβολική χρήση** αυτού του μηχανισμού ενδέχεται να επιφέρει **μείωση απόδοσης**, λόγω της **αδυναμίας** του **λειτουργικού συστήματος** να μεταφέρει αυτή την περιοχή μνήμης στην **εικονική μνήμη**, αφού αυτή είναι **κλειδωμένη** και δεν **μετακινείται**.

Ωστόσο, η **σωστή χρήση** του, επιτρέπει την **χρονική επικάλυψη** ανάμεσα στις **διαδικασίες μεταφοράς δεδομένων** και στην εκτέλεση ενός **δεύτερου πυρήνα**, ενώ οδηγεί σε αύξηση του εύρους ζώνης.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Προηγμένες τεχνικές διαχείρισης μνήμης

Σε αυτόν τον τύπο διαχείρισης χρησιμοποιούνται **για κάθε** περιοχή μνήμης **δύο** διευθύνσεις, μία διεύθυνση για την **πρόσβαση** αυτής της περιοχής μνήμης από την **KME** και μία διεύθυνση για την **πρόσβαση** αυτής της περιοχής μνήμης από την **GPU**

Η **δέσμευση** και η **αποδέσμευση** μιας περιοχής που χρησιμοποιείται με αυτόν τον τρόπο, πραγματοποιείται από τις συναρτήσεις **cudaHostAlloc** και **cudaFreeHost**, ενώ η **ανάκτηση** της διεύθυνσης στην μνήμη της συσκευής που χρησιμοποιείται για την **προσπέλαση** αυτής της μνήμης από τον πυρήνα, πραγματοποιείται από την συνάρτηση **cudaHostDevicePointer**.

Στα λειτουργικά **Windows** και **Linux** των **64 bits** και για συσκευές με **υπολογιστική ικανότητα $\geq 6.x$** , η χρήση αυτού του δείκτη **δεν είναι** αναγκαία, αφού η χρήση του χώρου της **ενοποιημένης εικονικής διευθυνσιοδότησης** που χρησιμοποιεί **εικονικές διευθύνσεις** των **49 bits** καθιστά **περιττή** την διάκριση ανάμεσα σε αυτούς τους δείκτες, αφού αμφότεροι έχουν την **ίδια τιμή**.

Το χαρακτηριστικό της **ενοποιημένης μνήμης**, καταργεί τη **διάκριση** ανάμεσα στην **μνήμη της KME** και στη **μνήμη της GPU** και επιτρέπει την χρήση **πινάκων δεδομένων** οι οποίοι να είναι **προσπελάσιμοι** τόσο από την **KME** όσο και από την **GPU**.

Αυτό το χαρακτηριστικό χρησιμοποιείται είτε γράφοντας πριν από την δήλωση μιας μεταβλητής την δεσμευμένη λέξη **__managed__**, είτε μέσω της συνάρτησης

```
__host__ cudaError_t cudaMallocManaged (void** devPtr, size_t size,  
                                         unsigned int flags = cudaMemAttachGlobal);
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Προηγμένες τεχνικές διαχείρισης μνήμης

Η **CUDA** επιτρέπει την **χρονική διάταξη** των διαδικασιών **δέσμευσης** και **αποδέσμευσης** μνήμης, μέσω του μηχανισμού **SOMA (Stream Ordered Memory Allocation)**, κάτι που οδηγεί σε πιο **αποτελεσματική επαναχρησιμοποίηση** μνήμης και **διευκολύνει** την ανάπτυξη **πολιτικών διαχείρισης μνήμης υψηλής απόδοσης**. Οι **εμπλεκόμενες** συναρτήσεις ακολουθούν στη συνέχεια

<code>cudaFreeAsync</code>	<code>/* Frees memory with stream ordered semantics</code>	<code>*/</code>
<code>cudaMallocAsync</code>	<code>/* Allocates memory with stream ordered semantics</code>	<code>*/</code>
<code>cudaMallocFromPoolAsync</code>	<code>/* Allocates memory from a specified pool with stream ordered semantics</code>	<code>*/</code>
<code>cudaMemPoolCreate</code>	<code>/* Creates a memory pool</code>	<code>*/</code>
<code>cudaMemPoolDestroy</code>	<code>/* Destroys the specified memory pool</code>	<code>*/</code>
<code>cudaMemPoolExportPointer</code>	<code>/* Export data to share a memory pool allocation between processes</code>	<code>*/</code>
<code>cudaMemPoolExportToShareableHandle</code>	<code>/* Exports a memory pool to the requested handle type</code>	<code>*/</code>
<code>cudaMemPoolGetAccess</code>	<code>/* Returns the accessibility of a pool from a device</code>	<code>*/</code>
<code>cudaMemPoolGetAttribute</code>	<code>/* Gets attributes of a memory pool</code>	<code>*/</code>
<code>cudaMemPoolImportFromShareableHandle</code>	<code>/* Imports a memory pool from a shared handle</code>	<code>*/</code>
<code>cudaMemPoolImportPointer</code>	<code>/* Import a memory pool allocation from another process</code>	<code>*/</code>
<code>cudaMemPoolSetAccess</code>	<code>/* Controls visibility of pools between devices</code>	<code>*/</code>
<code>cudaMemPoolSetAttribute</code>	<code>/* Sets attributes of a memory pool</code>	<code>*/</code>
<code>cudaMemPoolTrimTo</code>	<code>/* Tries to release memory back to the OS</code>	<code>*/</code>

Στον **μηχανισμό SOMA**, η μνήμη που δεσμεύεται, περιλαμβάνεται στα **διαθέσιμα αποθέματα μνήμης**, τα οποία περιγράφουν τόσο την **πραγματική φυσική μνήμη**, όσο και τις **διευθύνσεις της εικονικής μνήμης** και τα οποία **προσπελούνται** και **χρησιμοποιούνται** σύμφωνα με τον τρόπο που υπαγορεύεται από ένα **κατάλληλα οριζόμενο σύνολο ιδιοτήτων**.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Προηγμένες τεχνικές διαχείρισης μνήμης

Η προεπιλεγμένη συμπεριφορά είναι η ελαχιστοποίηση του διαθέσιμου αποθέματος, ενώ για να ελαχιστοποιηθεί το πλήθος των σχετιζόμενων κλήσεων συστήματος, ορίζεται ένα αποτύπωμα μνήμης για το καθένα από τα διαθέσιμα αποθέματα που εκφράζει την ποσότητα της μνήμης σε bytes που θα πρέπει να περιέχεται στο απόθεμα, πριν την απελευθέρωσή της.

Στα πλαίσια εξυπηρέτησης ενός αιτήματος δέσμευσης μνήμης, το σύστημα, αρχικά θα προσπαθήσει να επαναχρησιμοποιήσει την μνήμη που είχε απελευθερωθεί στο παρελθόν μέσω της συνάρτησης `cudaFreeAsync`, πριν απευθυνθεί στο λειτουργικό.

Ο μηχανισμός `SOMA` προσφέρει πολιτικές επαναχρησιμοποίησης μνήμης, τα χαρακτηριστικά των οποίων μπορούν να ελεγχθούν με τη βοήθεια των ιδιοτήτων

`cudaMemPoolResuseFollowEventDependencies`
`cudaMemPoolResuseAllowOpportunistic`
`cudaMemPoolResuseAllowInternalDependencies`

Ο διαμοιρασμός των πόρων μνήμης στις διεργασίες που εκτελούνται στο σύστημα, περιλαμβάνει δύο φάσεις

- η πρώτη φάση σχετίζεται με την χορήγηση πρόσβασης στους πόρους μνήμης
- η δεύτερη φάση αφορά στον καθορισμό της περιοχής διευθύνσεων που θα χρησιμοποιηθούν από κάθε διεργασία, καθώς και της χρονικής στιγμής πραγματοποίησης των αναγκαίων διαδικασιών απεικόνισης.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Διαχείριση εικονικής μνήμης

Οι συναρτήσεις διαχείρισης εικονικής μνήμης προσφέρουν την δυνατότητα άμεσου χειρισμού του ενοποιημένου χώρου διευθύνσεων μνήμης, για την απεικόνιση φυσικών σε εικονικές διευθύνσεις προσπελάσιμες από την GPU αλλά και αλληλεπίδρασης με άλλες διεργασίες αλλά και βιβλιοθήκες γραφικών, όπως είναι οι Vulkan και OpenGL.

Το βασικό χαρακτηριστικό αυτών των συναρτήσεων, είναι η άρση της συσχέτισης που υφίσταται ανάμεσα στις έννοιες της διεύθυνσης και της μνήμης, επιτρέποντας στην εφαρμογή να τις διαχειριστεί ξεχωριστά και ανεξάρτητα τη μία από την άλλη.

Αυτές οι συναρτήσεις επιτρέπουν τον ορισμό και την κατάργηση απεικονίσεων ανάμεσα στην μνήμη και στην περιοχή των εικονικών διευθύνσεων, δια της εφαρμογής μιας διαδικασίας τεσσάρων βημάτων, που περιλαμβάνει

- την δέσμευση μιας περιοχής διευθύνσεων στην φυσική μνήμη
- την δέσμευση μιας περιοχής εικονικών διευθύνσεων μνήμης
- την απεικόνιση της φυσικής μνήμης που έχει δεσμευτεί, στην περιοχή των εικονικών διευθύνσεων
- την χορήγηση και τον έλεγχο των κατάλληλων δικαιωμάτων πρόσβασης σε αυτήν την περιοχή.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι συναρτήσεις διαχείρισης εικονικής μνήμης του περιβάλλοντος της CUDA

cuMemAddressFree	/* Free an address range reservation	*/
cuMemAddressReserve	/* Allocate an address range reservation	*/
cuMemCreate	/* Create a CUDA memory handle for a memory allocation	*/
cuMemExportToShareableHandle	/* Exports an allocation to a requested shareable handle type	*/
cuMemGetAccess	/* Get the access flags set for the given location and ptr	*/
cuMemGetAllocationGranularity	/* Calculates either the minimal or recommended granularity	*/
cuMemGetAllocationPropertiesFromHandle	/* Retrieve the contents of the structure with the handle properties	*/
cuMemImportFromShareableHandle	/* Imports an allocation from a requested shareable handle type	*/
cuMemMap	/* Maps an allocation handle to a reserved virtual address range	*/
cuMemMapArrayAsync	/* Maps or unmaps subregions of sparse CUDA arrays and mipmapped arrays	*/
cuMemRelease	/* Release a memory handle for a memory allocation	*/
cuMemRetainAllocationHandle	/* Returns the allocation handle of the backing memory allocation	*/
cuMemSetAccess	/* Set the access flags for each location for the given address range	*/
cuMemUnmap	/* Unmap the backing memory of a given address range	*/

Ο έλεγχος της υποστήριξης ή όχι αυτής της δυνατότητας στηρίζεται στην χρήση της συνάρτησης

```
CUresult result = cuDeviceGetAttribute(&deviceSupportsVmm,  
                                         CU_DEVICE_ATTRIBUTE_VIRTUAL_MEMORY_MANAGEMENT_SUPPORTED, device);
```

με την συσκευή να προσφέρει αυτή την υποστήριξη, εάν η τιμή της ακέραιας μεταβλητής **deviceSupportsVmm** που επιστρέφεται από την συνάρτηση, είναι **διάφορη** του μηδενός



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Ανάπτυξη εφαρμογών στο περιβάλλον της CUDA

Τα **βασικά βήματα** που ακολουθούνται κατά την ανάπτυξη μιας **εφαρμογής** στο **περιβάλλον της CUDA**, είναι τα εξής:

1. Προσδιορίζονται οι **παράμετροι διαμόρφωσης** της συσκευής (**πλήθος** και **χαρακτηριστικά GPUs**)
2. Προσδιορίζονται οι **συναρτήσεις** που θα εκτελεστούν στην **KME**, στην **GPU** ή σε **αμφότερες** αυτές τις διατάξεις.
3. Προσδιορίζονται οι τιμές των δεικτών **blockIdx** και **threadIdx** για το κάθε νήμα
4. **Δεσμεύεται** η αναγκαία ποσότητα μνήμης στην **KME** και στην **GPU** με τις συναρτήσεις **malloc** και **cudaMalloc**
5. Πραγματοποιείται η **μεταφορά των κατάλληλων δεδομένων** στα νήματα με την συνάρτηση **cudaMemcpy**.
6. **Εκτελείται** μία **συνάρτηση πυρήνα** εφοδιασμένη με τις κατάλληλες πληροφορίες
7. Επιβάλλεται **συγχρονισμός τύπου χρονικής φραγής** διά της χρήσεως της συνάρτησης **cudaThreadSynchronize**
8. **Αποδεσμεύονται** οι πόροι που δεν χρειάζονται πλέον

Η επιλογή των **βέλτιστων τιμών** για τις **διαστάσεις** του **πλέγματος** και το **πλήθος** των **νημάτων** της κάθε ομάδας στο **Βήμα 6**, είναι θέμα **εμπειρίας** και απαιτεί **έντονο πειραματισμό** και **μελέτη του χρόνου εκτέλεσης** και της **απόδοσης**

Ο **συγχρονισμός** των **νημάτων** στο **Βήμα 7** είναι **αναγκαίος** διότι το κάθε νήμα **ανάλογα με την ταυτότητά του** εκτελεί **διαφορετικά τμήματα κώδικα** και χρησιμοποιεί **διαφορετικά δεδομένα** με αποτέλεσμα την **άρση** του συγχρονισμού.

Επιπλέον, το **πλήθος των νημάτων** συνήθως είναι **μεγαλύτερο από το πλήθος των πυρήνων**, με αποτέλεσμα την **αναμονή** για τη **εκτέλεσή τους**, ενώ κάποιες άλλες φορές τα νήματα **δημιουργούνται** αλλά **δεν χρησιμοποιούνται**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Ανάπτυξη εφαρμογών στο περιβάλλον της CUDA

```
#include <stdlib.h>

/* Allocates the requested memory on the host machine and returns a pointer to it */
void *malloc (size_t size);

/* Deallocates the memory previously allocated by a call to calloc, malloc, or realloc */
void free (void *ptr);

#include <cuda_runtime.h>

/* Allocates memory on the device */
__host__ __device__ cudaError_t cudaMalloc (void **devPtr, size_t size);

/* Frees memory on the device */
__host__ __device__ cudaError_t cudaFree (void *devPtr);

/* Copies data between host and device */
__host__ cudaError_t cudaMemcpy (void *dst, const void *src, size_t count,
                                cudaMemcpyKind kind);

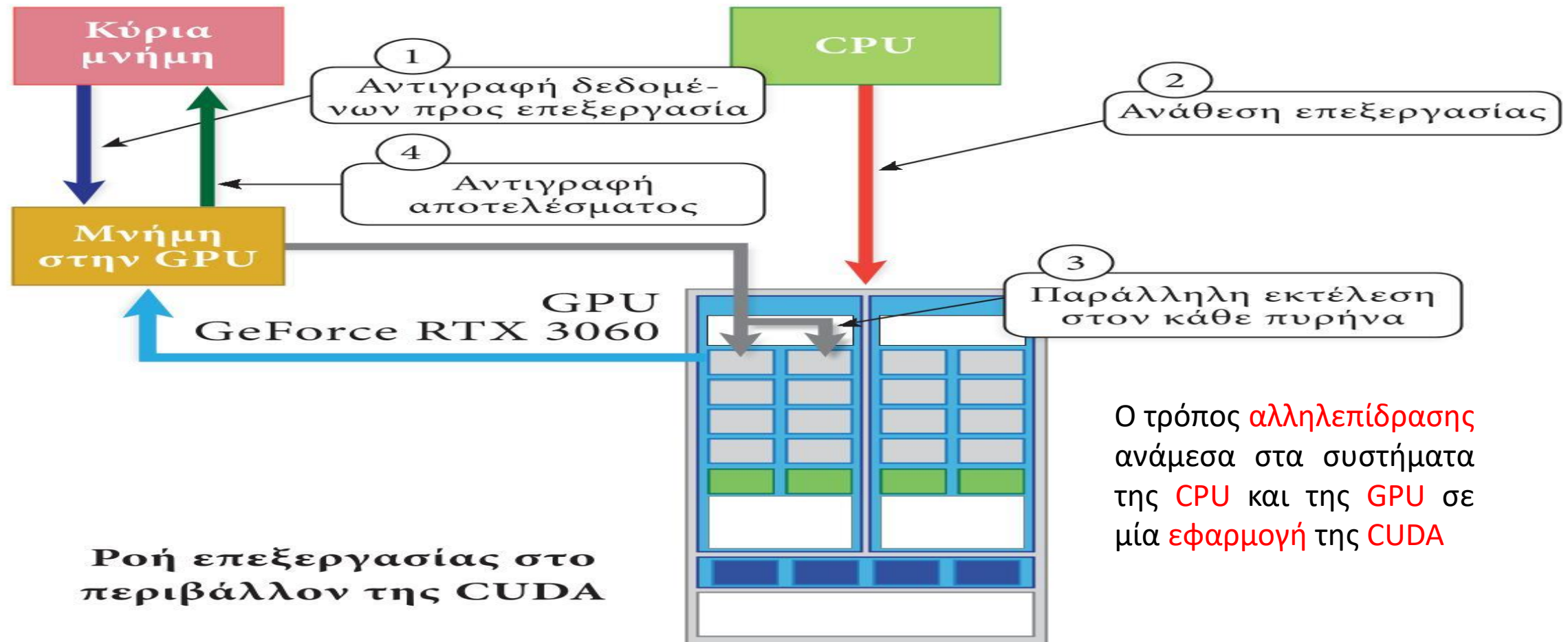
enum cudaMemcpyKind {
    cudaMemcpyHostToHost = 0,      /* Direction of copy: Host -> Host */
    cudaMemcpyHostToDevice = 1,   /* Direction of copy: Host -> Device */
    cudaMemcpyDeviceToHost = 2,   /* Direction of copy: Device -> Host */
    cudaMemcpyDeviceToDevice = 3, /* Direction of copy: Device -> Device */
    cudaMemcpyDefault = 4 }      /* Direction is inferred from pointer values */
```

Τα **πρωτότυπα** των συναρτήσεων που εμπλέκονται στην **ανταλλαγή δεδομένων** ανάμεσα στην **KME** και στην **GPU**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Ανάπτυξη εφαρμογών στο περιβάλλον της CUDA



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η διαδικασία της μεταγλώττισης στην CUDA

Η ανάπτυξη εφαρμογών CUDA μπορεί να πραγματοποιηθεί σε αρκετές γλώσσες προγραμματισμού όπως C, C++ και Python

Εναλλακτικά μπορούμε να καταφύγουμε στην αρχιτεκτονική συνόλου εντολών της CUDA (Parallel Thread Execution, PTX)

Στην περίπτωση της C θα πρέπει να χρησιμοποιήσουμε τα αρχεία επικεφαλίδας

- `cuda.h` που ορίζει τις συναρτήσεις και τους τύπους δεδομένων του κεντρικού υπολογιστή για την βιβλιοθήκη συναρτήσεων (API) του οδηγού της CUDA
- `cuda_runtime_api.h` που ορίζει τις ίδιες πληροφορίες για την βιβλιοθήκη συναρτήσεων του περιβάλλοντος χρόνου εκτέλεσης της CUDA (CUDA Runtime API)
- `cuda_runtime.h` που περιέχει οτιδήποτε υπάρχει στο αρχείο `cuda_runtime_api.h`, καθώς και πρόσθετες πληροφορίες που αφορούν στις επεκτάσεις της γλώσσας.

Η CUDA προσφέρει δύο βιβλιοθήκες συναρτήσεων ανάπτυξης παράλληλων εφαρμογών και πιο συγκεκριμένα

- μία βιβλιοθήκη χαμηλού επιπέδου (Driver API) που επιτρέπει την προσπέλαση εξειδικευμένων χαρακτηριστικών και η οποία μαζί με την υπολογιστική ικανότητα της συσκευής, υπαγορεύουν το σύνολο των δυνατοτήτων και των εργαλείων
- μία βιβλιοθήκη υψηλού επιπέδου (CUDA runtime) που είναι και αυτή που χρησιμοποιείται συνήθως στην πράξη.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η διαδικασία της μεταγλώττισης στην CUDA

Αυτή η **δεύτερη** βιβλιοθήκη, προσφέρει **συναρτήσεις** οι οποίες εκτελούνται στον **κεντρικό υπολογιστή**, για την πραγματοποίηση διαδικασιών, όπως είναι η **δέσμευση** και η **αποδέσμευση μνήμης** στην συσκευή, η **ανταλλαγή δεδομένων** ανάμεσα στην μνήμη του **κεντρικού υπολογιστή** και στην μνήμη της **συσκευής**, η διαχείριση συστημάτων με **πολλαπλές συσκευές**, κ.λ.π.

Στην **CUDA 12.0** και στις **μεταγενέστερες** εκδόσεις, αυτό το **περιβάλλον χρόνου εκτέλεσης** θα πρέπει να **αρχικοποιηθεί** για την συσκευή ενδιαφέροντος, κάτι που γίνεται με τις συναρτήσεις **cudaInitDevice** και **cudaSetDevice**

Η **έκδοση** των βιβλιοθηκών **CUDA Driver API** και **CUDA Runtime API** που είναι **εγκατεστημένες** στον υπολογιστή μας, προσδιορίζονται από τις σταθερές **CUDA_VERSION** και **CUDART_VERSION** με τιμή π.χ **11020** για την Έκδοση **11.2** της CUDA

Το **CUDA Driver API** χαρακτηρίζεται από **συμβατότητα προς τα πίσω**, όχι όμως και από **συμβατότητα προς τα εμπρός**

Επομένως, η **έκδοση** που θα **επιλέξουμε να χρησιμοποιήσουμε**, θα πρέπει να είναι **>=** της **μέγιστης έκδοσης** του **Driver API**, σύμφωνα με την οποία **μεταγλωττίστηκαν** οι **εφαρμογές** και οι **βιβλιοθήκες της CUDA** που χρησιμοποιούνται στο σύστημα.

Επιπλέον, όλες οι **πρόσθετες λειτουργίες** και οι **πάσης φύσεως βιβλιοθήκες** μίας εφαρμογής, θα πρέπει να χρησιμοποιούν **την ίδια έκδοση της βιβλιοθήκης CUDA Runtime**, εκτός εάν συνδεθούν στο εκτελέσιμο αρχείο με στατικό τρόπο.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι τύποι των αρχείων που χρησιμοποιούνται σε μία εφαρμογή CUDA

- **Αρχεία .cu:** Αρχεία **πηγαίου κώδικα** των **πυρήνων** που εκτελούνται στην **GPU** με κώδικα σε **C** και **CUDA**
- **Αρχεία .c και .cpp:** Αρχεία **πηγαίου κώδικα** στις γλώσσες **C** και **C++**, αντίστοιχα.
- **Αρχεία .h και .cuh:** Αρχεία **επικεφαλίδας** για συναρτήσεις της **C** και της **CUDA**.
- **Αρχεία .o (Unix / Linux) ή .obj (Windows):** Αρχεία **αντικειμενικού κώδικα**
- **Αρχεία .a (Linux) ή .lib (Windows):** Αρχεία **στατικών βιβλιοθηκών**
- **Αρχεία .so (Linux) ή .dll (Windows):** Αρχεία **δυναμικών βιβλιοθηκών**
- **Αρχεία .cubin:** Αρχεία **δυαδικού κώδικα** της **CUDA**, για **εκτέλεση** σε **συγκεκριμένη GPU**
- **Αρχεία .ptx:** Αρχεία κώδικα σε μία **ενδιάμεση γλώσσα Assembly**, για τις διατάξεις **GPU** της **NVIDIA**
- **Αρχεία .fatbin:** Αρχεία **δυαδικού κώδικα** που επιτρέπουν την χρήση **πολλαπλών αρχείων PTX** και **CUBIN**

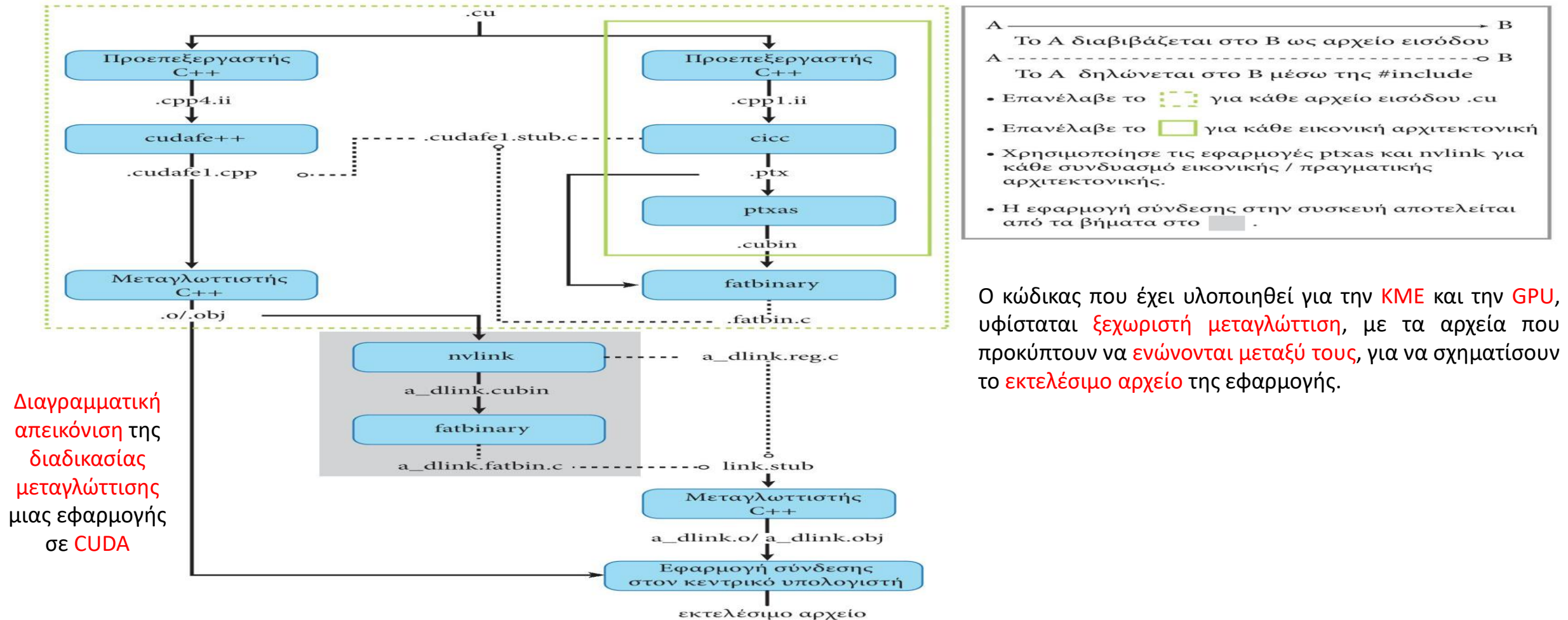
Η δημιουργία του **εκτελέσιμου αρχείου** μιας εφαρμογής σε **CUDA**, πραγματοποιείται με τον **μεταγλωττιστή nvcc**, που στην πραγματικότητα αποτελεί **αρχείο σεναρίου** που **αρχικοποιεί** τις κατάλληλες **μεταβλητές περιβάλλοντος** στις **κατάλληλες** τιμές τους και στη συνέχεια καλεί τον **μεταγλωττιστή gcc** με τους **κατάλληλους διακόπτες**.

Η χρήση αυτού του **βοηθητικού αρχείου**, **διευκολύνει σημαντικά** την διαδικασία της **μεταγλώττισης**, αφού **απαλλάσσει** τον προγραμματιστή από την πραγματοποίηση των κατάλληλων **διαδικασιών διαμόρφωσης**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Τα στάδια της μεταγλώττισης

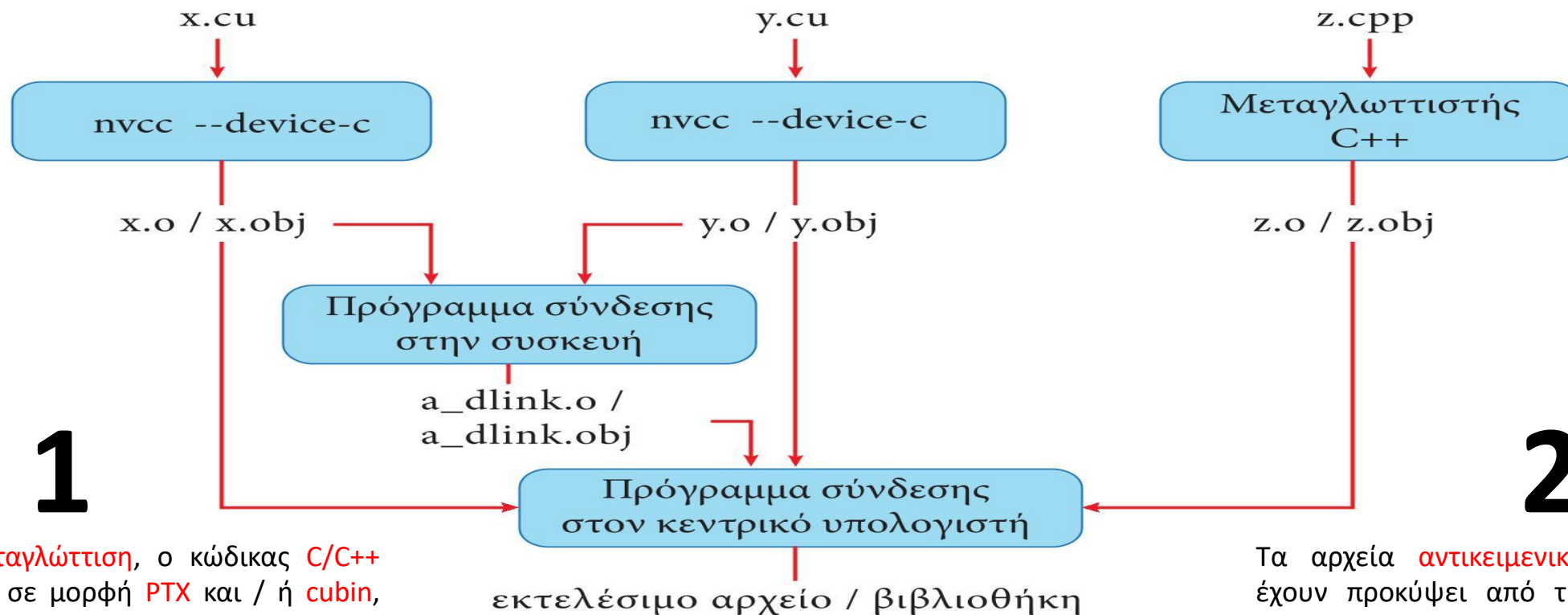


Διαγραμματική απεικόνιση της διαδικασίας μεταγλώττισης μιας εφαρμογής σε CUDA

Ο κώδικας που έχει υλοποιηθεί για την KME και την GPU, υφίσταται ξεχωριστή μεταγλώττιση, με τα αρχεία που προκύπτουν να ενώνονται μεταξύ τους, για να σχηματίσουν το εκτελέσιμο αρχείο της εφαρμογής.

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Τα στάδια της μεταγλώττισης



1

2

Κατά την **μεταγλώττιση**, ο κώδικας **C/C++** μετατρέπεται σε μορφή **PTX** και / ή **cubin**, ενώ η σύνταξη **<<<...>>>** αντικαθίσταται από μία **συνάρτηση χρόνου εκτέλεσης**, έτσι ώστε να καταστεί δυνατή η **φόρτωση** και η **εκτέλεση** κάθε **μεταγλωττισμένου** πυρήνα

Η διαδικασία **ξεχωριστής μεταγλώττισης** μιας **εφαρμογής** σε **CUDA**

Τα αρχεία **αντικειμενικού κώδικα** που έχουν προκύψει από την **μεταγλώττιση** των **αρχείων .cu** συνενώνονται μέσω μιας εφαρμογής **σύνδεσης** που αφορά στην **διάταξη GPU**, με την **τελική σύνδεση** να πραγματοποιείται από το **πρόγραμμα σύνδεσης** του **κεντρικού υπολογιστή**

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

```
#include <stdio.h>

__global__ void cuda_hello(){
    printf("Hello World from GPU!\n"); }

int main() {
    cuda_hello <<<1,10>>> ();
    return 0; }
```

Το κάθε νήμα εκτυπώνει το μήνυμα **Hello World!**.

Η συνάρτηση **cuda_hello** αποτελεί **συνάρτηση πυρήνα**, η οποία θα κληθεί μέσα από την **main** που θα εκτελεστεί στην ΚΜΕ, προκειμένου να εκτελεστεί στην **GPU**.

Δηλώνεται ως **void** γιατί **δεν μπορεί να επιστρέψει** τιμή

Η συμβολοσειρά **<<<1,10>>>**, οδηγεί στην δημιουργία ενός **μονοδιάστατου πλέγματος** με **μία και μοναδική** ομάδα νημάτων, η οποία περιλαμβάνει 10 νήματα.

```
D:\VBoxSharedFolder\CUDA>nvcc -c hello.cu
hello.cu

D:\VBoxSharedFolder\CUDA>nvcc -o hello hello.obj
hello.obj
    Creating library hello.lib and object hello.exp

D:\VBoxSharedFolder\CUDA>hello
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!
Hello World from GPU!

D:\VBoxSharedFolder\CUDA>
```



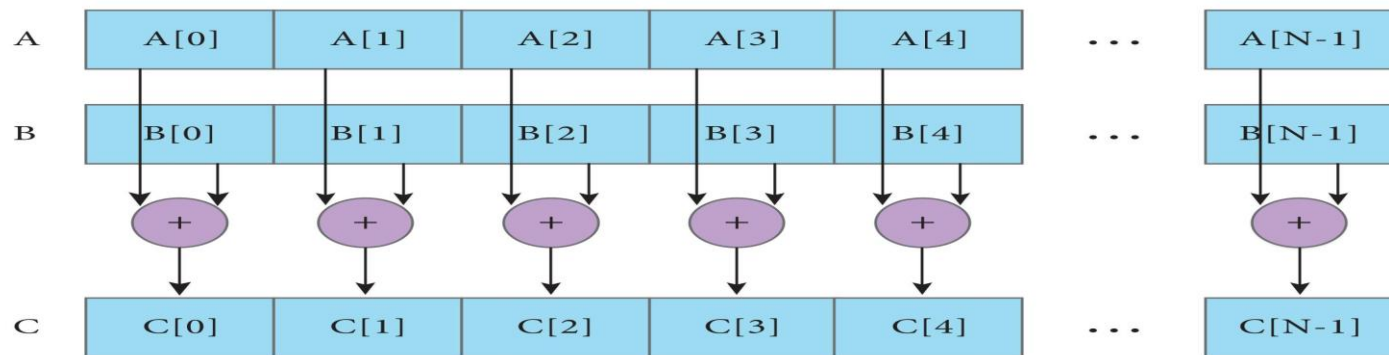
Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Παράλληλη πρόσθεση διανυσμάτων Η διαδικασία της **πρόσθεσης** δύο **διανυσμάτων** **a** και **b**, μήκους **N** στοιχείων που οδηγεί σε ένα **τρίτο** διάνυσμα **c** του **ιδίου μήκους**, επιδέχεται **πλήρη παραλληλισμό**, αφού χαρακτηρίζεται από **απουσία εξαρτήσεων**.

Η **σειριακή έκδοση** του κώδικα αυτής της εφαρμογής, σε **γλώσσα C**, υλοποιείται μέσω ενός **απλού βρόχου** της μορφής

```
for (i=0;i<N;i++) c[i] = a[i] + b[i];
```



Η διαδικασία της **πρόσθεσης** δύο διανυσμάτων οδηγεί στην δημιουργία ενός **τρίτου διανύσματος** το κάθε στοιχείο του οποίου προκύπτει **από το άθροισμα** των **αντίστοιχων** στοιχείων των δύο διανυσμάτων

Ο **πιο απλός τύπος** παραλληλοποίησης σε **CUDA** συνίσταται στον ορισμό ενός **μονοδιάστατου πλέγματος** **N νημάτων** το καθένα των οποίων πραγματοποιεί **τον υπολογισμό της τιμής ενός μοναδικού στοιχείου του διανύσματος c** εκτελούμενο **ταυτόχρονα** με τα υπόλοιπα, στον **δικό του πυρήνα CUDA** του πολυεπεξεργαστή (αν και γενικά το **κάθε** νήμα υπολογίζει **ομάδα στοιχείων**).

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Λαμβάνοντας υπόψη, πως στο πρόβλημα εμπλέκονται **τρία** διανύσματα **a**, **b** και **c** και πως η διαδικασία της **πρόσθεσης διανυσμάτων** θα πραγματοποιηθεί στην **GPU**, απαιτείται η χρήση **έξι συνολικά διανυσμάτων** (**τρία** για την **KME** και **τρία** για την **GPU**) τα στοιχεία των οποίων αρχικοποιούνται σε **τυχαίες πραγματικές τιμές** (τύπου **double**), στην περιοχή τιμών **0** έως **MAXVAL**

Η **δέσμευση μνήμης** για τα **τρία διανύσματα** της **KME** πραγματοποιείται ως

```
cpuA = (double*)malloc(Length*sizeof(double));  
cpuB = (double*)malloc(Length*sizeof(double));  
cpuC = (double*)malloc(Length*sizeof(double));
```

ενώ αντίστοιχα, για τα **τρία διανύσματα** της **GPU** θα έχουμε

```
cudaMalloc(&gpuA, Length*sizeof(double));  
cudaMalloc(&gpuB, Length*sizeof(double));  
cudaMalloc(&gpuC, Length*sizeof(double));
```

Η αρχικοποίηση των τριών διανυσμάτων της **KME** πραγματοποιείται ως

```
for (int i=0;i<Length;i++) {  
    cpuA[i] = MAXVAL*((double)rand()/RAND_MAX);  
    cpuB[i] = MAXVAL*((double)rand()/RAND_MAX); }  
}
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

ενώ η μεταφορά των περιεχομένων των διανυσμάτων `cpuA` και `cpuB` στην GPU για να προστεθούν γίνεται με τις εντολές

```
cudaMemcpy(gpuA, cpuA, Length*sizeof(double), cudaMemcpyHostToDevice);  
cudaMemcpy(gpuB, cpuB, Length*sizeof(double), cudaMemcpyHostToDevice);
```

Ο κώδικας της συνάρτησης του πυρήνα έχει τη μορφή

```
__global__ void addVector (double *a, double *b, double *c) {  
    int threadId = blockDim.x * blockIdx.x + threadIdx.x;  
    if (threadId < Length)  
        c[threadId] = a[threadId] + b[threadId]; }
```

ενώ η συνάρτηση του πυρήνα καλείται ως

```
addVector <<< blocksInGrid, TPB >>>(gpuA, gpuB, gpuC);
```

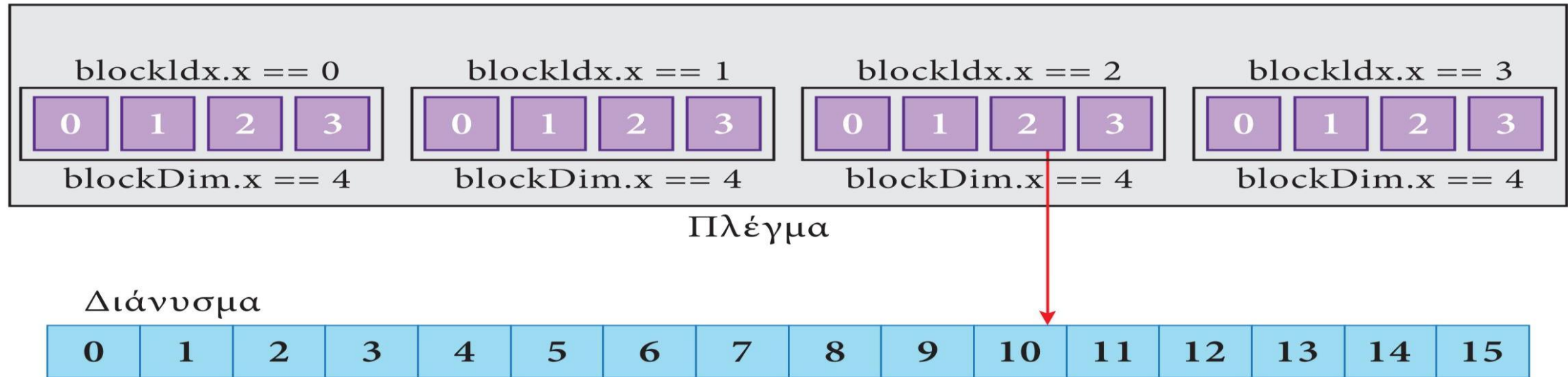
όπου `blocksInGrid = Length / TPB` για τιμές `Length = 1048576` και `TPB = 256`. Επομένως το κάθε νήμα υπολογίζει την τιμή του υπ' αριθμόν `threadId` στοιχείου του πίνακα γινομένου, όπου

```
threadId = blockDim.x * blockIdx.x + threadIdx.x
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA



Η συσχέτιση ανάμεσα στα **νήματα της εφαρμογής** και στα **στοιχεία των διανυσμάτων** που **προστίθενται**

Μετά την **ολοκλήρωση** της διαδικασίας, το περιεχόμενο της **περιοχής μνήμης gpuC** της διάταξης **GPU** που περιέχει το **ζητούμενο αποτέλεσμα**, θα πρέπει να μεταφερθεί στην **περιοχή μνήμης cpuC** που βρίσκεται στην μνήμη της **CPU** προκειμένου το αποτέλεσμα να καταστεί **διαθέσιμο** στον κεντρικό επεξεργαστή, κάτι που γίνεται με τη συνάρτηση

```
cudaMemcpy(cpuC, gpuC, Length*sizeof(double), cudaMemcpyDeviceToHost);
```

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

```
#include <stdio.h>
#include <stdlib.h>
```

Ο πηγαίος κώδικας της εφαρμογής πρόσθεσης διανυσμάτων σε CUDA

```
#define Length 1048576 /* This is the vectors size */ /* Init vectors cpuA and cpuB in random values */
#define MAXVAL 10000 /* This is the maximum data value */ /* in the interval [0->MAXVAL-1] */
#define TPB 256 /* The number of threads per block */ for (int i=0;i<Length;i++) {
    cpuA[i] = MAXVAL*((double)rand()/RAND_MAX);
    cpuB[i] = MAXVAL*((double)rand()/RAND_MAX); }

/* The source code of the kernel for the vector addition */
__global__ void addVector (double *a, double *b, double *c) {
    int threadId = blockDim.x * blockIdx.x + threadIdx.x;
    if (threadId < Length)
        c[threadId] = a[threadId] + b[threadId]; }

/* The main program running at host machine */
int main() {
    int blocksInGrid = ceil(float(Length)/TPB);
    double *cpuA, *cpuB, *cpuC; /* Vectors on host */
    double *gpuA, *gpuB, *gpuC; /* Vectors on device */
    // Allocate memory for vectors A, B, and C on host
    cpuA = (double*)malloc(Length*sizeof(double));
    cpuB = (double*)malloc(Length*sizeof(double));
    cpuC = (double*)malloc(Length*sizeof(double));
    // Allocate memory for vectors gpuA, gpuB and gpuC
    cudaMalloc(&gpuA, Length*sizeof(double));
    cudaMalloc(&gpuB, Length*sizeof(double));
    cudaMalloc(&gpuC, Length*sizeof(double));

    /* Copy data from host vectors cpuA and cpuB */
    /* to device vectors gpuA and gpuB */
    cudaMemcpy(gpuA, cpuA, Length*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(gpuB, cpuB, Length*sizeof(double),
               cudaMemcpyHostToDevice);

    // Launch kernel in GPU to perform vector addition
    addVector <<< blocksInGrid, TPB >>>(gpuA, gpuB, gpuC);
    // Copy data from device vector gpuC to host vector cpuC
    cudaMemcpy(cpuC, gpuC, Length*sizeof(double),
               cudaMemcpyDeviceToHost);

    // Free memory in host device
    free(cpuA); free(cpuB); free(cpuC);
    // Free memory in GPU device
    cudaFree(gpuA); cudaFree(gpuB); cudaFree(gpuC);
    return 0; }
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Το **τίμημα που πληρώνουμε** για να **εκμεταλλευτούμε** τις **υπολογιστικές δυνατότητες** της **CUDA**, είναι η **δημιουργία αντιγράφων** πινάκων δεδομένων τόσο για την **KME** όσο και για την **GPU** καθώς και η **μεταφορά δεδομένων** ανάμεσα σε δύο αυτές μνήμες, διαδικασίες, οι οποίες δεν απαιτούνται για την **εκτέλεση σειριακού κώδικα** στην **KME**

Ωστόσο, εάν ο κώδικας έχει αναπτυχθεί **έξυπνα** και **σωστά**, η επιτάχυνση που προκύπτει είναι **πάρα πολύ μεγάλη** και το κέρδος που αποκομίζουμε από την **παραλληλοποίηση**, είναι **πολύ μεγαλύτερο** από το **κόστος** των παραπάνω διαδικασιών

Έξυπνες **τεχνικές ανάπτυξης κώδικα** που μπορούν να συμβάλουν στην **αύξηση της απόδοσης**, περιλαμβάνουν μεταξύ άλλων

- την **μεταφορά** όλων των **αναγκαίων δεδομένων** από την μνήμη της **KME** στη μνήμη της **GPU**, για **μία και μοναδική φορά** κατά την **έναρξη** της εφαρμογής και την **παραμονή τους εκεί** για **όλη τη διάρκεια** της εκτέλεσής της
- την σχεδίαση του **πυρήνα** με **τέτοιο τρόπο**, ώστε να εκτελείται **όσο το δυνατόν μεγαλύτερος** όγκος υπολογισμών
- την **αποθήκευση** των **αποτελεσμάτων** της εκτέλεσης του κώδικα στην μνήμη της **KME**, μία και μοναδική φορά, κατά την **ολοκλήρωση** της εκτέλεσης της εφαρμογής

Τεχνικές που **βελτιώνουν** την **απόδοση της εφαρμογής**, περιλαμβάνουν την **συγχώνευση πολλαπλών κλήσεων συναρτήσεων πυρήνα**, σε μια προσπάθεια **ελαχιστοποίησης** του **όγκου** των **διακινούμενων δεδομένων**, καθώς και την **χρονική επικάλυψη** ανάμεσα στις **διαδικασίες μεταφοράς δεδομένων** και **εκτέλεσης συναρτήσεων πυρήνα**, εφόσον κάτι τέτοιο είναι επιτρεπτό



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Παράλληλος πολλαπλασιασμός δισδιάστατων πινάκων Η μετάβαση από την μονοδιάστατη στην δισδιάστατη περίπτωση απαιτεί την χρήση δύο παραμέτρων περιγραφής του νήματος, μιας παραμέτρου για την κάθε διάσταση. Θα είναι λοιπόν

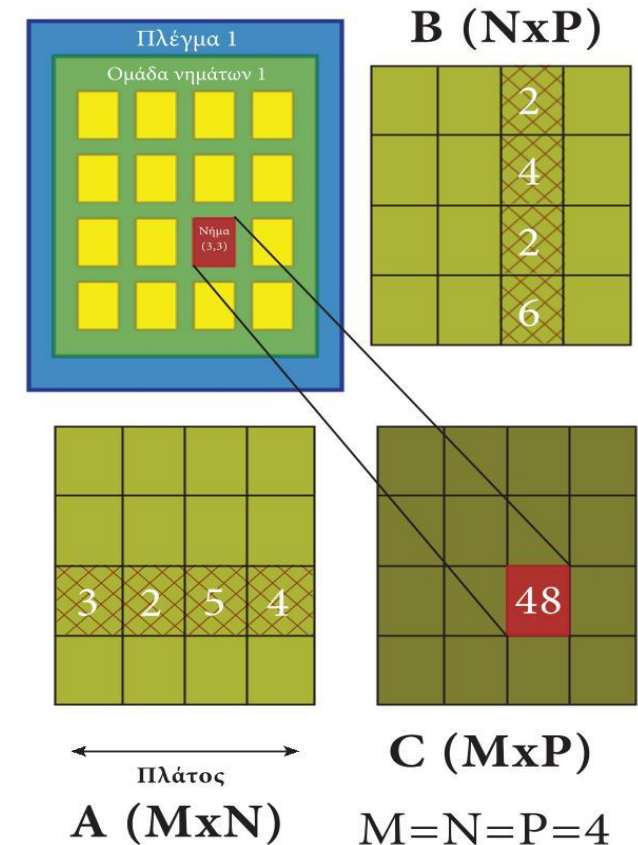
```
columns = threadIdx.x = blockIdx.x * blockDim.x + threadIdx.x
rows = threadIdx.y = blockIdx.y * blockDim.y + threadIdx.y
```

με τις μεταβλητές `threadIDy` και `threadIDx` να αντιστοιχούν στους μετρητές `i` και `j` του διπλού ένθετου βρόχου που χρησιμοποιούνται για την σάρωση των γραμμών και των στηλών ενός δισδιάστατου πίνακα στους σειριακούς κώδικες.

Η οργάνωση των νημάτων μπορεί να προσαρμοστεί στην οργάνωση των δεδομένων του δισδιάστατου πίνακα έτσι ώστε το νήμα που βρίσκεται στο κελί (i,j) της δισδιάστατης ομάδας νημάτων, να επεξεργαστεί το αντίστοιχο κελί του δισδιάστατου πίνακα και ως εκ τούτου, η χρήση του διπλού ένθετου βρόχου να είναι εντελώς περιττή.

Εάν το **πλήθος των στηλών** του πίνακα είναι ίσο με **width**, μπορούμε, αντί για τις παραμέτρους **rows** και **columns**, να περιγράψουμε ένα νήμα μέσω της παραμέτρου

```
int threadID = rows * width + columns
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

```
__global__ void matMult (double * matA, double * matB,
                        double * matC, int N) {
    int row = threadIdx.y, column = threadIdx.x;
    double temp=0;
    if (row<N && column<N) {
        for (int i=0;i<N;i++) {
            temp += matA[row*N+i]*matB[i*N+column];
        }
        matC[row*N+column]=temp;
    }
}

void multiply (double * matA, double * matB, double * matC, int N){
    double *gpuA, *gpuB, *gpuC; /* Matrices on device */
    dim3 threads (N,N); /* Each block has NxN threads */
    dim3 blocks(1,1); /* Grid contains only one block */
    // Allocate memory for device matrices (gpuA, gpuB, gpuC)
    cudaMalloc(&gpuA, N*N*sizeof(double));
    cudaMalloc(&gpuB, N*N*sizeof(double));
    cudaMalloc(&gpuC, N*N*sizeof(double));
    /* Copy data from host vectors matA and matB */
    /* to device vectors gpuA and gpuB */
    cudaMemcpy(gpuA, matA, N*N*sizeof(double),
               cudaMemcpyHostToDevice);
    cudaMemcpy(gpuB, matB, N*N*sizeof(double),
               cudaMemcpyHostToDevice);
    /* Call kernel function to perform the parallel */
    /* matrix multiplication */
    /* Synchronize threads in the grid */
    matMult <<<blocks,threads>>> (gpuA, gpuB, gpuC, N);
    cudaDeviceSynchronize();
    // Copy data from device (gpuC) to host (cpuC)
    cudaMemcpy(matC, gpuC, N*N*sizeof(double),
               cudaMemcpyDeviceToHost);
    // Free memory in GPU device
    cudaFree(gpuA); cudaFree(gpuB); cudaFree(gpuC); }

// Performs matrix multiplication C = A*B where A, B
// and C are NxN matrices
int main(int argc, char ** argv) {
    int size; /* size of square matrices */
    double *cpuA, *cpuB, *cpuC; /* Matrices on host */
    if (argc!=2) {
        printf ("Usage: matrixMulOneBlock size\n");
        return (0);
    }
    else size = atoi(argv[1]);
    if (size>32) {
        printf("Maxdim in this CUDA implementation is 32\n");
        printf("since 32x32=1024 is the max TPB value \n");
        printf("Use a smaller value for size of matrices\n");
        return (-1);
    }
    // Allocate memory for matrices A, B, and C on host
    cpuA = (double*)malloc(size*size*sizeof(double));
    cpuB = (double*)malloc(size*size*sizeof(double));
    cpuC = (double*)malloc(size*size*sizeof(double));
    for (int i=0;i<size;i++){
        for (int j=0;j<size;j++){
            cpuA[i*size+j] = (double)rand()/RAND_MAX;
            cpuB[i*size+j] = (double)rand()/RAND_MAX;
            cpuC[i*size+j] = 0.0;
        }
    }
    // Call multiply to take care of GPU stuff
    multiply (cpuA, cpuB, cpuC, size);
    // On return, cpuC contains the data of matrix product
    // Print data for debugging purposes
    for (int i=0;i<size;i++){
        for (int j=0;j<size;j++){
            printf ("%.3f ",cpuC[i*size+j]);
        }
        printf("\n");
    }
    // Free memory in host device
    free(cpuA); free(cpuB); free(cpuC);
    return 0; }
```

Παράλληλος πολλαπλασιασμός **δισδιάστατων τετραγωνικών πινάκων** τάξεως **N** χρησιμοποιώντας μία **απλή ομάδα νημάτων** που περιλαμβάνει **$N \times N = N^2$** νήματα και με το νήμα **(i,j)** να υπολογίζει την (μία και μοναδική) τιμή **C_{ij}**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Στην προηγούμενη υλοποίηση το **υπολογιστικό πλέγμα** περιέχει μόνο **μία ομάδα νημάτων** η οποία μπορεί να περιέχει **το πολύ 1024 νήματα** και κατά συνέπεια, η **μέγιστη διάσταση πίνακα** που μπορεί να χρησιμοποιηθεί είναι **$N = \text{sqrt}(1024) = 32$**

Επιπλέον, η χρήση **μιας και μοναδικής** ομάδας νημάτων, σημαίνει πως θα χρησιμοποιηθεί **μόνο ένας** από τους **διαθέσιμους πολυεπεξεργαστές** της **GPU** ενώ οι υπόλοιποι θα παραμένουν **αδρανείς**, κάτι που οδηγεί σε **φτωχή απόδοση**.

Η **σωστή χρήση** της εφαρμογής απαιτεί μία **συνθήκη ελέγχου** που **επιβάλλει** την εκτέλεση της εφαρμογής μόνο εάν είναι **$N \leq 32$** . Εάν αντίθετα προσπαθήσουμε να εκτελέσουμε τον κώδικα για τιμές **$N > 32$** , η εκτύπωση των περιεχομένων του πίνακα γινομένου θα οδηγήσει στην **εμφάνιση μηδενικών** τιμών, γεγονός που υποδηλώνει πως έχει λάβει χώρα κάποιο **σφάλμα**.

Για τον **χειρισμό σφαλμάτων**, μετά από κλήση συνάρτησης CUDA, καλούμε τη **cudaGetLastError**, που επιστρέφει κωδικό σφάλματος ή την τιμή **cudaSuccess** σε περίπτωση επιτυχίας και στην περίπτωση **εμφάνισης σφάλματος** μπορούμε καλώντας την συνάρτηση **cudaGetErrorString**, να **ανακτήσουμε** και να **εκτυπώσουμε** μία λεκτική περιγραφή για αυτό το σφάλμα, ως εξής:

```
cudaError_t err = cudaGetLastError();
if (err!=cudaSuccess) {
    printf("CUDA Error: %s\n", cudaGetErrorString(err));
    exit (-1) ; }
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Εάν υιοθετήσουμε αυτή την προσέγγιση και καταχωρήσουμε μία τιμή $N > 32$, η CUDA θα εκτυπώσει το μήνυμα σφάλματος

```
CUDA Error: invalid configuration argument
```

και η εφαρμογή θα τερματιστεί, καλώντας την συνάρτηση `exit`. Εάν δεν χρησιμοποιήσουμε την `exit` η εκτέλεση της εφαρμογής θα συνεχίσει στην ΚΜΕ, αλλά επειδή η συνάρτηση του πυρήνα δεν θα εκτελεστεί, το αποτέλεσμα θα είναι εσφαλμένο

Πολλαπλασιασμός πινάκων με αυθαίρετες διαστάσεις Με ποιον τρόπο μπορούμε να παρακάμψουμε τον παραπάνω περιορισμό, έτσι ώστε να είναι δυνατός ο πολλαπλασιασμός πινάκων με μεγαλύτερο πλήθος διαστάσεων;

Η μελέτη του προβλήματος αναδεικνύει την ύπαρξη τριών προσεγγίσεων:

- **Προσέγγιση tile:** χρησιμοποιείται μόνο μία ομάδα νημάτων το καθένα των οποίων υπολογίζει έναν υποπίνακα του πίνακα γινομένου τάξεως $M < N$ (για λόγους απλότητας χρησιμοποιούμε μόνο τετραγωνικούς πίνακες)
- **Προσέγγιση block:** χρησιμοποιούνται πολλές ομάδες νημάτων διαστάσεων $P \times P$ με την κάθε ομάδα να περιέχει το πολύ 1024 νήματα τα οποία ορίζουν έναν πίνακα νημάτων διαστάσεων 32×32 και το καθένα των οποίων υπολογίζει ένα απλό στοιχείο του πίνακα γινομένου
- **Προσέγγιση blockTile:** προκύπτει από τον συνδυασμό των δύο παραπάνω προσεγγίσεων και κατά συνέπεια χρησιμοποιούνται πολλές ομάδες νημάτων το καθένα των οποίων υπολογίζει έναν υποπίνακα τάξεως M .



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Προκειμένου η **πληρότητα** που σχετίζεται με το **ποσοστό της χρήσης των νημάτων** να είναι **100%**, η **διάσταση** των **τετραγωνικών πινάκων** θα πρέπει να είναι **ακέραιο πολλαπλάσιο** του **32**, ενώ στην αντίθετη περίπτωση κάποια νήματα δεν θα πραγματοποιούν υπολογισμούς, αφού θα περισσεύουν και ως εκ τούτου, η πληρότητα θα χαρακτηρίζεται από μικρότερη τιμή.

Η προσέγγιση tile: υλοποιείται από την **συνάρτηση πυρήνα matMultWithTile**, με τον ακόλουθο κώδικα

```
__global__ void matMultWithTile (double * matA, double * matB, double * matC, int N) {  
    int tiledim = N/32;  
    int startRow = threadIdx.y*tiledim;  
    int startColumn = threadIdx.x*tiledim;  
    int endRow = startRow + tiledim;  
    int endColumn = startColumn + tiledim;  
    double a,b,temp;  
    for (int i=startRow;i<endRow;i++) {  
        for (int j=startColumn;j<endColumn;j++) {  
            temp = 0.0;  
            for (int k=0;k<N;++k) {  
                a = matA[i*N+k];  
                b = matB[k*N+j];  
                temp += a*b; }  
            matC[i*N+j] = temp; }}}
```



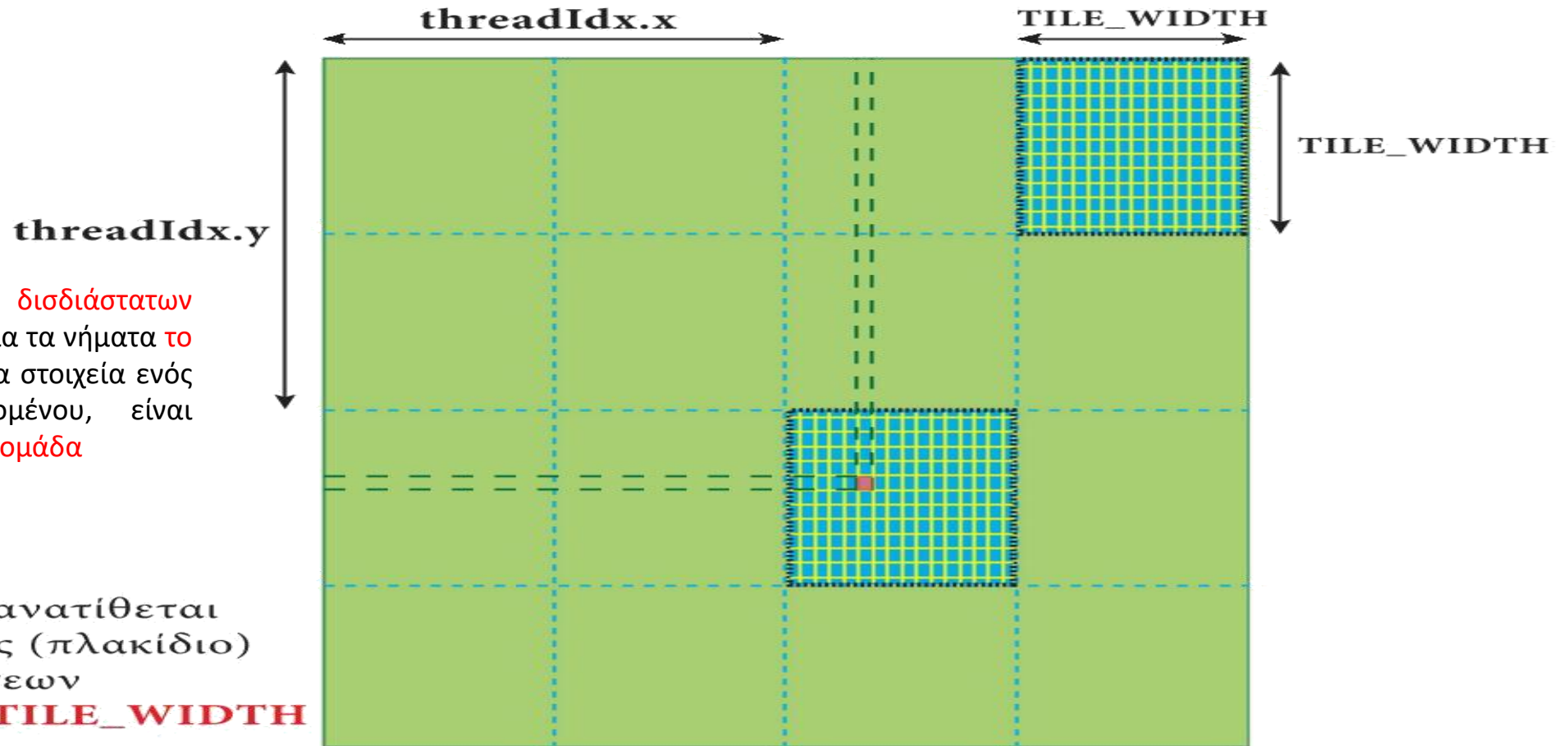
Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Η προσέγγιση tile

Διαδικασία πολλαπλασιασμού **δισδιάστατων τετραγωνικών πινάκων**, στην οποία τα νήματα **το καθένα των οποίων** υπολογίζει τα στοιχεία ενός **υποπίνακα** του πίνακα γινομένου, είναι οργανωμένα σε **μία και μοναδική ομάδα**

Σε κάθε νήμα ανατίθεται
ένας υποπίνακας (πλακίδιο)
διαστάσεων
 $TILE_WIDTH \times TILE_WIDTH$



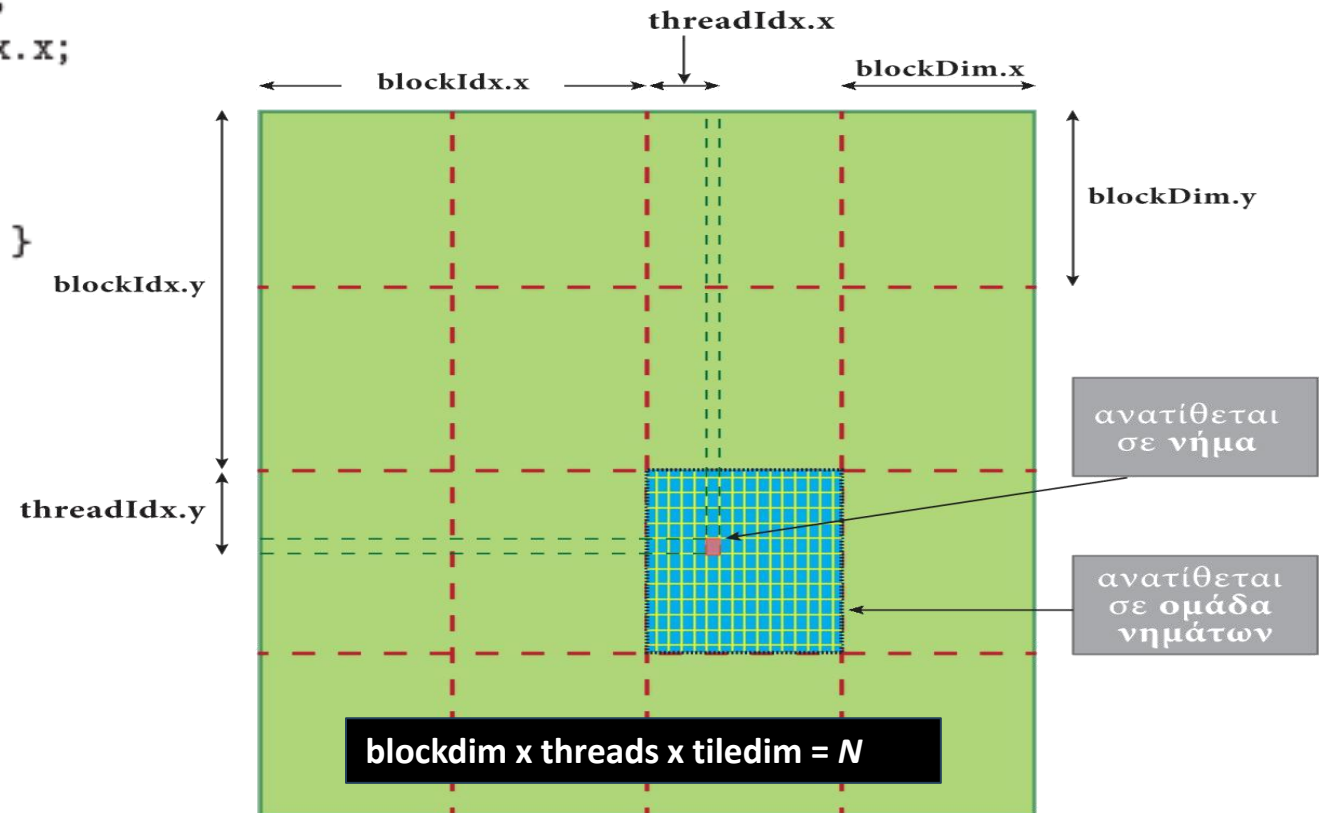
Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Η προσέγγιση block: υλοποιείται από την **συνάρτηση πυρήνα** **matMultWithBlock**, με τον ακόλουθο κώδικα

```
__global__ void matMultWithBlock (double * matA, double * matB, double * matC, int N) {  
    int row = blockIdx.y * blockDim.y + threadIdx.y;  
    int column = blockIdx.x * blockDim.x + threadIdx.x;  
    double temp=0;  
    if (row<N && column<N) {  
        for (int i=0;i<N;i++) {  
            temp += matA[row*N+i]*matB[i*N+column]; }  
        matC[row*N+column]=temp; }  
}
```

Τα νήματα είναι οργανωμένα στο **κατάλληλο πλήθος ομάδων**, έτσι ώστε κατά την **εκτέλεση** της εφαρμογής, να χρησιμοποιείται το **κατάλληλο πλήθος πολυεπεξεργαστών**. Η κάθε ομάδα έχει διαστάσεις **32 x 32** και επειδή οι πίνακες που χρησιμοποιούνται, έχουν διαστάσεις **N x N**, οι ομάδες θα σχηματίσουν έναν **δισδιάστατο** πίνακα με μέγεθος **P x P** όπου **P = N/32**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

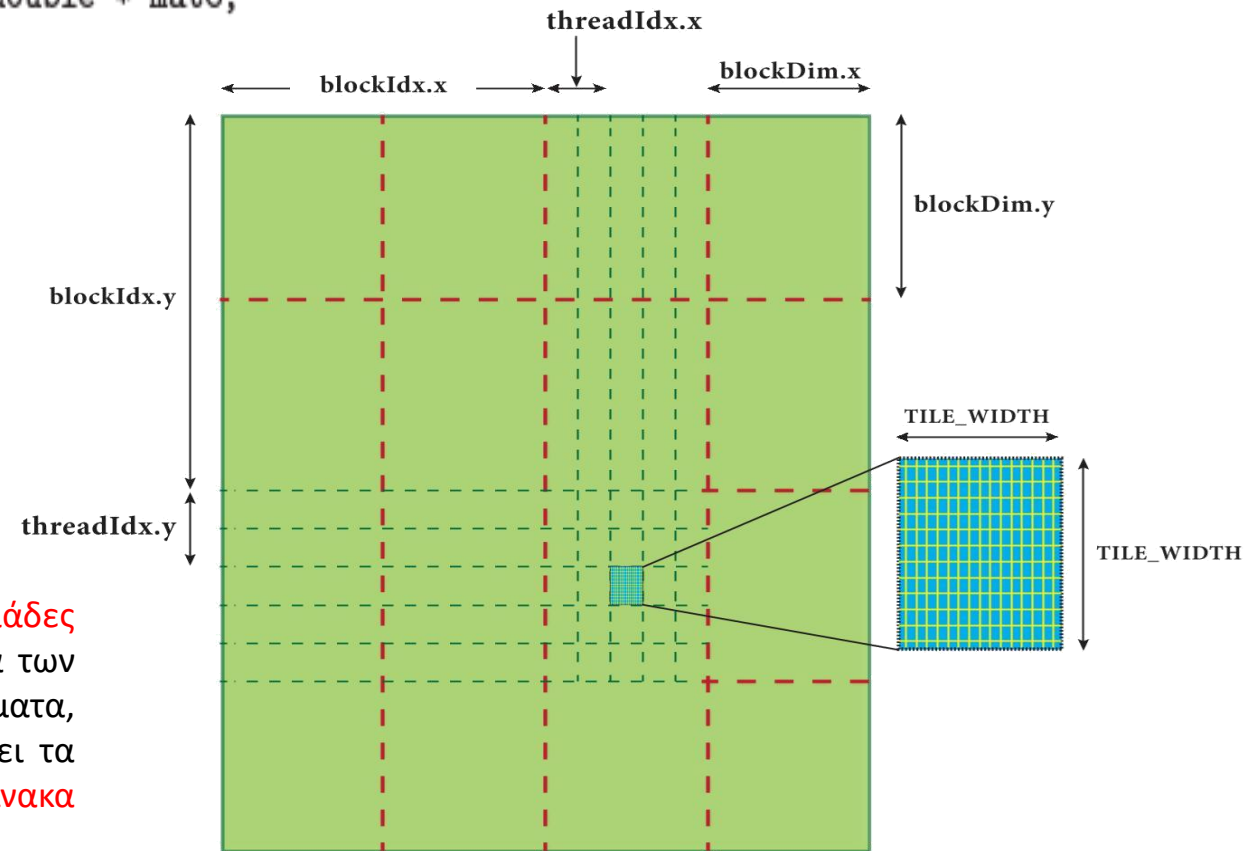
Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Η προσέγγιση **blockTile**: υλοποιείται από την **συνάρτηση πυρήνα** **matMultWithBlockTile**, με τον ακόλουθο κώδικα

```
__global__ void matMultWithBlockTile (double * matA, double * matB, double * matC,
                                     int N, int tiledim) {

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int column = blockIdx.x * blockDim.x + threadIdx.x;
    double a,b,temp;
    int startRow = row*tiledim;
    int startColumn = column*tiledim;
    int endRow = startRow + tiledim;
    int endColumn = startColumn + tiledim;
    for (int i=startRow;i<endRow;i++) {
        for (int j=startColumn;j<endColumn;j++) {
            temp = 0.0;
            for (int k=0;k<N;++k) {
                a = matA[i*N+k];
                b = matB[k*N+j];
                temp += a*b; }
            matC[i*N+j] = temp; }}}
```

Τα νήματα οργανώνονται σε **ομάδες** μεγέθους **blockdim**, η καθεμία των οποίων θα περιέχει **threads** νήματα, με το **κάθε νήμα** να υπολογίζει τα στοιχεία **τετραγωνικού υποπίνακα** μεγέθους **tiledim**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Η προσέγγιση `blockShared`: Ένας εναλλακτικός τρόπος υλοποίησης, που μπορεί να οδηγήσει σε πολύ μικρό χρόνο εκτέλεσης μιας εφαρμογής, εφόσον πραγματοποιηθεί σωστά και συντρέχουν οι λόγοι της εφαρμογής του, είναι η χρήση όχι της καθολικής μνήμης της συσκευής, αλλά της κοινόχρηστης μνήμης του πολυεπεξεργαστή, στον οποίο εκτελείται μία ομάδα νημάτων.

Η διαδικασία μοιάζει αρκετά με την προσέγγιση `blockTile`, αλλά στην προκειμένη περίπτωση τα τμήματα των πινάκων που ορίζουν τους κατάλληλους υποπίνακες, έχουν μεταφερθεί στην κοινόχρηστη μνήμη

Λαμβάνοντας υπόψη πως όλα τα νήματα της ίδιας ομάδας, διαμοιράζονται την ίδια κοινόχρηστη μνήμη, θα ορίσουμε τις διαστάσεις των υποπινάκων που χρησιμοποιούνται στην παραπάνω τεχνική, ίσες με τις διαστάσεις των ομάδων νημάτων.

Εάν, λοιπόν, ο χρήστης ορίσει από το πληκτρολόγιο μία τιμή για την παράμετρο `tiledim`, αυτή η τιμή θα χρησιμοποιηθεί και για τον ορισμό του μεγέθους των ομάδων νημάτων, οπότε θα είναι `blockDim.x = blockDim.y = tiledim` (εκτός εάν οι πίνακες είναι πολύ μικροί και μπορούν να φορτωθούν ολόκληροι στην κοινόχρηστη μνήμη)

Το μέγεθος της κοινόχρηστης μνήμης που δεσμεύεται στην συσκευή δεν είναι στατικά ορισμένο, αλλά δίδεται από τον χρήστη κατά την διάρκεια της εκτέλεσης. Κατά συνέπεια, δεν μπορούμε να χρησιμοποιήσουμε στον πυρήνα μία δήλωση της μορφής `__shared__ double array[tiledim]`, αλλά η διαδικασία που ακολουθείται είναι διαφορετική.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

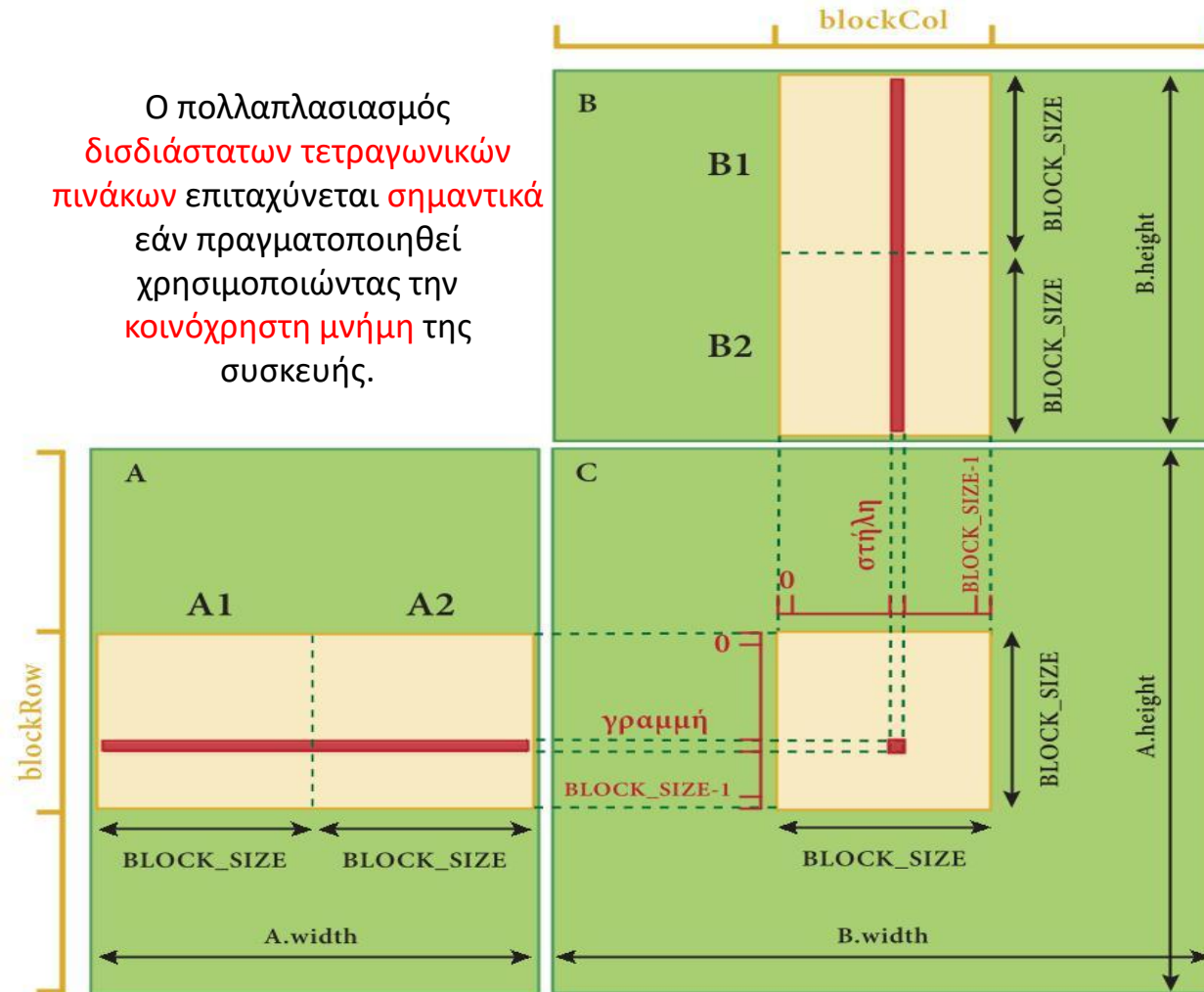
Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Επειδή το μέγεθος της **κοινόχρηστης** μνήμης **δεν είναι** πολύ μεγάλο, **δεν** θα φορτώσουμε το **σύνολο** των πινάκων **A** και **B**, αλλά τους **κατάλληλους υποπίνακες**, οι οποίοι αποτελούν **τετραγωνικούς** πίνακες μεγέθους **tileDim x tileDim**

Το **κάθε νήμα** θα υπολογίσει την τιμή **ακριβώς** ενός στοιχείου C_{ij} πολλαπλασιάζοντας **την υπ' αριθμόν i γραμμή** του πίνακα **A**, με την **υπ' αριθμόν j στήλη** του πίνακα **B**. Αυτή η διαδικασία θα πραγματοποιηθεί **τμηματικά**, μέσα από έναν **βρόχο** αφού **μόνο ένα τμήμα των δύο πινάκων** θα είναι διαθέσιμο κάθε φορά στην **κοινόχρηστη μνήμη**.

Στη **γενική περίπτωση**, αυτός ο βρόχος θα εκτελεστεί συνολικά $M = N / \text{tileDim}$ φορές, με την τιμή της παραμέτρου για την περίπτωση του σχήματος να είναι **M=2**

Ο πολλαπλασιασμός **δισδιάστατων τετραγωνικών πινάκων** επιταχύνεται **σημαντικά** εάν πραγματοποιηθεί χρησιμοποιώντας την **κοινόχρηστη μνήμη** της συσκευής.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Η δυναμική δέσμευση κοινόχρηστης μνήμης, με μέγεθος $L = 2 \times \text{tiledim} \times \text{tiledim} \times \text{sizeof}(\text{double})$, δεν γίνεται με την συνάρτηση `cudaMalloc` αλλά καταχωρώντας στη συνάρτηση του πυρήνα, ένα τρίτο όρισμα που προσδιορίζει το μέγεθος της μνήμης που επιθυμούμε να δεσμεύσουμε δυναμικά στην κοινόχρηστη μνήμη της συσκευής, ο οποίος, επομένως, καλείται ως

```
matMultWithBlockShared <<<blocks,threads,L>>>(gpuA, gpuB, gpuC, N, tile);
```

Επομένως, όταν ξεκινήσει η εκτέλεση του κώδικα που περιλαμβάνεται στην συνάρτηση του πυρήνα, η κοινόχρηστη μνήμη έχει ήδη δεσμευτεί και είναι έτοιμη προς χρήση

Ανεξάρτητα από το πλήθος, τον τύπο και το μέγεθος των πινάκων που θα χρησιμοποιήσουμε, απαιτείται η χρήση ενός και μοναδικού δείκτη τύπου `extern` με τη βοήθεια του οποίου μπορούμε να διαχωρίσουμε την περιοχή μνήμης που προσδιορίζεται από αυτόν, χρησιμοποιώντας βοηθητικούς δείκτες και τις κατάλληλες διαδικασίες μετατροπής τύπου.

Εδώ, ο δείκτης `sharedA` προσδιορίζει την αρχή της περιοχής μνήμης που έχει δεσμευτεί στην κοινόχρηστη μνήμη της συσκευής, ενώ ο δείκτης `sharedB` προσδιορίζει μία διεύθυνση μνήμης, εντός αυτής της περιοχής, η οποία απέχει από την αρχή της προηγούμενης περιοχής απόσταση ίση με $L/2 = \text{tiledim} \times \text{tiledim} \times \text{sizeof}(\text{double})$ bytes.

Απαιτείται συγχρονισμός νημάτων δύο φορές, για να διασφαλιστεί πως όλα τα νήματα (α) έχουν μεταφέρει τα στοιχεία των πινάκων στην κοινόχρηστη μνήμη και (β) έχουν πραγματοποιήσει τον υπολογισμό της τιμής του αθροίσματος `sum`



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

Ο **πηγαίος κώδικας** του **πυρήνα** για αυτή την περίπτωση, ακολουθεί στη συνέχεια

```
__global__ void matMultWithBlockShared (double * matA, double * matB,
                                         double * matC, int N, int tiledim) {
    int row = blockIdx.y, column = blockIdx.x, tiles = N/tiledim;
    int tileRow = threadIdx.y, tileColumn = threadIdx.x;
    double sum = 0.0;
    extern __shared__ double shGPUMem[];
    double *sharedA = shGPUMem;
    double *sharedB = (double *)&sharedA[tiledim*tiledim];
    double *cellA, *cellB, *cellC = &matC[N*tiledim*row+tiledim*column];
    for (int i=0;i<tiles;++i) {
        cellA = &matA[N*tiledim*row+tiledim*i];
        cellB = &matB[N*tiledim*i+tiledim*column];
        sharedA[tileRow*tiledim+tileColumn] = cellA[tileRow*N+tileColumn];
        sharedB[tileRow*tiledim+tileColumn] = cellB[tileRow*N+tileColumn];
        __syncthreads();
        for (int j=0;j<tiledim;j++)
            sum += sharedA[tileRow*tiledim+j]*sharedB[j*tiledim+tileColumn];
        __syncthreads(); }
    cellC[tileRow*N+tileColumn] = sum; }
```

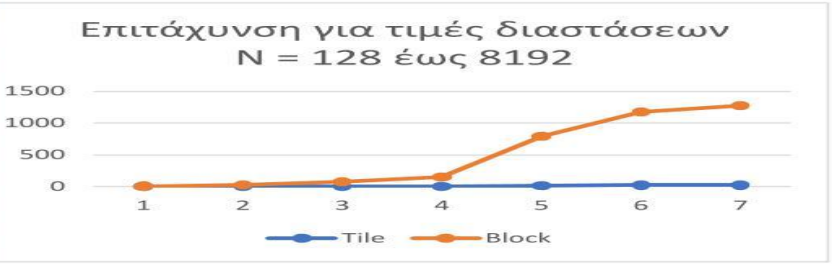


Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Παραδείγματα εφαρμογών στο περιβάλλον της CUDA

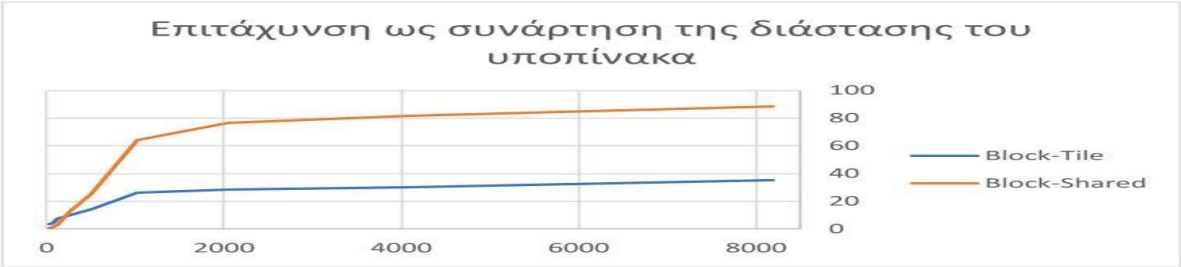
Ενδεικτικά αποτελέσματα εκτέλεσης της εφαρμογής παράλληλου πολλαπλασιασμού πινάκων στην CUDA, χρησιμοποιώντας όλες τις τεχνικές που παρουσιάσαμε στην θεωρητική περιγραφή του προβλήματος

DIM	EXECUTION TIME			SPEED-UP (SERIAL TIME / PARALLEL TIME)			
	SERIAL	TILE mode	BLOCK mode	TILE	BLOCK	BLOCK-TILE	BLOCK-SHARED
32	0,075	0,594	0,603	0,126	0,124	0,13686131	0,126903553
64	0,466	0,689	0,668	0,676	0,698	0,60362694	0,757723577
128	3,75	1,198	0,703	3,130	5,334	5,08819539	5,490483163
256	28,039	6,082	1,054	4,610	26,602	20,6472754	27,98303393
512	267,141	81,299	3,465	3,286	77,097	71,2756137	77,29774306
1024	2398,323	616,771	15,968	3,889	150,196	151,54322	150,4782909
2048	80998,671	4922,083	102,671	16,456	788,915	777,032751	764,0302882
4096	851648,941	39561,274	724,375	21,527	1175,702	1088,51393	1066,687885
8192	6977838,678	313044,854	5481,284	22,290	1273,030	1218,16874	1167,49189



DIM	BLOCK - TILE mode						BLOCK - SHARED mode					
	1	2	4	8	16	32	1	2	4	8	16	32
32	0,548	0,599	0,535	0,77	0,861	2,006	0,603	0,624	0,573	0,564	0,567	0,591
64	0,772	0,704	0,712	0,731	1,385	3,401	0,718	0,601	0,537	0,557	0,518	0,615
128	0,737	0,648	0,783	1,023	2,066	5,733	1,876	1,898	0,671	0,625	0,618	0,683
256	1,358	1,059	1,003	1,836	4,188	13,266	11,941	3,041	1,162	0,884	0,967	1,002
512	3,748	3,689	3,628	5,466	14,839	52,671	87,881	20,918	4,792	3,281	3,239	3,456
1024	15,826	15,592	19,484	30,549	108,364	413,171	1022,594	162,359	27,924	15,326	14,976	15,938
2048	104,241	104,612	199,686	29,588	782,728	2940,191	8134,963	1246,185	217,295	103,964	100,459	106,015
4096	782,396	871,674	1678,011	1822,215	6285,181	23429,301	65385,892	10191,71	1728,768	752,494	727,296	798,405
8192	5728,138	12076,324	13721,766	14869,177	52992,041	201797,411	528841,501	81143,51	13555,55	5653,85	5486,391	5976,777

DIM	SPEED-UP (MAX/MIN)	
	BLOCK-TILE	SHARED
32	3,660583942	1,02030457
64	4,405440415	1,16747967
128	7,778833107	2,74670571
256	9,768777614	11,9171657
512	14,05309498	25,4285301
1024	26,10710224	64,1607479
2048	28,20570601	76,7340754
4096	29,94557871	81,8956444
8192	35,22914619	88,4827225



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Πρωτογενείς συναρτήσεις διαχείρισης στημονιού

Η **CUDA** προσφέρει συναρτήσεις **συνεργασίας νημάτων** του ιδίου **στημονιού** που ομαδοποιούνται σε **τέσσερις** κατηγορίες

- συναρτήσεις **ψηφοφορίας**, που ζητούν από τα **32 νήματα** να **επιστρέψουν** κάποια τιμή
- συναρτήσεις **αναδιάταξης**, οι οποίες επιτρέπουν την **ανταλλαγή δεδομένων** ανάμεσα στα νήματα του στημονιού
- συναρτήσεις **αντιστοίχισης** οι οποίες επιτρέπουν την **εκπομπή μιας τιμής** σε όλα τα νήματα και **πραγματοποιούν** συγκρίσεις
- συναρτήσεις **πινάκων** που επιτρέπουν την **επιτάχυνση των πράξεων** οι οποίες πραγματοποιούνται **ανάμεσα** σε πίνακες

Το **πλεονέκτημα** αυτής της **προσέγγισης** είναι πως η **αλληλεπίδραση** μεταξύ των νημάτων **δεν στηρίζεται** στην χρήση της **κοινόχρηστη μνήμης**, αλλά στους (**πολύ πιο γρήγορους**) **καταχωρητές** του πολυεπεξεργαστή οδηγώντας σε **υψηλή απόδοση**.

Για την **αλληλεπίδραση** των νημάτων με αυτόν τον τρόπο, το **κάθε στημόνι** θεωρείται πως αποτελείται από **32 λωρίδες** οι οποίες καταλαμβάνονται από **ισάριθμα νήματα**, με το κάθε νήμα να χρησιμοποιεί **την δική του λωρίδα** και να περιγράφεται από ένα **αναγνωριστικό** το οποίο λαμβάνει **μοναδική τιμή** στην περιοχή τιμών **[0, 31]**.

Ο τρόπος με τον οποίο **υποδηλώνεται** εάν το **καθένα** από τα **32 νήματα του στημονιού** συμμετέχει ή όχι στην εμπλεκόμενη διαδικασία, είναι ακριβώς ο ίδιος με τον τρόπο με τον οποίο λαμβάνει χώρα η **διαχείριση των σημάτων στο Linux**

Δηλαδή, τα **32 νήματα** του **στημονιού** αντιμετωπίζονται από κοινού ως μία **ομάδα νημάτων**, με την χρήση μιας **μάσκας μήκους 32 bits** και με το **υπ' αριθμόν i bit** να έχει τιμή **1** με **0**, με κριτήριο το εάν το νήμα με κωδικό **i** συμμετέχει στην διαδικασία ή **όχι**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Πρωτογενείς συναρτήσεις διαχείρισης στημονιού

```
/* WRAP SHUFFLE FUNCTIONS (Compute capability >=3.x) */
```

```
T __shfl_sync (unsigned mask, T var, int srcLane, int width=warpSize); /* Exchanges info between threads */
T __shfl_up_sync (unsigned mask, T var, unsigned int delta, int width=warpSize); /* Copies from lane with lower ID */
T __shfl_down_sync (unsigned mask, T var, unsigned int delta, int width=warpSize); /* Copies from lane with higher ID */
T __shfl_xor_sync (unsigned mask, T var, int laneMask, int width=warpSize) /* Copies from lane based on XOR */
```

```
/* WRAP VOTE FUNCTIONS (Compute capability >=7.x) */
```

```
__all_sync (unsigned mask, int predicate); /* Checks if a predicate is true for all the threads in the warp's mask */
__any_sync (unsigned mask, int predicate); /* Checks if a predicate is true for any of the threads in the warp */
__ballot_sync (unsigned mask, int predicate); /* Evaluates the predicate for all participating threads creating mask */
__activemask(); /* Returns a 32-bit mask of all currently active threads in warp */
```

```
/* WRAP MATCH FUNCTIONS (Compute capability >=7.x) */
```

```
__match_any_sync (unsigned mask, T value); /* Returns a mask of all the threads with the same "value" */
__match_all_sync (unsigned mask, T value, int *pred); /* Returns mask if all threads have the same value */
```

```
/* WRAP REDUCE FUNCTIONS (Compute capability >=8.x) */
```

```
unsigned __reduce_add_sync (unsigned mask, unsigned value); /* Returns the result of a reduction for an add operation */
unsigned __reduce_min_sync (unsigned mask, unsigned value); /* Returns the result of a reduction for a min operation */
unsigned __reduce_max_sync (unsigned mask, unsigned value); /* Returns the result of a reduction for a max operation */
int __reduce_add_sync (unsigned mask, int value); /* Returns the result of a reduction for an add operation */
int __reduce_min_sync (unsigned mask, int value); /* Returns the result of a reduction for a min operation */
int __reduce_max_sync (unsigned mask, int value); /* Returns the result of a reduction for a max operation */
unsigned __reduce_and_sync (unsigned mask, unsigned value); /* Returns the result of a reduction for an and operation */
unsigned __reduce_or_sync (unsigned mask, unsigned value); /* Returns the result of a reduction for an or operation */
unsigned __reduce_xor_sync (unsigned mask, unsigned value); /* Returns the result of a reduction for a xor operation */
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Συνεργατικές συλλογές νημάτων

Η **CUDA** προσφέρει την δυνατότητα της **ομαδοποίησης** των νημάτων σε κατάλληλα διαμορφωμένες **συλλογές νημάτων**, το **ελάχιστο μέγεθος** των οποίων είναι το **μέγεθος ενός στημονιού** και οι οποίες επιτρέπουν τον **συγχρονισμό** ενός **πλήθους νημάτων**, η τιμή του οποίου είναι **μικρότερη** από το μέγεθος της **ομάδας** του πλέγματος.

Αυτό το **προγραμματιστικό μοντέλο** (**cooperative groups**), αποτελείται από τα **επόμενα** στοιχεία:

- Ένα σύνολο **τύπων δεδομένων** για την **αναπαράσταση** μιας **συλλογής** από **συνεργαζόμενα** μεταξύ τους νήματα.
- Μία **συλλογική συνάρτηση**, που επιτρέπει την **υποδιαίρεση** των **υφιστάμενων ομάδων νημάτων** σε νέες ομάδες.
- Ένα σύνολο **συλλογικών διαδικασιών** που επιτρέπουν την **ανταλλαγή δεδομένων** ανάμεσα στα νήματα που ορίζουν αυτή την **συλλογή**, καθώς και την **τροποποίηση** των τιμών τους.
- Έναν **μηχανισμό συγχρονισμού** της λειτουργίας των νημάτων που ανήκουν **στην ίδια** συλλογή.
- Έναν **μηχανισμό ελέγχου** των τιμών των ιδιοτήτων που περιγράφουν μία ομάδα νημάτων.
- Ένα **σύνολο συλλογικών διαδικασιών χαμηλού επιπέδου**, που σχετίζονται με τις ομάδες νημάτων και η λειτουργία των οποίων μπορεί να **επιταχυνθεί** με την βοήθεια του **υλικού** της συσκευής.

Υφίστανται **δύο δυνατότητες**, όσον αφορά στην **δημιουργία συλλογών από νήματα**, ανάλογα με το εάν τα νήματα που ορίζουν μία τέτοια **συλλογή** ανήκουν **στην ίδια ομάδα** νημάτων (**intrablock**) ή σε **διαφορετικές** ομάδες νημάτων (**interblock**)



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Συνεργατικές συλλογές νημάτων

```
class thread_block /* Represents a thread block in the cgroups mechanism */
/* Constructed via: */ thread_block g = this_thread_block();

sync          /* Synchronize the threads named in the group */
barrier_arrive /* Arrive on the thread_block barrier */
barrier_wait  /* Wait on the thread_block barrier */
thread_rank   /* Rank of the calling thread within [0, num_threads) */
group_index   /* 3-Dimensional index of the block within the grid */
thread_index  /* 3-Dimensional index of the thread within the block */
dim_threads   /* Dimensions of the launched block in units of threads */
num_threads   /* Total number of threads in the group */

class coalesced_group /* Represents a coalesced group of threads */
/* Constructed via: */ coalesced_group active = coalesced_threads();

sync          /* Synchronize the threads named in the group */
num_threads   /* Total number of threads in the group */
thread_rank   /* Rank of the calling thread within [0, num_threads) */
meta_group_size /* Returns the number of created groups via partitioned */
meta_group_rank /* Linear rank of the group within the set of tiles */

/* Shuffle funcs */ shfl, shfl_up, shfl_down
/* Vote func */ any, all, ballot
/* Match funcs */ match_any, match_all

class cluster_group /* Represents all threads launched in a cluster */
/* Constructed via: */ cluster_group g = this_cluster();

sync          /* Synchronize the threads named in the group */
barrier_arrive /* Arrive on the cluster barrier */
barrier_wait  /* Wait on the cluster barrier */
thread_rank   /* Rank of the calling thread within [0, num_threads) */
block_rank    /* Rank of the calling block within [0, num_blocks) */
num_threads   /* Total number of threads in the group */
num_blocks    /* Total number of blocks in the group */
dim_threads   /* Dimensions of launched cluster in units of threads */
dim_blocks    /* Dimensions of launched cluster in units of blocks */
block_index   /* 3-Dimensional index of the calling block in cluster */
query_shared_rank /* Obtain the block rank for a shared memory address */
map_shared_rank /* Obtain the address of a shared memory variable */
```

```
class multi_grid_group /* represents all launched threads across all devs */
/* Constructed via: */ multi_grid_group g = this_multi_grid();

is_valid      /* Returns whether the multi_grid_group can be used */
sync          /* Synchronize the threads named in the group */
num_threads   /* Total number of threads in the group */
thread_rank   /* Rank of the calling thread within [0, num_threads) */
grid_rank     /* Rank of the grid within [0,num_grids] */
num_grids     /* Total number of grids launched */

template <unsigned int Size, typename ParentT = void> /* A template of a */
class thread_block_tile; /* tiled group with a size parameter */

/* Constructed via: */ template <unsigned int Size, typename ParentT>
CG_QUALIFIER thread_block_tile<Size, ParentT>
tiled_partition (const ParentT& g)

sync          /* Synchronize the threads named in the group */
num_threads   /* Total number of threads in the group */
thread_rank   /* Rank of the calling thread within [0, num_threads) */
meta_group_size /* Returns the number of created groups via partitioned */
meta_group_rank /* Linear rank of the group within the set of tiles */

/* Shuffle, Vote and Match functions - See corresponding warp functions */

/* Shuffle funcs */ shfl, shfl_up, shfl_down, shfl_xor
/* Vote funcs */ any, all, ballot
/* Match funcs */ match_any, match_all
```

Οι κλάσεις του **μηχανισμού** των
συνεργατικών συλλογών της **CUDA**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Συνεργατικές συλλογές νημάτων

Οι κλάσεις `thread_group` και `thread_block` Η θεμελιώδης κλάση από την οποία προκύπτουν όλες οι υπόλοιπες κλάσεις της ιεραρχίας κλάσεων, είναι η κλάση `thread_group`, που υλοποιεί μία συλλογή νημάτων και προσφέρει έναν χειριστή προς μία τέτοια συλλογή, καθώς και ένα σύνολο μεθόδων, οι οποίες κληρονομούνται σε όλες τις παράγωγες κλάσεις όπως

- η μέθοδος `size` που επιστρέφει το μέγεθος `N` μιας συλλογής νημάτων δηλαδή το πλήθος των νημάτων που περιέχει
- η μέθοδος `thread_rank` που επιστρέφει το `TID` του νήματος που την καλεί μέσα στην συλλογή του
- η μέθοδος `is_valid`, που επιστρέφει `True` ή `False` ανάλογα με το εάν το νήμα ανήκει σε μία έγκυρη συλλογή νημάτων ή όχι
- η μέθοδος `sync`, που προκαλεί τον συγχρονισμό της λειτουργίας των νημάτων της θεωρούμενης συλλογής

Για μία συλλογή που να περιέχει μόνο το τρέχον νήμα χρησιμοποιούμε την συνάρτηση `this_thread`

Η δημιουργία μιας συλλογής νημάτων που να περιλαμβάνει όλα τα νήματα μιας ομάδας, γίνεται με την συνάρτηση `this_thread_block`, της κλάσης `thread_block` η οποία αποτελεί παράγωγη κλάση της `thread_group`. Αυτό γίνεται γράφοντας

```
thread_block bthr = this_thread_block();
```

Η δημιουργία μιας συλλογής νημάτων που να περιέχει όλα τα νήματα μιας ομάδας νημάτων είναι αναγκαία, αφού οι δυνατότητες αυτού του μηχανισμού των συνεργατικών ομάδων εφαρμόζονται μόνο πάνω σε αντικείμενα τύπου `thread_block`.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Συνεργατικές συλλογές νημάτων

Η διαμέριση μιας συλλογής νημάτων σε συλλογές νημάτων μικρότερου μεγέθους, οι οποίες χαρακτηρίζονται ως πλακίδια, πραγματοποιείται χρησιμοποιώντας την παραμετρική συνάρτηση `tiled_partition` το πρωτότυπο της οποίας έχει τη μορφή

```
template <unsigned int tileSize, typename ParentT>
    thread_block_tile <tileSize> tiled_partition (const parentT &g);
```

Η συνάρτηση που είναι παρεμποδιστική και συλλογική και επομένως αναστέλλει την λειτουργία της εφαρμογής μέχρι να κληθεί από όλα τα νήματα της συλλογής που πρόκειται να διαμεριστεί, επιστρέφει ένα αντικείμενο τύπου `thread_block_tile`, το οποίο αναπαριστά το καθένα από τα πλακίδια που δημιουργούνται, ως αποτέλεσμα της παραπάνω διαμέρισης.

Αυτή η διαμέριση εφαρμόζεται σε μία γονική συλλογή νημάτων που θα πρέπει υποχρεωτικά να ανήκει σε κάποιον από τους τύπους `thread_block` και `thread_block_tile` και παραδείγματα εφαρμογής της είναι τα

```
thread_block_tile<32> tile32 = tiled_partition<32>(this_thread_block());
thread_block_tile<4>  tile4  = tiled_partition<4> (this_thread_block());
```

Ενδιαφέρον παρουσιάζει επίσης η συνάρτηση `coalesced_group cg = coalesced_threads();` η οποία επιτρέπει την κατασκευή μιας συλλογής νημάτων, που περιέχει όλα τα νήματα ενός στημονιού που ακολουθούν την διαδρομή εκτέλεσης του νήματος που καλεί αυτή τη συνάρτηση και τα οποία χαρακτηρίζονται ως μία ομάδα συνυφασμένων νημάτων.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Συνεργατικές συλλογές νημάτων

Επιπλέον, υφίσταται η δυνατότητα **εφαρμογής** μιας διαδικασίας **υποβιβασμού δεδομένων** ανάμεσα στα νήματα που ανήκουν σε μία **συλλογή** τύπου **thread_block_tile** ή **coalesced_group** στην οποία χρησιμοποιούνται οι **αριθμητικές** και **δυαδικές** πράξεις **plus**, **less**, **greater**, **bit_and**, **bit_or** και **bit_xor** και πραγματοποιείται από τη συνάρτηση

```
template <typename TyGroup, typename TyArg, typename TyOp>
    auto reduce(const TyGroup& group, TyArg&& val, TyOp&& op) -> decltype(op(val, val));
```

όπου **group** είναι η **συλλογή των νημάτων** που περιέχουν τη μεταβλητή **val**, επί της οποίας θα εφαρμοσθεί η πράξη του **υποβιβασμού** η οποία προσδιορίζεται από το όρισμα **op**

Εάν επιθυμούμε να προχωρήσουμε στον **συγχρονισμό** των νημάτων μιας **συλλογής**, μπορούμε να χρησιμοποιήσουμε κάποια από τις **διαθέσιμες συναρτήσεις** όπως είναι οι **barrier_arrive** και **barrier_wait** που επιτρέπουν την υλοποίηση ενός **συγχρονισμού τύπου χρονικής φραγής** και οι συναρτήσεις **wait** και **wait_prior** που **αναστέλλουν** την λειτουργία των νημάτων, μέχρι την ολοκλήρωση των **ασύγχρονων διαδικασιών μεταφοράς δεδομένων** που πραγματοποιούνται από την **memcpy_async**.

Τέλος, η **CUDA** υποστηρίζει και **συλλογές νημάτων**, οι οποίες σχετίζονται με **πολλαπλές** ομάδες και επιτρέπουν τον **συγχρονισμό** σε **επίπεδο πλέγματος**, δυνατότητα, που υποστηρίζεται από συσκευές με υπολογιστική ικανότητα **6.0** ή μεγαλύτερη.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Συνεργατικές συλλογές νημάτων

Αυτή η δυνατότητα **συγχρονισμού των νημάτων**, προσφέρεται από την συνάρτηση **sync** των αντικειμένων **grid_group**, τα οποία δημιουργούνται με μία δήλωση της μορφής

```
grid_group g = this_grid()
```

Προκειμένου να χρησιμοποιηθεί αυτή η **δυνατότητα** εφόσον αυτή **υποστηρίζεται** από την συσκευή, **αντί να καλέσουμε** την συνάρτηση του πυρήνα με τον **συνήθη τρόπο**, θα τον **καλέσουμε** μέσω της συνάρτησης

```
cudaLaunchCooperativeKernel(const T *func, dim3 gridDim, dim3 blockDim,  
void **args, size_t sharedMem = 0, cudaStream_t stream = 0);
```

Προκειμένου η εφαρμογή μας να εκτελεστεί **με επιτυχία**, θα πρέπει, **πριν από την κλήση** αυτής της συνάρτησης, να ελέγξουμε εάν η συσκευή μας **υποστηρίζει** αυτή την δυνατότητα καλώντας τη συνάρτηση **cudaDeviceGetAttribute** με την μορφή

```
int dev = 0, supportsCoopLaunch = 0;  
cudaDeviceGetAttribute(&supportsCoopLaunch, cudaDevAttrCooperativeLaunch, dev);
```

η οποία θέτει την παράμετρο **supportsCoopLaunch** στην τιμή **1** ή **0**, εάν αυτή η δυνατότητα **υποστηρίζεται ή όχι**, αντίστοιχα



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Δημιουργία και διαχείριση ρευμάτων εντολών

Τα **ρεύματα εντολών** υποστηρίζουν το χαρακτηριστικό του **ταυτοχρονισμού** και ορίζονται ως **ακολουθίες εντολών**, οι οποίες εκτελούνται **με τη σειρά**, η μία μετά την άλλη και ενδεχομένως προέρχονται από **διαφορετικά** νήματα

Ωστόσο, εάν οι εντολές ανήκουν σε **διαφορετικά ρεύματα**, εκτελούνται **χωρίς να ορίζεται** κάποια **συγκεκριμένη** σειρά εκτέλεσης, ενώ στην περίπτωση αυτή, είναι δυνατή και η **ταυτόχρονη** εκτέλεση τους

Εάν οι **εντολές** ανήκουν **στο ίδιο** ρεύμα, η **εκτέλεση** θα πρέπει να πραγματοποιηθεί έτσι ώστε να **ικανοποιούνται** οι **εξαρτήσεις** που υφίστανται ανάμεσά τους, οι οποίες, μπορεί να ορίζονται και ανάμεσα σε **εντολές** που ανήκουν σε **διαφορετικά ρεύματα**.

Η **ολοκλήρωση** των εντολών ενός ρεύματος, πιστοποιείται από την **επιτυχή εκτέλεση** της κατάλληλης **εντολής συγχρονισμού**.

Τα **ρεύματα** αναπαρίστανται από αντικείμενα **cudaStream_t**, ενώ η δημιουργία τους πραγματοποιείται με την συνάρτηση **cudaStreamCreate** εκτός εάν απαιτείται ο ορισμός τιμής **προτεραιότητας**, οπότε καλείται η συνάρτηση **cudaStreamCreateWithPriority** ή κάποιων **παραμέτρων λειτουργίας** οπότε χρησιμοποιείται η συνάρτηση **cudaStreamCreateWithFlags**

Προκειμένου να εντάξουμε τις **πάσης φύσεως εντολές** στα ρεύματα που δημιουργούμε, θα πρέπει να χρησιμοποιήσουμε ως **κατάλληλο όρισμα** σε αυτές τις εντολές, ένα αντικείμενο τύπου **cudaStream_t**.

Εάν στις συναρτήσεις **πυρήνα** και **μεταφοράς δεδομένων**, δεν καθοριστεί κάποιο ρεύμα, ή εάν ως τιμή για αυτό το **επιπλέον** όρισμα, χρησιμοποιηθεί η **μηδενική** τιμή, τότε, κατά την εκτέλεσή τους, χρησιμοποιείται ένα **προεπιλεγμένο ρεύμα**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Δημιουργία και διαχείριση ρευμάτων εντολών

Σε περιπτώσεις κατά τις οποίες, χρησιμοποιούνται **περισσότερα από ένα ρεύματα**, οι εντολές που **περιλαμβάνονται** σε αυτά, μπορούν να εκτελεστούν **παράλληλα** οπότε εγείρονται **ζητήματα συγχρονισμού** που θα πρέπει να **διευθετηθούν**

Δύο **εντολές** οι οποίες ανήκουν σε **διαφορετικά ρεύματα**, δεν είναι σε θέση να εκτελεστούν **ταυτόχρονα**, εάν ανάμεσά τους εκτελούνται διαδικασίες που **εισάγουν εξαρτήσεις** ανάμεσα στα τμήματα της εφαρμογής και περιλαμβάνουν μεταξύ άλλων

- την **δέσμευση** μιας **κλειδωμένης περιοχής μνήμης** στον κεντρικό υπολογιστή
- την **δέσμευση** και την **αρχικοποίηση** περιοχής μνήμης στη συσκευή
- την **αντιγραφή δεδομένων** ανάμεσα σε δύο περιοχές μνήμης της συσκευής

Η **CUDA** προσφέρει τις συναρτήσεις **cudaEventRecond** και **cudaStreamWaitEvent** με τις οποίες είναι δυνατός ο **χειρισμός** των **εξαρτήσεων** ανάμεσα σε **διαφορετικά ρεύματα**, δια της **καταγραφής συμβάντων** και την προσθήκη τους σε **δομή γράφου**.

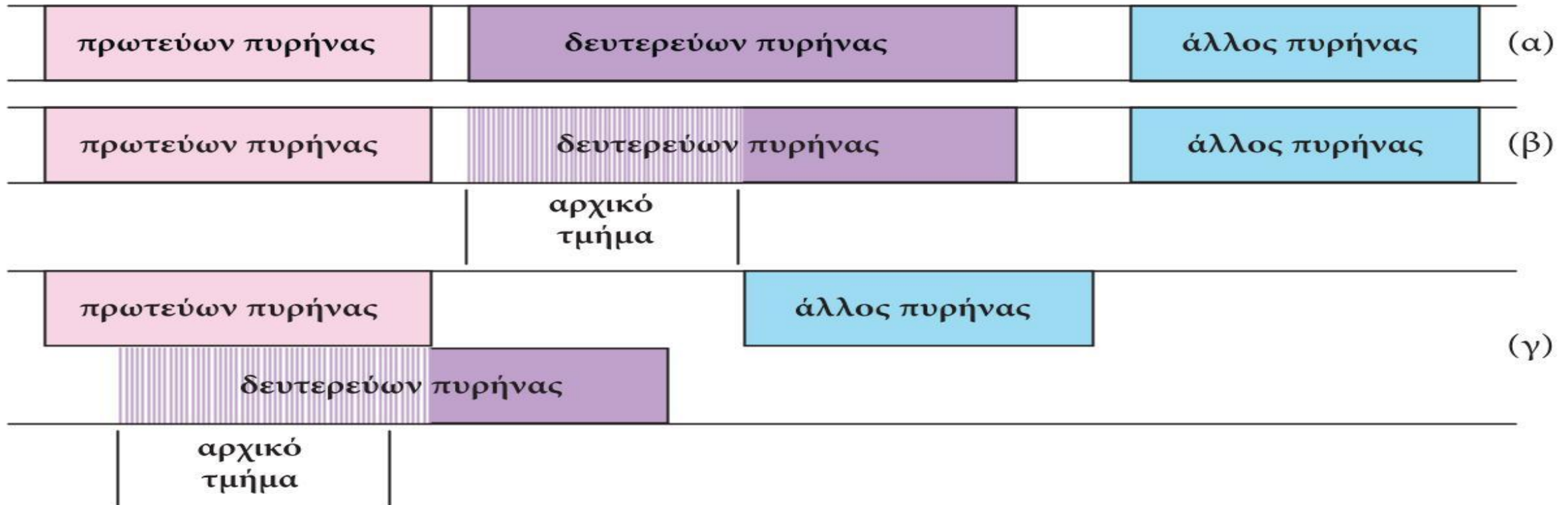
Ο **συγχρονισμός** της εκτέλεσης των **εντολών** που ανήκουν σε **διαφορετικά ρεύματα** πραγματοποιείται ανάλογα με την περίπτωση, από κάποια από τις συναρτήσεις **cudaDeviceSynchronize**, **cudaStreamSynchronize** και **cudaStreamWaitEvent**

Εάν ένα ρεύμα περιέχει δύο πυρήνες **A** και **B** εκ των οποίων ο **B εξαρτάται από τον A** και κατά συνέπεια, θα πρέπει να εκτελεστεί **μετά από αυτόν**, υπάρχει η δυνατότητα **χρονικής επικάλυψης**, σε κάποιο βαθμό, σε περιπτώσεις κατά τις οποίες το ρεύμα περιλαμβάνει διαδικασίες, οι οποίες **δεν εξαρτώνται** από τα αποτελέσματα του πυρήνα **A** και κατά συνέπεια, μπορούν να εκτελεστούν **παράλληλα με αυτόν**. Αυτή η δυνατότητα είναι γνωστή ως **προγραμματική εξαρτημένη εκκίνηση**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Δημιουργία και διαχείριση ρευμάτων εντολών



Στην **προγραμματική εξαρτημένη εκκίνηση**, υφίσταται η δυνατότητα της **χρονικής επικάλυψης** της εκτέλεσης του **πρωτεύοντος** πυρήνα με ένα **υποσύνολο διαδικασιών** του **δευτερεύοντος** πυρήνα εφόσον **δεν υφίστανται** εξαρτήσεις ανάμεσά τους, γεγονός που μπορεί να οδηγήσει σε **σημαντική επιτάχυνση**

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Δημιουργία και διαχείριση ρευμάτων εντολών

<code>cudaCtxResetPersistingL2Cache</code>	<code>/* Resets all persisting lines in cache to normal status</code>	<code>*/</code>
<code>cudaStreamAddCallback</code>	<code>/* Add a callback to a compute stream</code>	<code>*/</code>
<code>cudaStreamAttachMemAsync</code>	<code>/* Attach memory to a stream asynchronously</code>	<code>*/</code>
<code>cudaStreamBeginCapture</code>	<code>/* Begins graph capture on a stream</code>	<code>*/</code>
<code>cudaStreamBeginCaptureToGraph</code>	<code>/* Begins graph capture on a stream to an existing graph</code>	<code>*/</code>
<code>cudaStreamCopyAttributes</code>	<code>/* Copies attributes from source stream to destination stream</code>	<code>*/</code>
<code>cudaStreamCreate</code>	<code>/* Create an asynchronous stream</code>	<code>*/</code>
<code>cudaStreamCreateWithFlags</code>	<code>/* Create an asynchronous stream</code>	<code>*/</code>
<code>cudaStreamCreateWithPriority</code>	<code>/* Create an asynchronous stream with the specified priority</code>	<code>*/</code>
<code>cudaStreamDestroy</code>	<code>/* Destroys and cleans up an asynchronous stream</code>	<code>*/</code>
<code>cudaStreamEndCapture</code>	<code>/* Ends capture on a stream, returning the captured graph</code>	<code>*/</code>
<code>cudaStreamGetAttribute</code>	<code>/* Queries stream attribute</code>	<code>*/</code>
<code>cudaStreamGetCaptureInfo</code>	<code>/* Query a stream's capture state</code>	<code>*/</code>
<code>cudaStreamGetCaptureInfo_v3</code>	<code>/* Query a stream's capture state (12.3+)</code>	<code>*/</code>
<code>cudaStreamGetFlags</code>	<code>/* Query the flags of a stream</code>	<code>*/</code>
<code>cudaStreamGetId</code>	<code>/* Query the Id of a stream</code>	<code>*/</code>
<code>cudaStreamGetPriority</code>	<code>/* Query the priority of a stream</code>	<code>*/</code>
<code>cudaStreamIsCapturing</code>	<code>/* Returns a stream's capture status</code>	<code>*/</code>
<code>cudaStreamQuery</code>	<code>/* Queries an asynchronous stream for completion status</code>	<code>*/</code>
<code>cudaStreamSetAttribute</code>	<code>/* Sets stream attribute</code>	<code>*/</code>
<code>cudaStreamSynchronize</code>	<code>/* Waits for stream tasks to complete</code>	<code>*/</code>
<code>cudaStreamUpdateCaptureDependencies</code>	<code>/* Update the set of dependencies in a capturing stream (11.3+)</code>	<code>*/</code>
<code>cudaStreamUpdateCaptureDependencies_v2</code>	<code>/* Update the set of dependencies in a capturing stream (12.3+)</code>	<code>*/</code>
<code>cudaStreamWaitEvent</code>	<code>/* Make a compute stream wait on an event</code>	<code>*/</code>
<code>cudaThreadExchangeStreamCaptureMode</code>	<code>/* Swaps the stream capture interaction mode for a thread</code>	<code>*/</code>

Οι συναρτήσεις διαχείρισης ρευμάτων της CUDA

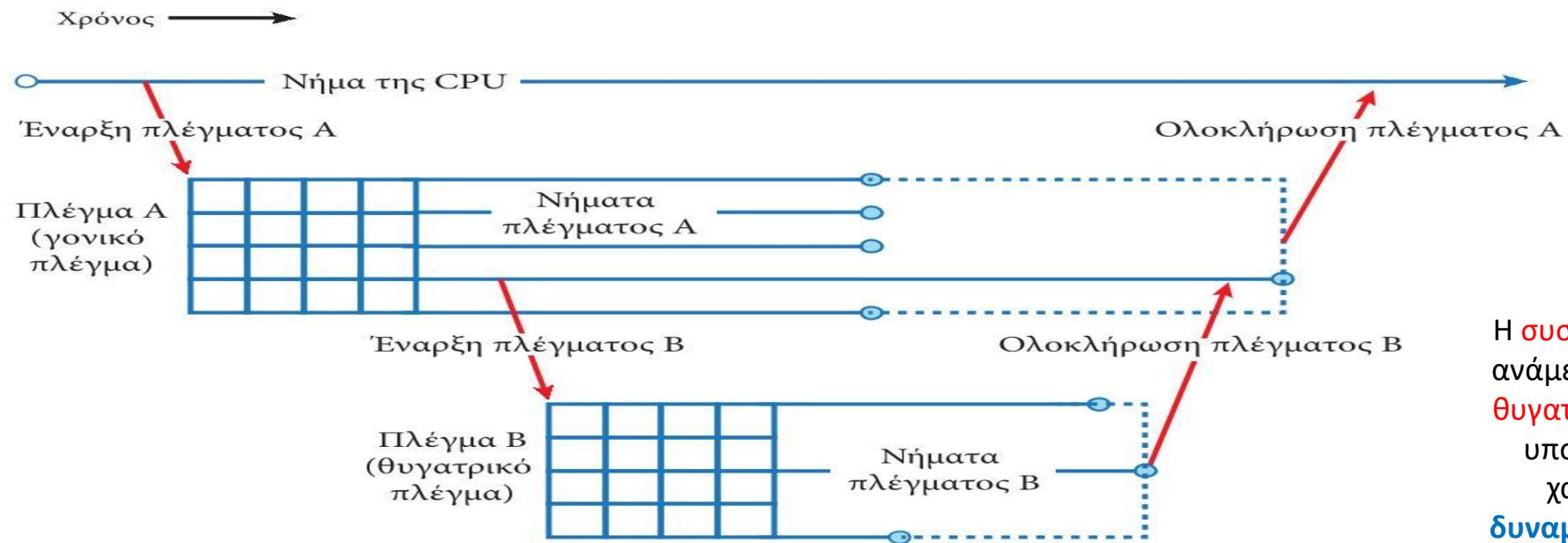


Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Εκτέλεση πυρήνα μέσα από πυρήνα – δυναμικός παραλληλισμός

Η **CUDA** υποστηρίζει το χαρακτηριστικό του **δυναμικού παραλληλισμού**, που ορίζεται ως η **δυνατότητα της εκτέλεσης ενός πυρήνα** μέσα από μία άλλη συνάρτηση πυρήνα, εάν η διάταξη έχει **υπολογιστική ικανότητα μεγαλύτερη** ή ίση από 3.5

Το πλέγμα που δημιουργείται από την **συνάρτηση πυρήνα** που εκτελείται στην **συσκευή**, χαρακτηρίζεται ως **θυγατρικό πλέγμα**, προκειμένου να διακριθεί από το **πλέγμα του πυρήνα** που καλεί αυτή την συνάρτηση, το οποίο ονομάζεται **γονικό πλέγμα** και η λειτουργία του **ολοκληρώνεται**, όταν **ολοκληρωθεί** η λειτουργία όλων των θυγατρικών πλεγμάτων



Η **συσχέτιση** που υφίσταται ανάμεσα στο **γονικό** και στο **θυγατρικό** πλέγμα τα οποία υποστηρίζονται από το χαρακτηριστικό του **δυναμικού παραλληλισμού** της **CUDA**

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Εκτέλεση πυρήνα μέσα από πυρήνα – δυναμικός παραλληλισμός

Η δυνατότητα παραγωγής υπολογιστικού έργου απευθείας στην GPU, μειώνει σε σημαντικό βαθμό την ανάγκη μεταφοράς του ελέγχου της ροής εκτέλεσης του προγράμματος καθώς και των αναγκαίων δεδομένων ανάμεσα στην ΚΜΕ και στην GPU αφού τα νήματα της συσκευής μπορούν να λάβουν σε πραγματικό χρόνο αποφάσεις σχετικά με τις παραμέτρους εκτέλεσης.

Οποιοδήποτε από τα νήματα του γονικού πλέγματος, μπορεί να εκκινήσει διαδικασίες συγχρονισμού με στόχο τον έλεγχο της σειράς εκτέλεσης του θυγατρικού πλέγματος από οποιοδήποτε άλλο νήμα, χρησιμοποιώντας ρεύματα εντολών που και αυτά με τη σειρά τους έχουν δημιουργηθεί από οποιοδήποτε νήμα

Λαμβάνοντας υπόψη πως υπάρχουν πολλά νήματα μέσα στο ίδιο πλέγμα που ξεκινούν τη λειτουργία τους χρησιμοποιώντας το ίδιο ρεύμα, η διάταξη των νημάτων μέσα στο ρεύμα υπαγορεύεται από την διαδικασία της χρονοδρομολόγησης των νημάτων μέσα στο πλέγμα, η οποία μπορεί να ελεγχθεί καταφεύγοντας σε τεχνικές συγχρονισμού

Το γονικό και το θυγατρικό πλέγμα διαμοιράζονται την ίδια καθολική μνήμη και μνήμη σταθερών, αλλά χρησιμοποιούν διαφορετική κοινόχρηστη και τοπική μνήμη. Αμφότερα τα πλέγματα, χαρακτηρίζονται από σύμφωνη πρόσβαση στην καθολική μνήμη και κατά συνέπεια, είναι σε θέση να αντιληφθούν τις όποιες μεταβολές αφορούν στο περιεχόμενό της.

Ωστόσο, ανάμεσά τους υφίσταται μόνο ασθενής συνέπεια, κάτι που σημαίνει, πως δεν είναι εγγυημένο, πως σε κάθε χρονική στιγμή θα διαθέτουν τα ίδια δεδομένα, με την ισχυρή συνέπεια ανάμεσα στα πλέγματα να υφίσταται μόνο κατά την χρονική στιγμή κατά την οποία λαμβάνει χώρα η εκκίνηση της εκτέλεσης του θυγατρικού από το γονικό πλέγμα.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Ένας αποδοτικός τρόπος περιγραφής των λειτουργιών μίας εφαρμογή CUDA όπως είναι οι διαδικασίες μεταφοράς δεδομένων και η εκτέλεση συναρτήσεων πυρήνα, στηρίζεται στην χρήση δομών γράφων, οι οποίες επιτρέπουν στο σύστημα, να προχωρήσει σε διαδικασίες βελτιστοποίησης που δεν είναι δυνατόν να πραγματοποιηθούν, χωρίς αυτές τις δομές.

Σε αυτούς τους γράφους, που αποτελούν ένα εναλλακτικό μοντέλο υποβολής εργασιών για διεκπεραίωση στο περιβάλλον της CUDA, οι κορυφές αντιστοιχούν στις πάσης φύσεως διαδικασίες ενώ οι ακμές αντιστοιχούν στις εξαρτήσεις ανάμεσά τους

Ένας γράφος της CUDA, ορίζεται μία και μοναδική φορά, ενώ στη συνέχεια εκτελείται όσες φορές αυτό είναι αναγκαίο.

Η σωστή χρήση ενός γράφου της CUDA οδηγεί σε ελαχιστοποίηση του χρόνου της προετοιμασίας που απαιτείται για την πραγματοποίηση ενός συνόλου υπολογισμών στην ΚΜΕ, αλλά και στην αύξηση του βαθμού ταυτοχρονισμού, μέσω της χρονικής επικάλυψης διαδικασιών, οι οποίες δεν είναι ανταγωνιστικές ή εξαρτώμενες η μία από την άλλη.

Ωστόσο, η τροποποίηση των εκτελούμενων διαδικασιών, απαιτεί την αντίστοιχη τροποποίηση και την εκ νέου ενεργοποίηση του γράφου, κάτι που ισχύει για εκτεταμένες και ριζικές τροποποιήσεις, όπως είναι η τοπολογία και η μεταβολή του τύπου των κόμβων του γράφου, διαδικασίες οι οποίες οδηγούν σε μείωση του βαθμού της ωφέλειας που αποκομίζεται από την χρήση του



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Η χρήση ενός γράφου απαιτεί την πραγματοποίηση ενός **συνόλου διαδικασιών** και πιο συγκεκριμένα

- τον **ορισμό των κορυφών του γράφου**, καθώς και των **ακμών** που τους συνδέουν
- την κατάλληλη **προετοιμασία** που θα μετατρέψει μία **δομή γράφου της CUDA** σε **εκτελέσιμη** μορφή
- την **εκτέλεση** αυτού του γράφου

Ειδικότερα, η **υποβολή εργασιών προς εκτέλεση** μέσω ενός γράφου, μπορεί να θεωρηθεί ως μία **διαδικασία τριών φάσεων**

- την **φάση του ορισμού** που περιλαμβάνει την **περιγραφή των διαδικασιών** που θα εκτελεστούν μέσω του γράφου καθώς και των **εξαρτήσεων** που υφίστανται ανάμεσά τους
- την **φάση της ενεργοποίησης** που δημιουργεί ένα **στιγμιότυπο της δομής του γράφου** καθώς και μία **εκτελέσιμη εκδοχή** του, **επικυρώνοντας** την δομή του και πραγματοποιώντας τις όποιες **διαδικασίες αρχικοποίησης** είναι αναγκαίες
- την **φάση της εκτέλεσης**, που περιλαμβάνει την **εκτέλεση** του γράφου μέσω ενός **ρεύματος εντολών**

Ένας γράφος δημιουργείται είτε **χρησιμοποιώντας το κατάλληλο σύνολο συναρτήσεων**, είτε **καταφεύγοντας στην χρήση ρευμάτων εντολών**, με την πρώτη τεχνική να είναι **πιο ισχυρή** και να χαρακτηρίζεται από **μεγαλύτερη** περιγραφική ικανότητα



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Η δημιουργία ενός γράφου στην CUDA πραγματοποιείται από τη συνάρτηση `cudaGraphCreate` με πρωτότυπο

```
__host__ cudaError_t cudaGraphCreate (cudaGraph_t* pGraph, unsigned int flags);
```

Μετά την δημιουργία του γράφου προσθέτουμε το κατάλληλο πλήθος κορυφών οι οποίες ανήκουν στους επόμενους τύπους:

- **host node** Σχετίζεται με την εκτέλεση συναρτήσεων στην ΚΜΕ (δομή `cudaHostNodeParams`)
- **kernel node** Σχετίζεται με την εκτέλεση συναρτήσεων πυρήνα στην συσκευή GPU (δομή `cudaKernelNodeParams`)
- **memCpy node** Σχετίζεται με την ανταλλαγή δεδομένων ανάμεσα στην ΚΜΕ και στην GPU (δομή `cudaMemcpy3DParms`)
- **memSet node** Σχετίζεται με την αρχικοποίηση περιοχών μνήμης στην συσκευή GPU (δομή `cudaMemsetParams`)
- **memAlloc node** Σχετίζεται με τις διαδικασίες δέσμευσης μνήμης (δομή `cudaMemAllocNodeParams`)
- **memFree node** Σχετίζεται με τις διαδικασίες αποδέσμευσης μνήμης (δομή `cudaGraphAddMemFreeNode`)
- **event record node** Σχετίζεται με τις διαδικασίες καταγραφής συμβάντων (δομή `cudaGraphAddEventRecordNode`)
- **event wait node** Σχετίζεται με τις διαδικασίες αναμονής εμφάνισης συμβάντων (δομή `cudaGraphAddEventWaitNode`)
- **external semaphores signal node** Σχετίζεται με την χρήση σημαφόρων (δομή `cudaGraphAddExternalSemaphoresSignalNode`)
- **external semaphores wait node** Σχετίζεται με την χρήση σημαφόρων (δομή `cudaGraphAddExternalSemaphoresWaitNode`)
- **child graph node** Σχετίζεται με την προσθήκη ένθετου γράφου σε υφιστάμενο γράφο (δομή `cudaGraphAddChildGraphNode`)
- **empty node** Σχετίζεται με την δημιουργία μιας κενής κορυφής (δομή `cudaGraphAddEmptyNode`)



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Η χρήση των παραπάνω **δομών δεδομένων** επιτρέπει τον **καθορισμό των τιμών των παραμέτρων**, οι οποίες δεν είναι άλλες από αυτές που χρησιμοποιούνται με τον **συνήθη τρόπο**. Μετά την προσθήκη των **κορυφών**, προσθέτουμε στον **γράφο** τις κατάλληλες **ακμές** που **συνδέουν** αυτές τις κορυφές και οι οποίες περιγράφουν **εξαρτήσεις**, καλώντας την συνάρτηση

```
__host__ cudaError_t cudaGraphAddDependencies (cudaGraph_t graph, const cudaGraphNode_t* from,
                                                const cudaGraphNode_t* to, size_t numDependencies);
```

Εάν επιθυμούμε να απομακρύνουμε **κορυφές** και **ακμές**, αυτό μπορεί να γίνει καλώντας τις συναρτήσεις

```
__host__ cudaError_t cudaGraphDestroyNode (cudaGraphNode_t node);
```

```
__host__ cudaError_t cudaGraphRemoveDependencies (cudaGraph_t graph, const cudaGraphNode_t* from,
                                                  const cudaGraphNode_t* to, size_t numDependencies);
```

Η **CUDA** προσφέρει την δυνατότητα **ορισμού** και **χρήσης** δεδομένων που σχετίζονται με τις **ακμές** ενός γράφου, επιτρέπουν την **τροποποίηση των χαρακτηριστικών** της εξάρτησης που περιγράφεται από την **αντίστοιχη** ακμή και αποτελούνται από

- μία **εξερχόμενη θύρα** που προσδιορίζει την **χρονική στιγμή ενεργοποίησης** της ακμής
- μία **εισερχόμενη θύρα** που προσδιορίζει το **τμήμα της κορυφής** που εξαρτάται από μία ακμή του γράφου
- έναν τύπο που επιτρέπει την **τροποποίηση της συσχέτισης** που υφίσταται ανάμεσα σε αυτές στις δύο παραπάνω θύρες



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων – Δομές και συναρτήσεις προσθήκης νέων κόμβων

```
// Creates a host execution node and adds it to a graph
__host__ cudaError_t cudaGraphAddHostNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph,
                                           const cudaGraphNode_t* pDependencies, size_t numDependencies,
                                           const cudaHostNodeParams* pNodeParams )

struct cudaHostNodeParams {
    cudaHostFn_t fn;          /* The function to call when the node executes */
    void * userData; };      /* Argument to pass to the function */

// Creates a kernel execution node and adds it to a graph
__host__ cudaError_t cudaGraphAddKernelNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph,
                                              const cudaGraphNode_t* pDependencies, size_t numDependencies,
                                              const cudaKernelNodeParams* pNodeParams);

struct cudaKernelNodeParams {
    dim3 blockDim;           /* Block dimensions */
    dim3 * extra;            /* Pointer to kernel arguments in the "extra" format */
    void * func;             /* Kernel to launch */
    dim3 gridDim;           /* Grid dimensions */
    dim3 * kernelParams;    /* Array of pointers to individual kernel arguments */
    unsigned int sharedMemBytes; }; /* Dynamic shared-memory size per thread block in bytes */
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων – Δομές και συναρτήσεις προσθήκης νέων κόμβων

```
// Creates a memcpy node and adds it to a graph
__host__ cudaError_t cudaGraphAddMemcpyNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph,
                                              const cudaGraphNode_t* pDependencies, size_t numDependencies,
                                              const cudaMemcpy3DParms* pCopyParams);

struct cudaMemcpy3DParms {
    cudaArray_t dstArray;          /* Destination memory address */
    struct cudaPos dstPos;         /* Destination position offset */
    struct cudaPitchedPtr dstPtr; /* Pitched destination memory address */
    struct cudaExtent extent;      /* Requested memory copy size */
    enum cudaMemcpyKind kind;      /* Type of transfer */
    cudaArray_t srcArray;          /* Source memory address */
    struct cudaPos srcPos;         /* Source position offset */
    struct cudaPitchedPtr srcPtr; /* Pitched source memory address */
};

// Creates a memset node and adds it to a graph
__host__ cudaError_t cudaGraphAddMemsetNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph,
                                              const cudaGraphNode_t* pDependencies, size_t numDependencies,
                                              const cudaMemsetParams* pMemsetParams);

struct cudaMemsetParams {
    void * dst; /* Destination device pointer */
    unsigned int elementSize; /* Size of each element in bytes. Must be 1, 2, or 4 */
    size_t height; /* Number of rows */
    size_t pitch; /* Pitch of destination device pointer. Unused if height is 1 */
    unsigned int value; /* Value to be set */
    size_t width; /* Width of the row in elements */
};
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων – Δομές και συναρτήσεις προσθήκης νέων κόμβων

```
// Creates an allocation node and adds it to a graph
__host__ cudaError_t cudaGraphAddMemAllocNode (cudaGraphNode_t* pNode, cudaGraph_t graph,
    const cudaGraphNode_t* pDependencies, size_t numDependencies,
    cudaMemAllocNodeParams* nodeParams )

struct cudaMemAllocNodeParams {
    size_t accessDescCount;           /* in: Number of 'accessDescs's */
    cudaMemAccessDesc * accessDescs; /* in: number of memory access descriptors */
    size_t bytesize;                  /* in: size in bytes of the requested allocation */
    void * dptr;                      /* out: address of the allocation returned by CUDA */
    struct cudaMemPoolProps poolProps; /* in: location where the allocation should reside */
}

// Creates a memory free node and adds it to a graph
__host__ cudaError_t cudaGraphAddMemFreeNode (cudaGraphNode_t* pNode, cudaGraph_t graph,
    const cudaGraphNode_t* pDependencies, size_t numDependencies, void* dptr);

// Creates a child graph node and adds it to a graph
__host__ cudaError_t cudaGraphAddChildGraphNode (cudaGraphNode_t* pNode, cudaGraph_t graph,
    const cudaGraphNode_t* pDependencies,
    size_t numDependencies, cudaGraph_t childGraph);
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων – Δομές και συναρτήσεις προσθήκης νέων κόμβων

```
// Creates an empty node and adds it to a graph
__host__ cudaError_t cudaGraphAddEmptyNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph,
const cudaGraphNode_t* pDependencies, size_t numDependencies )

// Adds a node of arbitrary type to a graph
__host__ cudaError_t cudaGraphAddNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph,
const cudaGraphNode_t* pDependencies, size_t numDependencies,
cudaGraphNodeParams* nodeParams);

// Creates an event record node and adds it to a graph
__host__ cudaError_t cudaGraphAddEventRecordNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph,
const cudaGraphNode_t* pDependencies, size_t numDependencies, cudaEvent_t event);
// Creates an event wait node and adds it to a graph
__host__ cudaError_t cudaGraphAddEventWaitNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph,
const cudaGraphNode_t* pDependencies, size_t numDependencies, cudaEvent_t event )

struct cudaGraphNodeParams {
    struct cudaMemAllocNodeParamsV2 alloc;           /* Memory allocation node params */
    struct cudaConditionalNodeParams conditional;     /* Conditional node params */
    struct cudaEventRecordNodeParams eventRecord;    /* Event record node params */
    struct cudaEventWaitNodeParams eventWait;        /* Event wait node params */
    struct cudaExternalSemaphoreSignalNodeParamsV2 extSemSignal; /* External semaphore signal node params */
    struct cudaExternalSemaphoreWaitNodeParamsV2 extSemWait;    /* External semaphore wait node params */
    struct cudaMemFreeNodeParams free;               /* Memory free node params */
    struct cudaChildGraphNodeParams graph;           /* Child graph node params */
    struct cudaHostNodeParamsV2 host;               /* Host node params */
    struct cudaKernelNodeParamsV2 kernel;           /* Kernel node params */
    struct cudaMemcpyNodeParams memcpy;             /* Memcpy node params */
    struct cudaMemsetParamsV2 memset;               /* Memset node params */
    int reserved0[3]; /* Reserved. */                /* Must be zero */
    long long reserved1[29]; /* Unused bytes must be zero */
    long long reserved2; /* Reserved bytes. Must be zero */
    enum cudaGraphNodeType type; };                  /* Type of the node */
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων – Συναρτήσεις διαχείρισης γράφων

<code>cudaDeviceGetGraphMemAttribute</code>	<code>/* Query asynchronous allocation graph attributes */</code>
<code>cudaDeviceGraphMemTrim</code>	<code>/* Free unused memory cached for use with graphs */</code>
<code>cudaDeviceSetGraphMemAttribute</code>	<code>/* Set asynchronous allocation graph attributes */</code>
<code>cudaGetCurrentGraphExec</code>	<code>/* Get the currently running device graph id */</code>
<code>cudaGraphAddChildGraphNode</code>	<code>/* Creates child graph node and adds it to graph */</code>
<code>cudaGraphAddDependencies</code>	<code>/* Adds dependency edges to a graph */</code>
<code>cudaGraphAddDependencies_v2</code>	<code>/* Adds dependency edges to a graph. (12.3+) */</code>
<code>cudaGraphAddEmptyNode</code>	<code>/* Creates empty node and adds it to a graph */</code>
<code>cudaGraphAddEventRecordNode</code>	<code>/* Creates event record node and adds it to graph */</code>
<code>cudaGraphAddEventWaitNode</code>	<code>/* Creates event wait node and adds it to a graph */</code>
<code>cudaGraphAddExternalSemaphoresSignalNode</code>	<code>/* Creates & adds external semaphore signal node */</code>
<code>cudaGraphAddExternalSemaphoresWaitNode</code>	<code>/* Creates & adds external semaphore wait node */</code>
<code>cudaGraphAddHostNode</code>	<code>/* Creates & adds a host execution node */</code>
<code>cudaGraphAddKernelNode</code>	<code>/* Creates & adds a kernel execution node */</code>
<code>cudaGraphAddMemAllocNode</code>	<code>/* Creates & adds an allocation node */</code>
<code>cudaGraphAddMemFreeNode</code>	<code>/* Creates & adds a memory free node */</code>
<code>cudaGraphAddMemcpyNode</code>	<code>/* Creates & adds a memcpy node */</code>
<code>cudaGraphAddMemcpyNode1D</code>	<code>/* Creates & adds a 1D memcpy node */</code>
<code>cudaGraphAddMemcpyNodeFromSymbol</code>	<code>/* Creates & adds a memcpy (from symbol) node */</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Χρησιμοποιώντας δομές γράφων – Συναρτήσεις διαχείρισης γράφων

<code>cudaGraphAddMemcpyNodeToSymbol</code>	<code>/* Creates & adds a memcpy (to symbol) node</code>	<code>*/</code>
<code>cudaGraphAddMemsetNode</code>	<code>/* Creates a memset node and adds it to a graph</code>	<code>*/</code>
<code>cudaGraphAddNode</code>	<code>/* Adds a node of arbitrary type to a graph</code>	<code>*/</code>
<code>cudaGraphAddNode_v2</code>	<code>/* Adds a node of arbitrary type to a graph</code>	<code>*/</code>
<code>cudaGraphChildGraphNodeGetGraph</code>	<code>/* Gets a handle to the graph of a child graph</code>	<code>*/</code>
<code>cudaGraphClone</code>	<code>/* Clones a graph</code>	<code>*/</code>
<code>cudaGraphConditionalHandleCreate</code>	<code>/* Create a conditional handle</code>	<code>*/</code>
<code>cudaGraphCreate</code>	<code>/* Creates a graph</code>	<code>*/</code>
<code>cudaGraphDebugDotPrint</code>	<code>/* Write a DOT file describing graph structure</code>	<code>*/</code>
<code>cudaGraphDestroy</code>	<code>/* Destroys a graph</code>	<code>*/</code>
<code>cudaGraphDestroyNode</code>	<code>/* Remove a node from the graph</code>	<code>*/</code>
<code>cudaGraphEventRecordNodeGetEvent</code>	<code>/* Returns the event of an event record node</code>	<code>*/</code>
<code>cudaGraphEventRecordNodeSetEvent</code>	<code>/* Sets an event record node's event</code>	<code>*/</code>
<code>cudaGraphEventWaitNodeGetEvent</code>	<code>/* Returns the event of an event wait node</code>	<code>*/</code>
<code>cudaGraphEventWaitNodeSetEvent</code>	<code>/* Sets an event wait node's event</code>	<code>*/</code>
<code>cudaGraphExecChildGraphNodeSetParams</code>	<code>/* Updates node parameters in a child graph node</code>	<code>*/</code>
<code>cudaGraphExecDestroy</code>	<code>/* Destroys an executable graph</code>	<code>*/</code>
<code>cudaGraphExecEventRecordNodeSetEvent</code>	<code>/* Sets the event for an event record node</code>	<code>*/</code>
<code>cudaGraphExecEventWaitNodeSetEvent</code>	<code>/* Sets the event for an event wait node</code>	<code>*/</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων – Συναρτήσεις διαχείρισης γράφων

<code>cudaGraphExecExternalSemaphoresSignalNodeSetParams</code>	<code>/* Sets params for an external sem signal node</code>	<code>*/</code>
<code>cudaGraphExecExternalSemaphoresWaitNodeSetParams</code>	<code>/* Sets params for an external sem wait node</code>	<code>*/</code>
<code>cudaGraphExecGetFlags</code>	<code>/* Query the instantiation flags of an exec graph</code>	<code>*/</code>
<code>cudaGraphExecHostNodeSetParams</code>	<code>/* Sets params for a host node in a graphExec</code>	<code>*/</code>
<code>cudaGraphExecKernelNodeSetParams</code>	<code>/* Sets params for a kernel node in a graphExec</code>	<code>*/</code>
<code>cudaGraphExecMemcpyNodeSetParams</code>	<code>/* Sets params for a memcpy node in a graphExec</code>	<code>*/</code>
<code>cudaGraphExecMemcpyNodeSetParams1D</code>	<code>/* Sets params for a memcpy node (1-d copy)</code>	<code>*/</code>
<code>cudaGraphExecMemcpyNodeSetParamsFromSymbol</code>	<code>/* Sets params for a memcpy (from symbol) node</code>	<code>*/</code>
<code>cudaGraphExecMemcpyNodeSetParamsToSymbol</code>	<code>/* Sets params for a memcpy (to symbol) node</code>	<code>*/</code>
<code>cudaGraphExecMemsetNodeSetParams</code>	<code>/* Sets params for a memset node in a graphExec</code>	<code>*/</code>
<code>cudaGraphExecNodeSetParams</code>	<code>/* Update's a graph node's params</code>	<code>*/</code>
<code>cudaGraphExecUpdate</code>	<code>/* Check if an executable graph can be updated</code>	<code>*/</code>
<code>cudaGraphExternalSemaphoresSignalNodeGetParams</code>	<code>/* Returns an external sem signal node's params</code>	<code>*/</code>
<code>cudaGraphExternalSemaphoresSignalNodeSetParams</code>	<code>/* Sets an external sem signal node's params</code>	<code>*/</code>
<code>cudaGraphExternalSemaphoresWaitNodeGetParams</code>	<code>/* Returns an external sem wait node's parameters</code>	<code>*/</code>
<code>cudaGraphExternalSemaphoresWaitNodeSetParams</code>	<code>/* Sets an external sem wait node's parameters</code>	<code>*/</code>
<code>cudaGraphGetEdges</code>	<code>/* Returns a graph's dependency edges</code>	<code>*/</code>
<code>cudaGraphGetEdges_v2</code>	<code>/* Returns a graph's dependency edges (12.3+)</code>	<code>*/</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων – Συναρτήσεις διαχείρισης γράφων

cudaGraphGetNodes
cudaGraphGetRootNodes
cudaGraphHostNodeGetParams
cudaGraphHostNodeSetParams
cudaGraphInstantiate
cudaGraphInstantiateWithFlags
cudaGraphInstantiateWithParams
cudaGraphKernelNodeCopyAttributes
cudaGraphKernelNodeGetAttribute
cudaGraphKernelNodeGetParams
cudaGraphKernelNodeSetAttribute
cudaGraphKernelNodeSetParams
cudaGraphLaunch
cudaGraphMemAllocNodeGetParams
cudaGraphMemFreeNodeGetParams
cudaGraphMemcpyNodeGetParams
cudaGraphMemcpyNodeSetParams
cudaGraphMemcpyNodeSetParams1D

```
/* Returns a graph's nodes */  
/* Returns a graph's root nodes */  
/* Returns a host node's parameters */  
/* Sets a host node's parameters */  
/* Creates an executable graph from a graph */  
/* Creates an executable graph from a graph */  
/* Creates an executable graph from a graph */  
/* Copies attributes from source to dest node */  
/* Queries node attribute */  
/* Returns a kernel node's parameters */  
/* Sets node attribute */  
/* Sets a kernel node's parameters */  
/* Launches an executable graph in a stream */  
/* Returns a memory alloc node's parameters */  
/* Returns a memory free node's parameters */  
/* Returns a memcpy node's parameters */  
/* Sets a memcpy node's parameters */  
/* Sets a memcpy node's params (1-d copy) */
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων – Συναρτήσεις διαχείρισης γράφων

<code>cudaGraphMemcpyNodeSetParamsFromSymbol</code>	<code>/* Sets a memcpy node's params (copy from symbol) */</code>
<code>cudaGraphMemcpyNodeSetParamsToSymbol</code>	<code>/* Sets a memcpy node's params (copy to symbol) */</code>
<code>cudaGraphMemsetNodeGetParams</code>	<code>/* Returns a memset node's parameters */</code>
<code>cudaGraphMemsetNodeSetParams</code>	<code>/* Sets a memset node's parameters */</code>
<code>cudaGraphNodeFindInClone</code>	<code>/* Finds a cloned version of a node */</code>
<code>cudaGraphNodeGetDependencies</code>	<code>/* Returns a node's dependencies */</code>
<code>cudaGraphNodeGetDependencies_v2</code>	<code>/* Returns a node's dependencies (12.3+) */</code>
<code>cudaGraphNodeGetDependentNodes</code>	<code>/* Returns a node's dependent nodes */</code>
<code>cudaGraphNodeGetDependentNodes_v2</code>	<code>/* Returns a node's dependent nodes (12.3+) */</code>
<code>cudaGraphNodeGetEnabled</code>	<code>/* Query whether a node in a graphExec is enabled */</code>
<code>cudaGraphNodeGetType</code>	<code>/* Returns a node's type */</code>
<code>cudaGraphNodeSetEnabled</code>	<code>/* Enables or disables a node in a graphExec */</code>
<code>cudaGraphNodeSetParams</code>	<code>/* Update's a graph node's parameters */</code>
<code>cudaGraphReleaseUserObject</code>	<code>/* Release a user object reference from a graph */</code>
<code>cudaGraphRemoveDependencies</code>	<code>/* Removes dependency edges from a graph */</code>
<code>cudaGraphRemoveDependencies_v2</code>	<code>/* Removes dependency edges from a graph (12.3+) */</code>
<code>cudaGraphRetainUserObject</code>	<code>/* Retain a reference to a user object from graph */</code>
<code>cudaGraphSetConditional</code>	<code>/* Sets the condition value for conditional node */</code>
<code>cudaGraphUpload</code>	<code>/* Uploads an executable graph in a stream */</code>
<code>cudaUserObjectCreate</code>	<code>/* Create a user object */</code>
<code>cudaUserObjectRelease</code>	<code>/* Release a reference to a user object */</code>
<code>cudaUserObjectRetain</code>	<code>/* Retain a reference to a user object */</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Ο **δεύτερος τρόπος** κατασκευής ενός γράφου στην **CUDA** συνίσταται στην **καταγραφή** όλων των διαδικασιών που θα πρέπει να πραγματοποιήσουμε, δια της χρήσεως ενός **ρεύματος εντολών** και στην **αυτόματη δημιουργία** ενός γράφου, με τη συνάρτηση

```
__host__ cudaError_t cudaStreamBeginCapture (cudaStream_t stream, cudaStreamCaptureMode mode);
```

με το όρισμα **mode**, να προσδιορίζει το **είδος των κλήσεων** που επιτρέπεται να πραγματοποιηθούν κατά τη διάρκεια αυτής της περιόδου καταγραφής και με την **προεπιλεγμένη** συμπεριφορά να **μην επιτρέπει** την (**μη ασφαλή**) καταγραφή συμβάντων

Οι διαδικασίες που **καταγράφονται** με αυτόν τον τρόπο, **δεν δρομολογούνται** για εκτέλεση, αλλά καταχωρούνται ως **κορυφές** σε μία **εσωτερική δομή γράφου**, ενώ η **καταγραφή** ολοκληρώνεται καλώντας τη συνάρτηση **cudaStreamEndCapture** η οποία **επιστρέφει** τον γράφο που έχει δημιουργηθεί από αυτή τη διαδικασία.

Από τις δύο παραπάνω **τεχνικές κατασκευής γράφου**, η κατασκευή δια της χρήσεως των **κατάλληλων συναρτήσεων**, αποτελεί την **πλέον κατάλληλη επιλογή**, αφού μας επιτρέπει να **αναδείξουμε** τον **υποκείμενο παραλληλισμό**, σε αντίθεση με την κατασκευή του γράφου **μέσω ρεύματος εντολών**, η οποία σε καμία περίπτωση, **δεν προσφέρει** αυτή την ευελιξία.

Ωστόσο, η **CUDA** επιτρέπει την **καταγραφή διαδικασιών** χρησιμοποιώντας **πολλαπλά ρεύματα**, ενώ μπορούμε να **ζητήσουμε** από ένα ρεύμα να **αναμείνει** κάποιο συμβάν που σχετίζεται **με άλλο ρεύμα**, καλώντας τη συνάρτηση **cudaStreamWaitEvent**.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Δέσμευση και αποδέσμευση μνήμης σε δομές γράφων Οι περιοχές μνήμης που δεσμεύονται / αποδεσμεύονται μέσω των κορυφών **memAlloc** και **memFree** χαρακτηρίζονται από **εικονικές διευθύνσεις**, οι οποίες δημιουργούνται κατά την φάση της δημιουργίας της κορυφής του γράφου και είναι **σταθερές** για **όλη την διάρκεια** της ζωής του γράφου

Αντίθετα, το **περιεχόμενο** της μνήμης στο οποίο αναφέρονται αυτές οι διευθύνσεις, **δεν είναι σταθερό**, αλλά μπορεί να **μεταβάλλεται** κατά τη διάρκεια της εφαρμογής.

Οι **κορυφές μνήμης** ενός γράφου, διατάσσονται μέσω **ακμών** που υπαγορεύουν τις **υφιστάμενες εξαρτήσεις**, ενώ θα πρέπει να **διασφαλιστεί** πως οι λειτουργίες που σχετίζονται με την **προσπέλαση της μνήμης** ενός γράφου, είναι **διατεταγμένες** ως προς τη **χρονική στιγμή εκτέλεσης** κατά τη διάρκεια του χρόνου ανάμεσα στις διαδικασίες **δέσμευσης** και **αποδέσμευσης** μνήμης.

Η **CUDA** επιτρέπει την χρήση **τεχνικών βελτιστοποίησης** των παραπάνω διαδικασιών, μέσω **επαναχρησιμοποίησης** της **εικονικής** και της **φυσικής** μνήμης. Ειδικότερα

- υφίσταται η **δυνατότητα** χρήσης των **ίδιων περιοχών εικονικών διευθύνσεων** από **διαφορετικές** διαδικασίες δέσμευσης μνήμης, εφόσον οι **χρονικές διάρκειες** κατά τις οποίες αυτές υφίστανται, **δεν** χαρακτηρίζονται από **χρονική επικάλυψη**
- υφίσταται η **δυνατότητα** χρήσης της **ίδιας περιοχής φυσικής μνήμης**, από **διαφορετικές δομές γράφων**, για **διακριτές** μεταξύ τους **διαδικασίες δέσμευσης**, υπό την προϋπόθεση πως αυτές **δεν εκτελούνται** ταυτόχρονα.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Εκτέλεση γράφων από τη συσκευή Η **CUDA** προσφέρει την **δυνατότητα εκτέλεσης** ενός γράφου, **απευθείας** στην συσκευή, κάτι που επιτρέπει την εφαρμογή μιας **διαδικασίας δυναμικού ελέγχου ροής**. Ένας γράφος **GPU**, μπορεί να εκτελεστεί τόσο στην **KME** όσο και στην **συσκευή**, ενώ ένας γράφος **CPU**, μπορεί να εκτελεστεί **μόνο** στην **KME**.

Η **ταυτόχρονη** εκτέλεση του **ίδιου γράφου GPU**, στην **ίδια συσκευή** δεν είναι επιτρεπτή και οδηγεί στην **εμφάνιση σφάλματος**, ενώ η **ταυτόχρονη** εκτέλεση ενός **γράφου GPU** στην συσκευή και στην **KME**, οδηγεί σε **απροσδιόριστη** συμπεριφορά

Ένας γράφος χαρακτηρίζεται ως **γράφος συσκευής**, καλώντας τη συνάρτηση **cudaGraphInstantiate**, η οποία, όπως αναφέραμε, προκαλεί την **ενεργοποίηση** του γράφου, με την σημαία **cudaGraphInstantiateFlagDeviceLaunch** κάτι που γίνεται εφόσον οι **κόμβοι** του γράφου βρίσκονται όλοι σε μία **απλή συσκευή** ενώ είναι τέτοιοι ώστε να **σχετίζονται με**

- την **εκτέλεση πυρήνων** (kernel nodes)
- την **αρχικοποίηση διευθύνσεων μνήμης** (memset nodes)
- την **ανταλλαγή δεδομένων** (memcpy nodes)
- την **ένθεση γράφων** (child graph nodes)

Η χρήση του χαρακτηριστικού του **δυναμικού παραλληλισμού** σε αυτές τις περιπτώσεις, **δεν είναι επιτρεπτή**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Εφόσον **ισχύουν τα παραπάνω**, ο γράφος αρχικά θα πρέπει να **φορτωθεί** στη συσκευή, έτσι ώστε να **γνωστοποιηθούν** στο σύστημα και να **δεσμευτούν** οι **υπολογιστικοί πόροι** που **απαιτούνται** για την εκτέλεσή του, κάτι που γίνεται

- είτε από τον **προγραμματιστή** δια της χρήσεως της συνάρτησης **cudaGraphUpload**
- είτε κατά την **φάση της ενεργοποίησης** με τη συνάρτηση **cudaGraphInstantiateWithParams**
- είτε από το **σύστημα**, εκτελώντας τον γράφο στον **κεντρικό επεξεργαστή**

Η εκτέλεση ενός **γράφου συσκευής**, πραγματοποιείται με την συνάρτηση **cudaGraphLaunch**, τόσο από την **KME** μέσα από μία **άλλη δομή γράφου** όσο και από την **GPU** δια της χρήσεως κάποιου **νήματος**. Η εκτέλεση πραγματοποιείται δια της χρήσεως **ειδικών ρευμάτων εντολών** και χρησιμοποιώντας κάποια από τις επόμενες **τρεις** δυνατότητες:

Ρεύμα τύπου cudaStreamGraphFireandForget (Fire and Forget Launch Mode), όπου το **αίτημα για εκτέλεση** του γράφου της συσκευής, υποβάλλεται **απευθείας** στην **GPU** από τον γράφο που έχει αναλάβει αυτή την διαδικασία και στη συνέχεια, οι δύο γράφοι που σχετίζονται με συσχέτιση **γονέα - παιδιού**, εκτελούνται **παράλληλα** και **ανεξάρτητα** ο ένας από τον άλλο.

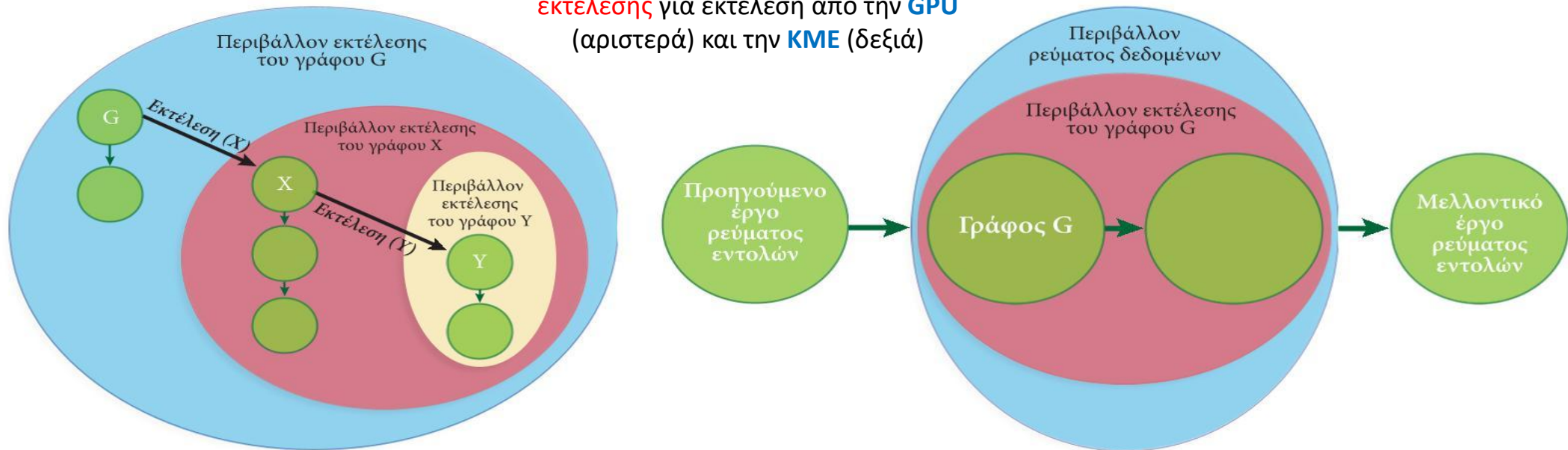
Ένας **θυγατρικός γράφος** που δημιουργείται με τον αυτόν τον τρόπο, μπορεί **με την σειρά του** να δημιουργήσει τους δικούς του **θυγατρικούς γράφους** και κατά συνέπεια, είναι δυνατή η δημιουργία μίας **δεντρικής ιεραρχίας γράφων**, η οποία χαρακτηρίζει επίσης και τα σχετιζόμενα **περιβάλλοντα εκτέλεσης**, έτσι ώστε να μιλούμε για **γονικό** και για **θυγατρικό περιβάλλον εκτέλεσης**.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Ιεραρχία γράφων και περιβαλλόντων
εκτέλεσης για εκτέλεση από την GPU
(αριστερά) και την KME (δεξιά)

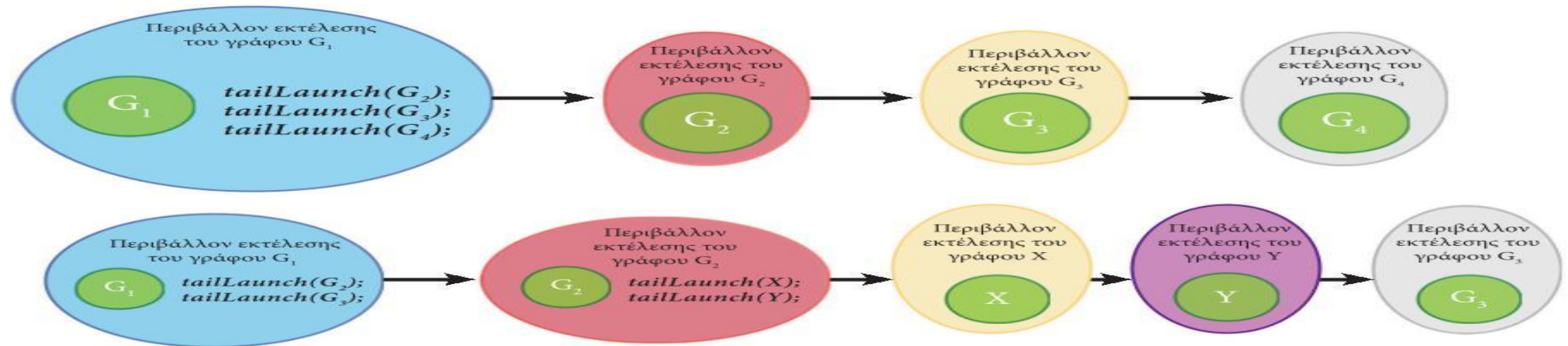


Ρεύμα τύπου **cudaStreamGraphTailLaunch** (Tail Launch Mode), όπου το περιβάλλον της CUDA προσφέρει την δυνατότητα συγχρονισμού για τους γράφους GPU και πιο συγκεκριμένα, μία λειτουργικότητα, παρόμοια με αυτή που προσφέρεται από τις συναρτήσεις **cudaDeviceSynchronize** και **cudaStreamSynchronize**, για τους γράφους CPU

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Χρησιμοποιώντας δομές γράφων

Ένας **γράφος** μπορεί να διαθέτει πολλούς **θυγατρικούς** γράφους προς **εκτέλεση** τοποθετημένους σε μία **ουρά εκτέλεσης**, προκειμένου να εκτελεστούν **ο ένας μετά τον άλλο**, ένας γράφος **κάθε φορά** σύμφωνα με την **πολιτική FIFO**



Η **σειρά εκτέλεσης γράφων** μέσω ρεύματος `cudaStreamGraphTailLaunch` για την περίπτωση **γονικού** γράφου (επάνω) και **πολλών γονικών** γράφων (κάτω)

Ρεύμα τύπου `cudaStreamGraphFireAndForgetAsSibling` (Fire and Forget as Sibling Mode), όπου ο **γράφος** που εκτελείται, **δεν** αποτελεί **θυγατρικό γράφο** του γράφου που τον ενεργοποίησε, αλλά **θυγατρικό γράφο** του **γονικού περιβάλλοντος** αυτού του γράφου και κατά συνέπεια, μπορεί να θεωρηθεί ως **αδελφικός γράφος** του γράφου που τον έχει **ενεργοποιήσει**

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Διαχείριση συμβάντων

```
/* The functions of the event API in CUDA */
cudaEventCreate          /* Creates an event object */
cudaEventCreateWithFlags /* Creates an event object with the specified flags */
cudaEventDestroy        /* Destroys an event object */
cudaEventElapsedTime    /* Computes the elapsed time between events */
cudaEventQuery          /* Queries an event's status */
cudaEventRecord          /* Records an event */
cudaEventRecordWithFlags /* Records an event */
cudaEventSynchronize    /* Waits for an event to complete */

/* The flags used in the event API functions */
cudaEventDefault /* Default event creation flag */
cudaEventBlockingSync /* Specifies that event should use blocking synchronization */
cudaEventDisableTiming /* Specifies that the created event does not need to record timing data */
cudaEventInterprocess /* Specifies that the created event may be used as an interprocess event */

cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start, 0);
multiply(cpuA, cpuB, cpuC, &config);
cudaEventRecord(stop, 0);
cudaEventSynchronize(stop);
cudaEventElapsedTime(&elapsedTime, start, stop);
cudaEventDestroy(start);
cudaEventDestroy(stop);
printf("GPU Time elapsed: %f milliseconds \n\n", elapsedTime);
```

Οι συναρτήσεις διαχείρισης συμβάντων της CUDA

Ο τρόπος καταμέτρησης της χρονικής διάρκειας της εκτέλεσης του κώδικα μιας εφαρμογής CUDA εκπεφρασμένη σε δευτερόλεπτα

← αναμονή της ολοκλήρωσης των διαδικασιών που σχετίζονται με το συμβάν stop

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Κατά καιρούς έχουν αναπτυχθεί **στρατηγικές** που διασφαλίζουν πως η **απόδοση της εφαρμογής** θα είναι **αρκετά υψηλή** και οι οποίες νοούνται ως **τεχνικές βελτιστοποίησης** του τρόπου ανάπτυξης της εφαρμογής. Αυτές οι **στρατηγικές** περιλαμβάνουν

- την **μεγιστοποίηση** του **βαθμού παραλληλισμού**, έτσι ώστε να επιτευχθεί η **καλύτερη δυνατή** χρήση **υπολογιστικών πόρων**
- την **βελτιστοποίηση** της **χρήσης της μνήμης** και των **εντολών** του συστήματος με στόχο την **μέγιστη δυνατή ρυθμαπόδοση**
- την **ελαχιστοποίηση** του **ρυθμού ανταλλαγής σελίδων μνήμης**, ανάμεσα στην πραγματική και στην εικονική μνήμη

Η **μεγιστοποίηση** του **βαθμού παραλληλισμού**, μπορεί να πραγματοποιηθεί σε **δύο διαφορετικά** επίπεδα αφαίρεσης:

- στο **επίπεδο της εφαρμογής**, όπου θα πρέπει να καταφύγουμε σε **καλές προγραμματιστικές τεχνικές**, όσον αφορά στο χαρακτηριστικό του **συγχρονισμού των νημάτων** που εκτελούνται, καθώς και της **διαδικασίας ανταλλαγής δεδομένων** ανάμεσα στις μνήμες της **KME** και της **GPU**
- στο **επίπεδο της συσκευής και του πολυεπεξεργαστή**, όπου θα πρέπει να επιλέξουμε με τον **κατάλληλο** τρόπο, τις **παραμέτρους εκτέλεσης του πυρήνα**, έτσι ώστε αυτός να προχωρήσει σε **όσο το δυνατόν** πιο αποδοτική χρήση της δομής του υποκειμένου **υπολογιστικού συστήματος**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Αναφορικά με τον **συγχρονισμό των νημάτων**, υπάρχουν **δύο** περιπτώσεις που συναντώνται στην πράξη, ανάλογα με το εάν τα εμπλεκόμενα νήματα, ανήκουν **στην ίδια** ή **σε διαφορετικές** ομάδες νημάτων

Στην **πρώτη** περίπτωση, τα νήματα **ανταλλάσσουν δεδομένα** μεταξύ τους δια μέσου της **κοινόχρηστης μνήμης**, μέσα από την **ίδια κλήση της συνάρτησης του πυρήνα**, ενώ ο **συγχρονισμός** τους πραγματοποιείται καλώντας τη συνάρτηση **__syncthreads**.

Στην **δεύτερη** περίπτωση, τα νήματα ανταλλάσσουν δεδομένα μέσω της **καθολικής μνήμης** της συσκευής και χρησιμοποιώντας **δύο κλήσεις συναρτήσεων πυρήνα**, εκ των οποίων η πρώτη πραγματοποιεί **διαδικασία ανάγνωσης**, ενώ η δεύτερη πραγματοποιεί **διαδικασία εγγραφής** στην **καθολική μνήμη**, προσέγγιση που θα πρέπει **να αποφεύγεται**.

Βελτιώσεις μπορούν να πραγματοποιηθούν και στο **επίπεδο του πολυεπεξεργαστή**, με στόχο την **μεγιστοποίηση** του βαθμού της **παράλληλης εκτέλεσης** των νημάτων ανάμεσα στις **υπολογιστικές μονάδες** που συνιστούν έναν τέτοιο επεξεργαστή.

Αυτή η **μεγιστοποίηση** του **βαθμού χρήσης του επεξεργαστή**, είναι **άρρηκτα συνδεδεμένη** με το **πλήθος** των **στημονιών** που εκτελούνται σε **κάθε** χρονική στιγμή. Η **βέλτιστη απόδοση** επιτυγχάνεται, όταν **όλοι οι χρονοδρομολογητές** είναι σε θέση να επιλέξουν κάποια εντολή την οποία θα αναθέσουν σε κάποιο στημόνι για εκτέλεση, έτσι ώστε αυτή η **εμπλεκόμενη αναπόφευκτη χρονική υστέρηση**, να περάσει **απαρατήρητη** στον **μέγιστο** δυνατό βαθμό.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Ο πιο συνηθισμένος λόγος για τον οποίο ένα στημόνι **δεν είναι έτοιμο** να εκτελέσει την **επόμενη** εντολή, είναι διότι οι τελεστές που χρησιμοποιούνται ως **είσοδοι** σε αυτή την εντολή, **δεν είναι** ακόμη διαθέσιμοι.

- εάν οι **τελεστές** προέρχονται από **καταχωρητές**, οπότε η υστέρηση οφείλεται στις **εξαρτήσεις που υφίστανται** ανάμεσα σε αυτούς τους καταχωρητές, η τιμή αυτής της υστέρησης είναι **ίση** με τον **χρόνο εκτέλεσης** της προηγούμενης εντολής
- εάν οι **τελεστές** της εντολής, προέρχονται από **διεύθυνση μνήμης** που βρίσκεται **έξω** από το ολοκληρωμένο κύκλωμα, η χρονική υστέρηση είναι **πολύ μεγαλύτερη** και συνήθως είναι της τάξεως των **αρκετών εκατοντάδων** κύκλων του ρολογιού

Επιπλέον, ένα στημόνι **δεν είναι έτοιμο να εκτελέσει εντολές**, εάν τελεί **υπό καθεστώ αναμονής**, είτε επειδή **αναμένει** την **προσκόμιση** κάποιου δεδομένου από τη μνήμη, είτε επειδή **συμμετέχει** σε κάποια **διαδικασία συγχρονισμού**.

Για να **ελαχιστοποιήσουμε** το **χρονικό διάστημα** κατά το οποίο ένας **πολυεπεξεργαστής** βρίσκεται σε **κατάσταση αδράνειας**, λόγω της αναμονής των νημάτων του στημονιού, μπορούμε να **αναθέσουμε** σε αυτόν **περισσότερες από μία** ομάδες νημάτων. Το **πλήθος** των **στημονιών** και των **ομάδων νημάτων** που μπορούν να **ανατεθούν** σε έναν πολυεπεξεργαστή, εξαρτάται

- από την **διαμόρφωση εκτέλεσης** της συνάρτησης του πυρήνα που εκτελείται
- από τους **διαθέσιμους πόρους του πολυεπεξεργαστή** όσον αφορά στην μνήμη που μπορεί να εκχωρηθεί
- από τις **απαιτήσεις της συνάρτησης του πυρήνα** που εκτελείται, σε πάσης φύσεως **υπολογιστικούς πόρους**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Η **συνάρτηση** `cudaGetDeviceProperties` επιτρέπει την **ανάκτηση** των **τιμών** των **υπολογιστικών δυνατοτήτων** της συσκευής, αρχικοποιώντας και επιστρέφοντας μία δομή τύπου `cudaDeviceProp`.

Η εφαρμογή σχεδιάζεται έτσι ώστε να χρησιμοποιείται στους υπολογισμούς το **μεγαλύτερο δυνατό πλήθος των πυρήνων CUDA** των πολυεπεξεργασιών της συσκευής, οι οποίοι θα πρέπει να εκτελούν υπολογισμούς για το **μεγαλύτερο δυνατό χρονικό διάστημα** και έτσι ώστε εάν απαιτείται **προσωρινή αναστολή** της λειτουργίας κάποιων νημάτων, για να διευθετηθούν καταστάσεις **σύγκρουσης** και **ανταγωνισμού**, οι **χρονικές υστερήσεις** να είναι **όσο το δυνατόν πιο μικρές**.

Εάν ανακτήσουμε τα **χαρακτηριστικά λειτουργίας** της συσκευής, μπορούμε να **προσαρμόσουμε** την εφαρμογή μας σε αυτά τα χαρακτηριστικά, έτσι ώστε να **μεγιστοποιήσουμε** την απόδοση.

Η γνώση της τιμής του πεδίου `multiProcessorCount` επιτρέπει τον υπολογισμό του **ελάχιστου πλήθους** των **ομάδων νημάτων του πλέγματος**, που διασφαλίζει την χρήση του **συνόλου των πολυεπεξεργασιών** της συσκευής.

Αυτό το **πλήθος των ομάδων**, θα πρέπει να είναι **τουλάχιστον ίσο** με το **πλήθος** των πολυεπεξεργασιών και στην ιδανική περίπτωση, **ακέραιο πολλαπλάσιό** του, έτσι ώστε σε **κάθε επεξεργαστή** να ανατεθούν για εκτέλεση **πολλές** τέτοιες ομάδες.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Το **πλήθος των ομάδων** που μπορούν να **ανατεθούν** σε έναν **πολυεπεξεργαστή**, εξαρτάται από το **μέγεθος της ομάδας** και από την **ποσότητα των υπολογιστικών πόρων** που απαιτούνται για την **εκτέλεση** των νημάτων της ομάδας, στα οποία θα πρέπει να ανατεθεί η **μεγαλύτερη δυνατή ποσότητα υπολογισμών**, οι οποίοι να μπορούν να εκτελεστούν **παράλληλα**.

Το **πλήθος των νημάτων** θα πρέπει να είναι **ακέραιο πολλαπλάσιο** του **μεγέθους του στημονιού** έτσι ώστε να μην παρατηρείται **σπατάλη υπολογιστικών πόρων** και κατά συνέπεια, **μείωση της απόδοσης**.

Εάν το **πλήθος των ομάδων νημάτων**, είναι **μικρότερο** από το **πλήθος των πολυεπεξεργασιών** και κατά συνέπεια, υπάρχουν **πολυεπεξεργαστές** οι οποίοι παραμένουν **αδρανείς**, είναι δυνατή η εκτέλεση σε αυτούς τους **άεργους επεξεργαστές**, μίας άλλης **συνάρτησης πυρήνα**, εάν βέβαια η υποκείμενη συσκευή **υποστηρίζει** αυτή τη δυνατότητα.

Το **πλήθος των νημάτων ανά ομάδα**, μπορεί να υπολογιστεί από τις τιμές των πεδίων **warpSize** και **maxThreadsPerBlock** καθώς και από τις απαιτήσεις του κάθε νήματος όσον αφορά στην **χρήση των καταχωρητών** και της **κοινόχρηστης μνήμης** και καταφεύγοντας στην χρήση των πεδίων **sharedMemPerBlock** και **regsPerBlock**, χρησιμοποιώντας την εξίσωση

$$\text{threadsPerBlock} = \min \left\{ \text{numWarpSchedulers} \times \text{warpSize}, \frac{\text{regsPerBlock}}{\text{registersPerThread}}, \frac{\text{sharedMem}}{\text{sharedPerThread}}, \text{maxThreadsPerSM} \right\}$$



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Στη συνέχεια, μπορούμε να υπολογίσουμε το **συνολικό μέγεθος του πλέγματος**, από την έκφραση

$$\text{totalBlocks} = \left\lceil \frac{\text{numberOfThreads}}{\text{threadsPerBlock}} \right\rceil$$

Για να επιτύχουμε την **βέλτιστη κατανομή υπολογιστικού φορτίου** στους **πολυεπεξεργαστές**, θα πρέπει να αναθέσουμε σε αυτούς, **το ίδιο πλήθος ομάδων νημάτων**, ενώ εάν κάτι τέτοιο **δεν είναι δυνατό**, θα πρέπει να επιλέξουμε εκείνη τη διαμόρφωση που οδηγεί στην **ελάχιστη τιμή** της **ανισοκατανομής υπολογιστικού φορτίου**

$$\text{imbalance} = \frac{\text{numberOfMultiprocessors} - (\text{totalBlocks} \% \text{numberOfMultiprocessors})}{\text{blocksPerMultiprocessor} + 1}$$

όπου **blocksPerSM = floor (totalBlocks / numberOfMultiprocessors)** είναι το **πλήθος των ομάδων νημάτων** που λαμβάνουν όλοι οι πολυεπεξεργαστές, με ένα πλήθος **totalBlocks % numberOfMultiprocessors** από αυτούς, να λαμβάνουν **μία επιπλέον ομάδα**, σε σχέση με αυτή που προσδιορίζεται από την παραπάνω τιμή



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η δομή δεδομένων cudaDeviceProp

```
struct cudaDeviceProp {  
    int ECCEnabled; /* Device has ECC support enabled */  
    int accessPolicyMaxWindowSize; /* The max value of cudaAccessPolicyWindow::num_bytes */  
    int asyncEngineCount; /* Number of asynchronous engines */  
    int canMapHostMemory; /* Device can map host memory with cudaHostAlloc/cudaHostGetDevicePointer */  
    int canUseHostPointerForRegisteredMem; /* Device can access host registered memory at the same virtual address as the CPU */  
    int clockRate; /* Deprecated, Clock frequency in kilohertz */  
    int clusterLaunch; /* Indicates device supports cluster launch */  
    int computeMode; /* Deprecated, Compute mode */  
    int computePreemptionSupported; /* Device supports Compute Preemption */  
    int concurrentKernels; /* Device can possibly execute multiple kernels concurrently */  
    int concurrentManagedAccess; /* Device can coherently access managed memory concurrently with the CPU */  
    int cooperativeLaunch; /* Device supports launching cooperative kernels */  
    int cooperativeMultiDeviceLaunch; /* Deprecated, cudaLaunchCooperativeKernelMultiDevice is deprecated */  
    int deferredMappingCudaArraySupported; /* 1 if the device supports deferred mapping CUDA arrays and CUDA mipmapped arrays */  
    int deviceOverlap; /* Device can concurrently copy memory and execute a kernel. Use asyncEngineCount */  
    int directManagedMemAccessFromHost; /* Host can directly access managed memory on the device without migration */  
    int globalL1CacheSupported; /* Device supports caching globals in L1 */  
    unsigned int gpuDirectRDMAFlushWritesOptions; /* Bitmask to be interpreted according to the cudaFlushGPUDirectRDMAWritesOptions */  
    int gpuDirectRDMASupported; /* 1 if the device supports GPUDirect RDMA APIs, 0 otherwise */  
    int gpuDirectRDMAWritesOrdering; /* See the cudaGPUDirectRDMAWritesOrdering enum for numerical values */  
    int hostNativeAtomicSupported; /* Link between the device and the host supports native atomic operations */  
    int hostRegisterReadOnlySupported; /* Device supports using the cudaHostRegister flag cudaHostRegisterReadOnly */  
    int hostRegisterSupported; /* Device supports host memory registration via cudaHostRegister */  
};
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η δομή δεδομένων cudaDeviceProp

```
int integrated; /* Device is integrated as opposed to discrete */
int ipcEventSupported; /* Device supports IPC Events */
int isMultiGpuBoard; /* Device is on a multi-GPU board */
int kernelExecTimeoutEnabled; /* Deprecated, Specified whether there is a run time limit on kernels */
int l2CacheSize; /* Size of L2 cache in bytes */
int localL1CacheSupported; /* Device supports caching locals in L1 */
char luid[8]; /* 8-byte locally unique identifier. Undefined on TCC and non-Windows platforms */
unsigned int luidDeviceNodeMask; /* LUID device node mask. Undefined on TCC and non-Windows platforms */
int major; /* Major compute capability */
int managedMemory; /* Device supports allocating managed memory on this system */
int maxBlocksPerMultiProcessor; /* Maximum number of resident blocks per multiprocessor */
int maxGridSize[3]; /* Maximum size of each dimension of a grid */
int maxSurface1D; /* Maximum 1D surface size */
int maxSurface1DLayered[2]; /* Maximum 1D layered surface dimensions */
int maxSurface2D[2]; /* Maximum 2D surface dimensions */
int maxSurface2DLayered[3]; /* Maximum 2D layered surface dimensions */
int maxSurface3D[3]; /* Maximum 3D surface dimensions */
int maxSurfaceCubemap; /* Maximum Cubemap surface dimensions */
int maxSurfaceCubemapLayered[2]; /* Maximum Cubemap layered surface dimensions */
int maxTexture1D; /* Maximum 1D texture size */
int maxTexture1DLayered[2]; /* Maximum 1D layered texture dimensions */
int maxTexture1DLinear; /* Deprecated. Use cudaDeviceGetTexture1DLinearMaxWidth */
int maxTexture1DMipmap; /* Maximum 1D mipmapped texture size */
int maxTexture2D[2]; /* Maximum 2D texture dimensions */
int maxTexture2DGather[2]; /* Maximum 2D texture dimensions if texture gather operations have to be performed */
int maxTexture2DLayered[3]; /* Maximum 2D layered texture dimensions */
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η δομή δεδομένων cudaDeviceProp

```
int maxTexture2DLinear[3];          /* Maximum dimensions (width, height, pitch) for 2D textures bound to pitched mem */
int maxTexture2DMipmap[2];          /* Maximum 2D mipmapped texture dimensions */
int maxTexture3D[3];                /* Maximum 3D texture dimensions */
int maxTexture3DAlt[3];              /* Maximum alternate 3D texture dimensions */
int maxTextureCubemap;               /* Maximum Cubemap texture dimensions */
int maxTextureCubemapLayered[2];    /* Maximum Cubemap layered texture dimensions */
int maxThreadsDim[3];                /* Maximum size of each dimension of a block */
int maxThreadsPerBlock;              /* Maximum number of threads per block */
int maxThreadsPerMultiProcessor;    /* Maximum resident threads per multiprocessor */
size_t memPitch;                     /* Maximum pitch in bytes allowed by memory copies */
int memoryBusWidth;                  /* Global memory bus width in bits */
int memoryClockRate;                 /* Deprecated, Peak memory clock frequency in kilohertz */
unsigned int memoryPoolSupportedHandleTypes; /* Bitmask of handle types supported with mempool-based IPC */
int memoryPoolsSupported;            /* 1 if the device supports cudaMallocAsync and cudaMemPool family of APIs */
int minor;                           /* Minor compute capability */
int multiGpuBoardGroupID;            /* Unique identifier for a group of devices on the same multi-GPU board */
int multiProcessorCount;             /* Number of multiprocessors on device */
char name[256];                      /* ASCII string identifying device */
int pageableMemoryAccess;            /* Device supports coherently accessing pageable memory without cudaHostRegister */
int pageableMemoryAccessUsesHostPageTables; /* Device accesses pageable memory via the host's page tables */
int pciBusID;                        /* PCI bus ID of the device */
int pciDeviceID;                     /* PCI device ID of the device */
int pciDomainID;                     /* PCI domain ID of the device */
int persistingL2CacheMaxSize;        /* Device's maximum l2 persisting lines capacity setting in bytes */
int regsPerBlock;                    /* 32-bit registers available per block */
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η δομή δεδομένων cudaDeviceProp

```
int regsPerMultiprocessor;      /* 32-bit registers available per multiprocessor */
int reserved[60];              /* Reserved for future use */
int reserved1[1];              /* Reserved for future use */
size_t reservedSharedMemPerBlock; /* Shared memory reserved by CUDA driver per block in bytes */
size_t sharedMemPerBlock;      /* Shared memory available per block in bytes */
size_t sharedMemPerBlockOptin; /* Per device maximum shared memory per block usable by special opt in */
size_t sharedMemPerMultiprocessor; /* Shared memory available per multiprocessor in bytes */
int singleToDoublePrecisionPerfRatio; /* Deprecated. Ratio of single precision (FLOPS) to double precision performance */
int sparseCudaArraySupported; /* 1 if the device supports sparse CUDA arrays and sparse CUDA mipmapped arrays */
int streamPrioritiesSupported; /* Device supports stream priorities */
size_t surfaceAlignment; /* Alignment requirements for surfaces */
int tccDriver; /* 1 if device is a Tesla device using TCC driver, 0 otherwise */
size_t textureAlignment; /* Alignment requirement for textures */
size_t texturePitchAlignment; /* Pitch alignment requirement for texture references bound to pitched memory */
int timelineSemaphoreInteropSupported; /* External timeline semaphore interop is supported on the device */
size_t totalConstMem; /* Constant memory available on device in bytes */
size_t totalGlobalMem; /* Global memory available on device in bytes */
int unifiedAddressing; /* Device shares a unified address space with the host */
int unifiedFunctionPointers; /* Indicates device supports unified pointers */
cudaUUID_t uuid; /* 16-byte unique identifier */
int warpSize; /* Warp size in threads */
}
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η δομή δεδομένων cudaDeviceProp – Ανάκτηση παραμέτρων συσκευής

```
#include <stdio.h>
#include <cuda.h>
#include <cuda_runtime_api.h>

int main() {
    int nDevices, MB=1048576,KB=1024;
    float GHZ=1000000.0;
    cudaGetDeviceCount(&nDevices);
    printf("\nNumber of devices: %d\n", nDevices);
    for (int i = 0; i < nDevices; i++) {
        cudaDeviceProp prop;
        cudaGetDeviceProperties(&prop, i);
        printf("\n ---- GPU %d ----\n",i+1);
        printf("Device Number: %d\n", i);
        printf("  Device name: %s\n", prop.name);
        printf("  Memory Clock Rate (MHz): %d\n",
            prop.memoryClockRate/1024);
        printf("  Can map devMem: %d\n",
            prop.canMapHostMemory);
        printf("  Memory Bus Width (bits): %d\n",
            prop.memoryBusWidth);
        printf("  Total global memory (Gb) %.1f\n",
            (float)(prop.totalGlobalMem)/MB);
        printf("  Constant Memory : %d Kb\n",
            (int)prop.totalConstMem/KB);
        printf("  Shared Memory : %d Kb\n",
            (int)prop.sharedMemPerBlock/KB);
        printf("  L2 Cache Memory : %d bytes\n",
            (int)prop.l2CacheSize);
        printf("  Block registers: %d Kb\n",
            (int)prop.regsPerBlock/KB);
    }
}
```

```
Number of devices: 1

---- GPU 1 ----
Device Number: 0
  Device name: NVIDIA GeForce RTX 3060
  Memory Clock Rate (MHz): 7325
  Can map devMem: 1
  Memory Bus Width (bits): 192
  Total global memory (Gb) 12287.5
  Constant Memory : 64 Kb
  Shared Memory : 48 Kb
  L2 Cache Memory : 2359296 bytes
  Block registers: 64 Kb
  32-bit registers available per SM: 65536
  Clock: 1.837
  Processors : 28
  Cores : 3584
  Peak Memory Bandwidth (GB/s): 360.0
  Shared memory per block (Kbytes) 48.0
  Capability (minor-major): 6-8
  Warp-size: 32
  Grid size : 2147483647 x 65535 x 65535
  Block size : 1024 x 1024 x 64
  Concurrent kernels: yes
  Concurrent comp / comm: yes
  Threads/Block : 1024
  Max number of resident threads per multiprocessor: 1536
```

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Το σύνολο των συναρτήσεων διαχείρισης συσκευής της CUDA

<code>cudaChooseDevice</code>	<code>/* Select compute-device which best matches criteria */</code>	<code>*/</code>
<code>cudaDeviceFlushGPUDirectRDMAWrites</code>	<code>/* Blocks until remote writes are visible to scope */</code>	<code>*/</code>
<code>cudaDeviceGetAttribute</code>	<code>/* Returns information about the device */</code>	<code>*/</code>
<code>cudaDeviceGetByPCIBusId</code>	<code>/* Returns a handle to a compute device */</code>	<code>*/</code>
<code>cudaDeviceGetCacheConfig</code>	<code>/* Returns the preferred cache configuration */</code>	<code>*/</code>
<code>cudaDeviceGetDefaultMemPool</code>	<code>/* Returns the default mempool of a device */</code>	<code>*/</code>
<code>cudaDeviceGetLimit</code>	<code>/* Return resource limits */</code>	<code>*/</code>
<code>cudaDeviceGetMemPool</code>	<code>/* Gets the current mempool for a device */</code>	<code>*/</code>
<code>cudaDeviceGetNvSciSyncAttributes</code>	<code>/* Return NvSciSync attributes that dev can support */</code>	<code>*/</code>
<code>cudaDeviceGetP2PAttribute</code>	<code>/* Queries attributes of link between two devices */</code>	<code>*/</code>
<code>cudaDeviceGetPCIBusId</code>	<code>/* Returns a PCI Bus Id string for the device */</code>	<code>*/</code>
<code>cudaDeviceGetSharedMemConfig</code>	<code>/* Returns the shared memory configuration */</code>	<code>*/</code>
<code>cudaDeviceGetStreamPriorityRange</code>	<code>/* Returns the values for min-max stream priorities */</code>	<code>*/</code>
<code>cudaDeviceGetTexture1DLinearMaxWidth</code>	<code>/* Returns the max element number in linear textures */</code>	<code>*/</code>
<code>cudaDeviceReset</code>	<code>/* Destroy allocations and reset all state on dev */</code>	<code>*/</code>
<code>cudaDeviceSetCacheConfig</code>	<code>/* Sets the preferred cache configuration */</code>	<code>*/</code>
<code>cudaDeviceSetLimit</code>	<code>/* Set resource limits */</code>	<code>*/</code>
<code>cudaDeviceSetMemPool</code>	<code>/* Sets the current memory pool of a device */</code>	<code>*/</code>
<code>cudaDeviceSetSharedMemConfig</code>	<code>/* Sets the shared memory configuration */</code>	<code>*/</code>
<code>cudaDeviceSynchronize</code>	<code>/* Wait for compute device to finish */</code>	<code>*/</code>
<code>cudaGetDevice</code>	<code>/* Returns which device is currently being used */</code>	<code>*/</code>
<code>cudaGetDeviceCount</code>	<code>/* Returns the number of compute-capable devices */</code>	<code>*/</code>
<code>cudaGetDeviceFlags</code>	<code>/* Gets the flags for the current device */</code>	<code>*/</code>
<code>cudaGetDeviceProperties</code>	<code>/* Returns information about the compute-device */</code>	<code>*/</code>
<code>cudaInitDevice</code>	<code>/* Initialize device to be used for GPU executions */</code>	<code>*/</code>
<code>cudaIpcCloseMemHandle</code>	<code>/* Attempts to close memory mapped */</code>	<code>*/</code>
<code>cudaIpcGetEventHandle</code>	<code>/* Gets an interproc handle for an allocated event */</code>	<code>*/</code>
<code>cudaIpcGetMemHandle</code>	<code>/* Gets an interproc handle for an existing malloc */</code>	<code>*/</code>
<code>cudaIpcOpenEventHandle</code>	<code>/* Opens an interproc handle for use in current proc */</code>	<code>*/</code>
<code>cudaIpcOpenMemHandle</code>	<code>/* Opens an interproc handle exported from a proc */</code>	<code>*/</code>
<code>cudaSetDevice</code>	<code>/* Set device to be used for GPU executions */</code>	<code>*/</code>
<code>cudaSetDeviceFlags</code>	<code>/* Sets flags to be used for device executions */</code>	<code>*/</code>
<code>cudaSetValidDevices</code>	<code>/* Set a list of devices that can be used for CUDA */</code>	<code>*/</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Μεγιστοποίηση του ρυθμού απόδοσης όσον αφορά στη χρήση της μνήμης Σε μία **συσσκευή GPU** υπάρχουν αρκετές διαφορετικές μνήμες που **διαφοροποιούνται** τόσο ως προς το **μέγεθος**, όσο και ως προς τον **χρόνο προσπέλασης** και κατά συνέπεια, οι **τεχνικές βελτιστοποίησης**, όσον αφορά **στην χρήση της μνήμης**, είναι **διαφορετικές** για την **κάθεμία** από αυτές.

Η τιμή του **ρυθμού απόδοσης** μπορεί να διαφέρει ακόμη και **κατά μία τάξη μεγέθους**, ανάλογα με τον **τύπο της μνήμης** και το **πλήθος των εκτελέσεων** μιας εντολής, που εξαρτάται από την **κατανομή** των **διευθύνσεων μνήμης** στα νήματα που την εκτελούν

Η **μεγιστοποίηση** του **ρυθμού απόδοσης**, απαιτεί την **ελαχιστοποίηση** των **μεταφορών δεδομένων** υπό συνθήκες **χαμηλού εύρους ζώνης**, όπως αυτές που πραγματοποιούνται ανάμεσα στις μνήμες της **KME** και της **GPU**

Η ομαδοποίηση **πολλαπλών μεταφορών δεδομένων μικρού μεγέθους**, σε **μία και μοναδική** μεταφορά δεδομένων **μεγάλου μεγέθους**, οδηγεί σε **υψηλότερη** απόδοση από εκείνη που προκύπτει, εάν οι **μικρές** μεταφορές δεδομένων γίνουν **ξεχωριστά**

Χρησιμοποιώντας **τεχνικές κλειδώματος μνήμης** και **απεικόνισης περιοχών κλειδωμένης μνήμης** μπορούμε να επιτύχουμε **υψηλότερη απόδοση**, αφού στις περιπτώσεις αυτές, οι μεταφορές δεδομένων πραγματοποιούνται **μόνο κατά την χρονική στιγμή** που ο πυρήνας **προσπελάει** την περιοχή της μνήμης που έχει **απεικονιστεί**



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Καθολική μνήμη: αποτελεί **μνήμη** της **συσκευής** και οι **διαδικασίες προσπέλασης** στις οποίες εμπλέκεται, εφαρμόζονται σε περιοχές **μεγέθους 32, 64 ή 128 bytes**, των οποίων η πρώτη διεύθυνση είναι **ακέραιο πολλαπλάσιο** του μεγέθους τους

Κάθε φορά που ένα **στημόνι** εκτελεί μία **εντολή προσπέλασης** της **καθολικής μνήμης**, τα **αιτήματα προσπέλασης** από όλα τα **νήματα** του στημονιού, συνδυάζονται σε **ένα ή περισσότερα αιτήματα**, ανάλογα με το **μέγεθος της λέξης** που προσπελάσσεται από το **κάθε νήμα**, καθώς και την **κατανομή των διευθύνσεων** της μνήμης στα νήματα του στημονιού.

Όσο **μεγαλύτερο** είναι το **πλήθος των αιτημάτων πρόσβασης** στην μνήμη που θα πρέπει να **διεκπεραιωθούν**, τόσο **μεγαλύτερο** είναι το **ποσοστό** των **περιττών λέξεων μνήμης**, οι οποίες αν και **δεν** χρειάζονται, χρησιμοποιούνται **υποχρεωτικά**, λόγω **ευθυγράμμισης**, κάτι που έχει ως αποτέλεσμα, την **μείωση του ρυθμού απόδοσης** που σχετίζεται με την χρήση των εντολών.

Προκειμένου να **μεγιστοποιήσουμε** την απόδοση, θα πρέπει να **μεγιστοποιήσουμε** τον **βαθμό συγχώνευσης** των **αιτημάτων πρόσβασης**, έτσι ώστε να προκύψει το **ελάχιστο δυνατό πλήθος προσπελάσεων** στην καθολική μνήμη.

Για να το κάνουμε αυτό, θα πρέπει να χρησιμοποιούμε πάντοτε **τύπους δεδομένων**, το μέγεθος των οποίων να **ικανοποιεί** τις **απαιτήσεις** που υπαγορεύονται από το **χαρακτηριστικό της ευθυγράμμισης δεδομένων**, χρησιμοποιώντας παράλληλα τις **βέλτιστες τεχνικές πρόσβασης**, όπως αυτές υπαγορεύονται από την **υπολογιστική ικανότητα** της χρησιμοποιούμενης συσκευής.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Τοπική μνήμη: χρησιμοποιείται από τον **μεταγλωττιστή** για την αποθήκευση **μεταβλητών**, κάθε φορά που αυτό είναι **αναγκαίο**, όπως στις περιπτώσεις **μεγάλων δομών δεδομένων** που καταλαμβάνουν **υπερβολικά μεγάλο χώρο** στους **καταχωρητές** ή **μεταβλητών** που απαιτούν **περισσότερους καταχωρητές** από εκείνους που είναι **διαθέσιμοι** σε κάθε χρονική στιγμή.

Οι προσπελάσεις που πραγματοποιούνται στην **τοπική μνήμη**, χαρακτηρίζονται από την ίδια **(υψηλή) χρονική υστέρηση** και από το ίδιο **(χαμηλό) εύρος ζώνης**, με αυτές που πραγματοποιούνται στην **καθολική μνήμη**.

Προκειμένου να είναι **δυνατή** η **συγχώνευση πολλαπλών αιτημάτων πρόσβασης** στην τοπική μνήμη σε ένα **απλό** αίτημα, θα πρέπει **όλα τα νήματα που ανήκουν στο ίδιο στημόνι**, να προσπελάσουν **την ίδια** σχετική διεύθυνση μνήμης

Στις σύγχρονες συσκευές, τα δεδομένα που προσπελούνται στην **τοπική μνήμη**, αποθηκεύονται **πάντοτε** στην **κρυφή μνήμη L2**, με τον ίδιο τρόπο με τον οποίο αυτό γίνεται, για τις **προσπελάσεις** της **καθολικής μνήμης**

Κοινόχρηστη μνήμη: Εδώ η κατάσταση είναι αρκετά **διαφορετική**, αφού αυτή η μνήμη ως **τμήμα** του **ολοκληρωμένου κυκλώματος**, χαρακτηρίζεται από **μικρότερη χρονική υστέρηση** και **μεγαλύτερο εύρος ζώνης**, σε σχέση με την **καθολική μνήμη**.

Προκειμένου να επιτύχουμε **υψηλό εύρος ζώνης**, η **κοινόχρηστη μνήμη** υποδιαιρείται σε **μικρότερες περιοχές**, οι οποίες είναι γνωστές ως **ζώνες** και μπορούν να προσπελαστούν **ταυτόχρονα**, η μία σε σχέση με την άλλη



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Ειδικότερα, οι διαδικασίες πρόσβασης που χρησιμοποιούν N διευθύνσεις που ανήκουν σε N διακριτές ζώνες, εξυπηρετούνται ταυτόχρονα, οδηγώντας σε εύρος ζώνης N φορές μεγαλύτερο, σε σχέση αυτό που σχετίζεται με μία απλή τέτοια ζώνη

Εάν στο αίτημα πρόσβασης εμπλέκονται δύο διευθύνσεις που ανήκουν στην ίδια ζώνη, εμφανίζεται σύγκρουση και η προσπέλαση πραγματοποιείται με σειριακό τρόπο.

Αυτή η διαδικασία πραγματοποιείται από το υλικό, που αναλαμβάνει να διαχωρίσει ένα αίτημα πρόσβασης στη μνήμη το οποίο χαρακτηρίζεται από τέτοιες συγκρούσεις, σε πολλά μικρά αιτήματα τα οποία είναι απαλλαγμένα συγκρούσεων, κάτι που έχει ως αποτέλεσμα, την μείωση της απόδοσης του συστήματος.

Προκειμένου, λοιπόν, να επιτύχουμε υψηλή απόδοση, είναι σημαντικό να κατανοήσουμε τον τρόπο με τον οποίο οι διευθύνσεις της μνήμης απεικονίζονται στις διάφορες ζώνες έτσι ώστε να ελαχιστοποιήσουμε τον αριθμό των συγκρούσεων

Μνήμη σταθερών: αποτελεί μνήμη της συσκευής και κάθε φορά που προσπελάζεται διεύθυνση που ανήκει σε αυτή την περιοχή, αυτή αποθηκεύεται στην αντίστοιχη κρυφή μνήμη, διαδικασία, που επίσης χαρακτηρίζει και την μνήμη υφής.

Ένα αίτημα πρόσβασης διαχωρίζεται σε τόσα ξεχωριστά αιτήματα, όσες είναι οι διαφορετικές διευθύνσεις μνήμης στο αρχικό αίτημα, οδηγώντας σε μείωση της απόδοσης κατά έναν συντελεστή με τιμή ίση με το πλήθος των ξεχωριστών αιτημάτων, με τον ρυθμό εξυπηρέτησης των αιτημάτων να υπαγορεύεται από την κρυφή μνήμη (cache hit) ή την καθολική μνήμη (cache miss)



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Περί ζητημάτων απόδοσης

Η **μεγιστοποίηση του ρυθμού απόδοσης όσον αφορά στη χρήση των εντολών** μπορεί να πραγματοποιηθεί

- ελαχιστοποιώντας την **χρήση των αριθμητικών εντολών χαμηλής απόδοσης**, ακόμα και αν χρειαστεί να **θυσιάσουμε** την ακρίβεια της αναπαράστασης υπό την προϋπόθεση φυσικά πως το τελικό αποτέλεσμα είναι **αποδεκτό**
- ελαχιστοποιώντας το **πλήθος των στημονιών**, τα νήματα των οποίων ακολουθούν **διαφορετικές διαδρομές εκτέλεσης**, εξαιτίας της χρήσεως **εντολών ελέγχου ροής**
- **μειώνοντας το πλήθος των εντολών**, όταν αυτό είναι **δυνατό** (π.χ. βελτιστοποιώντας την χρήση των **σημείων συγχρονισμού**)

Ελαχιστοποίηση της ανταλλαγής σελίδων ανάμεσα στην πραγματική και στην εικονική μνήμη Στις εφαρμογές που **δεσμεύουν** και **αποδεσμεύουν** μνήμη με **μεγάλη συχνότητα**, οι **συναρτήσεις δέσμευσης μνήμης** γίνονται προοδευτικά **όλο και πιο αργές**, μέχρι να προσεγγίσουν κάποιο **όριο**, λόγω της φύσεως της **διαδικασίας αποδέσμευσης της μνήμης** από το **λειτουργικό σύστημα**.

Γενικά, **δεν συνίσταται** η δέσμευση **όλης της διαθέσιμης μνήμης** με κάποια από τις συναρτήσεις **cudaMalloc**, **cudaMallocHost** και **cuMemCreate**, αφού αυτό θα λειτουργήσει **αποτρεπτικά**, όσον αφορά στην **χρήση** αυτής της μνήμης από **άλλες** εφαρμογές.

Αντίθετα, το **μέγεθος** της μνήμης που θα **δεσμευθεί**, θα πρέπει να είναι το **κατάλληλο**, έτσι όπως αυτό **υπαγορεύεται** από τις **ανάγκες της εφαρμογής σε μνήμη**, με την **δέσμευση** να πραγματοποιείται στα **αρχικά στάδια της εφαρμογής** και την **αποδέσμευση** να πραγματοποιείται **αμέσως μόλις διαπιστωθεί** πως αυτή η περιοχή μνήμης **δεν χρειάζεται** πλέον.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Παράμετροι λειτουργίας για τις διάφορες υπολογιστικές ικανότητες

Υπολογιστική ικανότητα συσκευής	5.0, 5.2	5.3	6.0	6.1	6.2	7.x	8.0	8.6	8.9	9.0
Πρόσθεση, πολλαπλασιασμός και πρόσθεση - πολλαπλασιασμός κινητής υποδιαστολής (16 bit)	N/A	256	128	2	256	128	256 ^a		128	256
Πρόσθεση, πολλαπλασιασμός και πρόσθεση - πολλαπλασιασμός κινητής υποδιαστολής (32 bit)	128		64	128		64		128		
Πρόσθεση, πολλαπλασιασμός και πρόσθεση - πολλαπλασιασμός κινητής υποδιαστολής (64 bit)	4		32	4		32 ^a	32	2		64
Εκθετικές, τριγωνομετρικές και άλλες συναρτήσεις (αντιστροφή αριθμού, τετραγωνική ρίζα κ.λ.π) (32 bit)	32		16	32		16				
Τύποι πρόσθεσης μεταξύ ακεραίων (32 bit)	128		64	128		64				
Πολλαπλασιασμός και πρόσθεση - πολλαπλασιασμός μεταξύ ακεραίων (32 bit)	Πολλαπλές εντολές					64 ^a				
Πολλαπλασιασμός ακεραίων (24 bit)	Πολλαπλές εντολές									
Ολίσθηση με ακεραίους (32 bit)	64		32	64						
Σύγκριση, υπολογισμός μέγιστου και ελάχιστου	64		32	64						
Αντιστροφή bit (32 bit)	64		32	64		16				
Εισαγωγή / εξαγωγή bits	64		32	64		Πολλαπλές εντολές			64	
Δυναδικές πράξεις AND, OR, XOR (32 bit)	128		64	128		64				
Καταμέτρηση αρχικών μηδενικών και προσδιορισμός του πιο σημαντικού bit της τιμής	32 16 32 16									
Καταμέτρηση πληθυσμού	32		16	32		16				
Αναδιάταξη νημάτων στημονιού	32					32 ^a	32			
Πράξη υποβιβασμού στα νήματα στημονιού	Πολλαπλές εντολές						16			
Επιστροφή τιμής από νήματα στημονιού	64									
Άθροισμα απόλυτων διαφορών	64		32	64						
Εντολές SIMD για video vabsdiff2	Πολλαπλές εντολές									
Εντολές SIMD για video vabsdiff4	Πολλαπλές εντολές					64				
Όλες οι υπόλοιπες εντολές SIMD για video	Πολλαπλές εντολές									
Μετατροπή ακεραίων των 8 και 16 bit σε ακεραίους των 32 bit	32		16	32		64				
Μετατροπή από και προς τύπους των 64 bits	4		16	4		16 ¹⁰	16	2	2	16
Όλες οι υπόλοιπες μετατροπές τύπων	32		16	32		16				
16-bit DPX	Πολλαπλές εντολές									128
32-bit DPX	Πολλαπλές εντολές									64



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι μεταβλητές περιβάλλοντος της CUDA

	/* Device Enumeration and Properties	*/
CUDA_VISIBLE_DEVICES	/* A comma-separated sequence of GPU identifiers	*/
CUDA_MANAGED_FORCE_DEVICE_ALLOC	/* Forces the driver to place all managed allocations in device memory	*/
CUDA_DEVICE_ORDER	/* Controls the ordering of supported devices	*/
	/* Compilation	*/
CUDA_CACHE_DISABLE	/* Disables or enables caching for just-in-time-compilation	*/
CUDA_CACHE_PATH	/* Specifies the folder where the JIT compiler caches binary codes	*/
CUDA_CACHE_MAXSIZE	/* Specifies the size in bytes of the cache used by the JIT compiler	*/
CUDA_FORCE_PTX_JIT	/* Forces (or not) the device driver to ignore any embedded bin code	*/
CUDA_DISABLE_PTX_JIT	/* Disables or enables the JIT compilation of embedded PTX code	*/
CUDA_FORCE_JIT	/* Forces (or not) the device driver to ignore any embedded bin code	*/
CUDA_DISABLE_JIT	/* Disables or the just-in-time compilation of embedded PTX code	*/
	/* Execution	*/
CUDA_LAUNCH_BLOCKING	/* Disables or enables asynchronous kernel launches	*/
CUDA_DEVICE_MAX_CONNECTIONS	/* Sets the number of compute and copy engine concurrent connections	*/
CUDA_AUTO_BOOST	/* Overrides the autoboot behavior of nvidia-smi	*/
CUDA_DEVICE_WAITS_ON_EXCEPTION	/* When set to 1, a CUDA application will halt in a device exception	*/
	/* Module loading	*/
CUDA_MODULE_LOADING	/* Specifies the module loading mode for the application	*/
CUDA_FORCE_PRELOAD_LIBRARIES	/* Forces (or not) the driver to preload the NVVM and JIT libraries	*/



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Τα αντικείμενα του Driver API

```
/* Objects Available in the CUDA Driver API */
/*
Object Type          Handle          /* Description
Device:              CUdevice         /* CUDA-enabled device
Context:              CUcontext        /* Roughly equivalent to a CPU process
Module:               CUmodule         /* Roughly equivalent to a dynamic library
Function:             CUfunction        /* Kernel
Heap memory:          CUdeviceptr      /* Pointer to device memory
CUDA array:           CUarray          /* Opaque container for 1-D or 2-D device data
Texture reference:    CUtexref         /* Object that describes how to interpret texture data
Surface reference:    CUsurfref        /* Object that describes how to read or write arrays
Stream:               CUSTream         /* Object that describes a CUDA stream
Event:                CUevent          /* Object that describes a CUDA event

/* Typedefs (Type definitions)
typedef cudaArray * cudaArray_const_t /* CUDA array (source copy arg)
typedef cudaArray * cudaArray_t       /* CUDA array
typedef CUeglStreamConnection_st * cudaEglStreamConnection /* CUDA EGLSream Connection
typedef enumcudaError cudaError_t     /* CUDA Error types
typedef CUevent_st * cudaEvent_t       /* CUDA event types
typedef CUexternalMemory_st * cudaExternalMemory_t /* CUDA external memory
typedef CUexternalSemaphore_st * cudaExternalSemaphore_t /* CUDA external semaphore
typedef CUfunc_st * cudaFunction_t     /* CUDA function
typedef unsigned long long cudaGraphConditionalHandle /* CUDA handle for cond. nodes
typedef CUGraphExec_st * cudaGraphExec_t /* CUDA executable graph
typedef CUGraphNode_st * cudaGraphNode_t /* CUDA graph node
typedef CUGraph_st * cudaGraph_t       /* CUDA graph
typedef cudaGraphicsResource * cudaGraphicsResource_t /* CUDA graphics resource types
typedef void (CUDART_CB* cudaHostFn_t) void* userData /* CUDA host function
typedef UKern_st * cudaKernel_t        /* CUDA kernel
typedef CUMemPoolHandle_st * cudaMemPool_t /* CUDA memory pool
typedef cudaMipmappedArray * cudaMipmappedArray_const_t /* CUDA mipmapped array, src arg
typedef cudaMipmappedArray * cudaMipmappedArray_t /* CUDA mipmapped array
typedef CUSTream_st * cudaStream_t     /* CUDA stream
typedef unsigned long long cudaSurfaceObject_t /* CUDA Surface object
typedef unsigned long long cudaTextureObject_t /* CUDA texture object
typedef CUUserObject_st * cudaUserObject_t /* CUDA user object for graphs
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι τύποι δεδομένων του περιβάλλοντος της CUDA

struct CUuuid_st	/* CUDA UUID types	*/
struct cudaAccessPolicyWindow	/* Specifies an access policy for a window	*/
struct cudaArrayMemoryRequirements	/* CUDA array and mipmapped array memory requirements	*/
struct cudaArraySparseProperties	/* Sparse CUDA array and mipmapped array properties	*/
struct cudaChannelFormatDesc	/* CUDA Channel format descriptor	*/
struct cudaChildGraphNodeParams	/* Child graph node parameters	*/
struct cudaConditionalNodeParams	/* CUDA conditional node parameters	*/
struct cudaDeviceProp	/* CUDA device properties	*/
struct cudaEglFrame	/* CUDA EGL Frame Descriptor	*/
struct cudaEglPlaneDesc	/* CUDA EGL Plane Descriptor	*/
struct cudaEventRecordNodeParams	/* Event record node parameters	*/
struct cudaEventWaitNodeParams	/* Event wait node parameters	*/
struct cudaExtent	/* CUDA extent	*/
struct cudaExternalMemoryBufferDesc	/* External memory buffer descriptor	*/
struct cudaExternalMemoryHandleDesc	/* External memory handle descriptor	*/
struct cudaExternalMemoryMipmappedArrayDesc	/* External memory mipmap descriptor	*/
struct cudaExternalSemaphoreHandleDesc	/* External semaphore handle descriptor	*/
struct cudaExternalSemaphoreSignalNodeParams	/* External semaphore signal node parameters	*/
struct cudaExternalSemaphoreSignalNodeParamsV2	/* External semaphore signal node parameters	*/

Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι τύποι δεδομένων του περιβάλλοντος της CUDA

struct	cudaExternalSemaphoreSignalParams	/* External semaphore signal parameters	*/
struct	cudaExternalSemaphoreSignalParams_v1	/* External semaphore signal parameters (deprecated)	*/
struct	cudaExternalSemaphoreWaitNodeParams	/* External semaphore wait node parameters	*/
struct	cudaExternalSemaphoreWaitNodeParamsV2	/* External semaphore wait node parameters	*/
struct	cudaExternalSemaphoreWaitParams	/* External semaphore wait parameters	*/
struct	cudaExternalSemaphoreWaitParams_v1	/* External semaphore wait parameters (deprecated)	*/
struct	cudaFuncAttributes	/* CUDA function attributes	*/
struct	cudaGraphEdgeData	/* Optional annotation for edges in a CUDA graph	*/
struct	cudaGraphExecUpdateResultInfo	/* Result information returned by cudaGraphExecUpdate	*/
struct	cudaGraphInstantiateParams	/* Graph instantiation parameters	*/
struct	cudaGraphNodeParams	/* Graph node parameters	*/
struct	cudaHostNodeParams	/* CUDA host node parameters	*/
struct	cudaHostNodeParamsV2	/* CUDA host node parameters	*/
struct	cudaIpcEventHandle_t	/* CUDA IPC event handle	*/
struct	cudaIpcMemHandle_t	/* CUDA IPC memory handle	*/
struct	cudaKernelNodeParams	/* CUDA GPU kernel node parameters	*/
struct	cudaKernelNodeParamsV2	/* CUDA host node parameters	*/
struct	cudaLaunchAttribute	/* Launch attribute	*/
union	cudaLaunchAttributeValue	/* Launch attributes union	*/
struct	cudaLaunchConfig_t	/* CUDA extensible launch configuration	*/



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Οι τύποι δεδομένων του περιβάλλοντος της CUDA

struct	cudaLaunchMemSyncDomainMap	/* Memory Synchronization Domain map	*/
struct	cudaLaunchParams	/* CUDA launch parameters	*/
struct	cudaMemAccessDesc	/* Memory access descriptor	*/
struct	cudaMemAllocNodeParams	/* Memory allocation node parameters	*/
struct	cudaMemAllocNodeParamsV2	/* Memory allocation node parameters	*/
struct	cudaMemFreeNodeParams	/* Memory free node parameters	*/
struct	cudaMemLocation	/* Specifies a memory location	*/
struct	cudaMemPoolProps	/* Specifies the properties of allocations from pool	*/
struct	cudaMemPoolPtrExportData	/* Opaque data for exporting a pool allocation	*/
struct	cudaMemcpy3DParms	/* CUDA 3D memory copying parameters	*/
struct	cudaMemcpy3DPeerParms	/* CUDA 3D cross-device memory copying parameters	*/
struct	cudaMemcpyNodeParams	/* Memcpy node parameters	*/
struct	cudaMemsetParams	/* CUDA Memset node parameters	*/
struct	cudaMemsetParamsV2	/* CUDA Memset node parameters	*/
struct	cudaPitchedPtr	/* CUDA Pitched memory pointer	*/
struct	cudaPointerAttributes	/* CUDA pointer attributes	*/
struct	cudaPos	/* CUDA 3D position	*/
struct	cudaResourceDesc	/* CUDA resource descriptor	*/
struct	cudaResourceViewDesc	/* CUDA resource view descriptor	*/
struct	cudaTextureDesc	/* CUDA texture descriptor	*/



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Η λίστα σφαλμάτων που χρησιμοποιείται στο περιβάλλον της CUDA

```
cudaSuccess = 0
cudaErrorInvalidValue = 1
cudaErrorMemoryAllocation = 2
cudaErrorInitializationError = 3
cudaErrorCudartUnloading = 4
cudaErrorProfilerDisabled = 5
cudaErrorInvalidConfiguration = 9
cudaErrorInvalidPitchValue = 12
cudaErrorInvalidSymbol = 13
cudaErrorInvalidTexture = 18
cudaErrorInvalidTextureBinding = 19
cudaErrorInvalidChannelDescriptor = 20
cudaErrorInvalidMemcpyDirection = 21
cudaErrorInvalidFilterSetting = 26
cudaErrorInvalidNormSetting = 27
cudaErrorStubLibrary = 34
cudaErrorInsufficientDriver = 35
cudaErrorCallRequiresNewerDriver = 36
cudaErrorInvalidSurface = 37
cudaErrorDuplicateVariableName = 43
cudaErrorDuplicateTextureName = 44
cudaErrorDuplicateSurfaceName = 45
```

```
/* The API call returned with no errors */
/* One or more of the parameters is out of range */
/* Not enough memory or other resources to complete the operation */
/* CUDA driver and runtime could not be initialized */
/* CUDA Runtime API call cannot be executed */
/* Profiler is not initialized for this run */
/* Kernel launch is requesting resources that can never be satisfied */
/* One or more of pitch-related parameters are out of range */
/* The symbol name / identifier is not valid */
/* The texture passed to the API call is not valid */
/* The texture binding is not valid */
/* The channel descriptor passed to the API call is not valid */
/* The direction of the memcpy is not specified by cudaMemcpyKind */
/* A non-float texture was being accessed with linear filtering */
/* An attempt made to read a non-float texture as a normalized float */
/* The CUDA driver that the application has loaded is a stub library */
/* The installed NVIDIA CUDA driver is older than the CUDART library */
/* The API call requires a newer CUDA driver */
/* The surface passed to the API call is not a valid surface */
/* Multiple global or constant variables share the same string name */
/* Multiple textures share the same string name */
/* Multiple surfaces share the same string name */
```



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η λίστα σφαλμάτων που χρησιμοποιείται στο περιβάλλον της CUDA

<code>cudaErrorDevicesUnavailable = 46</code>	<code>/* All CUDA devices are busy or unavailable at the current time */</code>	<code>*/</code>
<code>cudaErrorIncompatibleDriverContext = 49</code>	<code>/* The current context is not compatible with this the CUDA Runtime */</code>	<code>*/</code>
<code>cudaErrorMissingConfiguration = 52</code>	<code>/* The device function being invoked was not configured */</code>	<code>*/</code>
<code>cudaErrorLaunchMaxDepthExceeded = 65</code>	<code>/* A grid launch did not occur since the child grid depth is >= max */</code>	<code>*/</code>
<code>cudaErrorLaunchFileScopedTex = 66</code>	<code>/* A grid launch did not occur due to unsupported texture */</code>	<code>*/</code>
<code>cudaErrorLaunchFileScopedSurf = 67</code>	<code>/* A grid launch did not occur due to unsupported urfaces */</code>	<code>*/</code>
<code>cudaErrorSyncDepthExceeded = 68</code>	<code>/* A call to cudaDeviceSynchronize failed because of invalid depth */</code>	<code>*/</code>
<code>cudaErrorLaunchPendingCountExceeded = 69</code>	<code>/* A device grid launch failed because of limit exceed */</code>	<code>*/</code>
<code>cudaErrorInvalidDeviceFunction = 98</code>	<code>/* The requested device function does not exist */</code>	<code>*/</code>
<code>cudaErrorNoDevice = 100</code>	<code>/* No CUDA-capable devices were detected by the installed driver */</code>	<code>*/</code>
<code>cudaErrorInvalidDevice = 101</code>	<code>/* The user supplied dev ordinal does not correspond to a valid dev */</code>	<code>*/</code>
<code>cudaErrorDeviceNotLicensed = 102</code>	<code>/* The device doesn't have a valid Grid License */</code>	<code>*/</code>
<code>cudaErrorSoftwareValidityNotEstablished = 103</code>	<code>/* At least one of required tests has failed */</code>	<code>*/</code>
<code>cudaErrorStartupFailure = 127</code>	<code>/* An internal startup failure in the CUDA runtime occurred */</code>	<code>*/</code>
<code>cudaErrorInvalidKernelImage = 200</code>	<code>/* The device kernel image is invalid */</code>	<code>*/</code>
<code>cudaErrorDeviceUninitialized = 201</code>	<code>/* There is no context bound to the current thread */</code>	<code>*/</code>
<code>cudaErrorMapBufferObjectFailed = 205</code>	<code>/* The buffer object could not be mapped */</code>	<code>*/</code>
<code>cudaErrorUnmapBufferObjectFailed = 206</code>	<code>/* The buffer object could not be unmapped */</code>	<code>*/</code>
<code>cudaErrorArrayIsMapped = 207</code>	<code>/* The specified array is currently mapped and cannot be destroyed */</code>	<code>*/</code>
<code>cudaErrorAlreadyMapped = 208</code>	<code>/* The resource is already mapped */</code>	<code>*/</code>
<code>cudaErrorNoKernelImageForDevice = 209</code>	<code>/* There is no kernel image available, suitable for the device */</code>	<code>*/</code>
<code>cudaErrorAlreadyAcquired = 210</code>	<code>/* A resource has already been acquired */</code>	<code>*/</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η λίστα σφαλμάτων που χρησιμοποιείται στο περιβάλλον της CUDA

<code>cudaErrorNotMapped = 211</code>	<code>/* A resource is not mapped</code>	<code>*/</code>
<code>cudaErrorNotMappedAsArray = 212</code>	<code>/* A mapped resource is not available for access as an array</code>	<code>*/</code>
<code>cudaErrorNotMappedAsPointer = 213</code>	<code>/* A mapped resource is not available for access as a pointer</code>	<code>*/</code>
<code>cudaErrorECCUncorrectable = 214</code>	<code>/* An uncorrectable ECC error was detected during execution</code>	<code>*/</code>
<code>cudaErrorUnsupportedLimit = 215</code>	<code>/* The cudaLimit passed to the API call is not supported by device</code>	<code>*/</code>
<code>cudaErrorDeviceAlreadyInUse = 216</code>	<code>/* A call tried to access an exclusive-thread device already in use</code>	<code>*/</code>
<code>cudaErrorPeerAccessUnsupported = 217</code>	<code>/* P2P access is not supported across the given devices</code>	<code>*/</code>
<code>cudaErrorInvalidPtx = 218</code>	<code>/* A PTX compilation failed</code>	<code>*/</code>
<code>cudaErrorInvalidGraphicsContext = 219</code>	<code>/* An error occurred associated with the OpenGL or DirectX context</code>	<code>*/</code>
<code>cudaErrorNvlinkUncorrectable = 220</code>	<code>/* An uncorrectable NVLink error was detected during the execution</code>	<code>*/</code>
<code>cudaErrorJitCompilerNotFound = 221</code>	<code>/* The PTX JIT compiler library was not found</code>	<code>*/</code>
<code>cudaErrorUnsupportedPtxVersion = 222</code>	<code>/* The provided PTX was compiled with an unsupported toolchain</code>	<code>*/</code>
<code>cudaErrorJitCompilationDisabled = 223</code>	<code>/* The JIT compilation was disabled</code>	<code>*/</code>
<code>cudaErrorUnsupportedExecAffinity = 224</code>	<code>/* The provided execution affinity is not supported by the device</code>	<code>*/</code>
<code>cudaErrorUnsupportedDevSideSync = 225</code>	<code>/* The code to be compiled contains an unsupported call</code>	<code>*/</code>
<code>cudaErrorInvalidSource = 300</code>	<code>/* The device kernel source is invalid</code>	<code>*/</code>
<code>cudaErrorFileNotFound = 301</code>	<code>/* The file specified was not found</code>	<code>*/</code>
<code>cudaErrorSharedObjectSymbolNotFound = 302</code>	<code>/* A link to a shared object failed to resolve</code>	<code>*/</code>
<code>cudaErrorSharedObjectInitFailed = 303</code>	<code>/* The initialization of a shared object failed</code>	<code>*/</code>
<code>cudaErrorOperatingSystem = 304</code>	<code>/* An OS call failed</code>	<code>*/</code>
<code>cudaErrorInvalidResourceHandle = 400</code>	<code>/* A resource handle passed to the API call was not valid</code>	<code>*/</code>
<code>cudaErrorIllegalState = 401</code>	<code>/* A required resource is not in a valid state to perform operation</code>	<code>*/</code>
<code>cudaErrorLossyQuery = 402</code>	<code>/* An attempt was made to introspect an object in a wrong way</code>	<code>*/</code>
<code>cudaErrorSymbolNotFound = 500</code>	<code>/* A named symbol was not found</code>	<code>*/</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η λίστα σφαλμάτων που χρησιμοποιείται στο περιβάλλον της CUDA

<code>cudaErrorNotReady = 600</code>	<code>/* Asynchronous operations issued previously have not completed yet */</code>
<code>cudaErrorIllegalAddress = 700</code>	<code>/* Load or store instruction on an invalid memory address */</code>
<code>cudaErrorLaunchOutOfResources = 701</code>	<code>/* A launch did not occur because of inappropriate resources */</code>
<code>cudaErrorLaunchTimeout = 702</code>	<code>/* The device kernel took too long to execute */</code>
<code>cudaErrorLaunchIncompatibleTexturing = 703</code>	<code>/* A kernel launch used an incompatible texturing mode */</code>
<code>cudaErrorPeerAccessAlreadyEnabled = 704</code>	<code>/* An invalid cudaDeviceEnablePeerAccess() operation */</code>
<code>cudaErrorPeerAccessNotEnabled = 705</code>	<code>/* cudaDeviceDisablePeerAccess is trying to disable peer addressing */</code>
<code>cudaErrorSetOnActiveProcess = 708</code>	<code>/* The user has performed an invalid device related operation */</code>
<code>cudaErrorContextIsDestroyed = 709</code>	<code>/* The context current to the calling thread has been destroyed */</code>
<code>cudaErrorAssert = 710</code>	<code>/* An assert triggered in device code during kernel execution */</code>
<code>cudaErrorTooManyPeers = 711</code>	<code>/* The hardware resources for peer access have been exhausted */</code>
<code>cudaErrorHostMemoryAlreadyRegistered = 712</code>	<code>/* A cudaHostRegister related invalid operation */</code>
<code>cudaErrorHostMemoryNotRegistered = 713</code>	<code>/* A cudaHostUnregister related invalid operation */</code>
<code>cudaErrorHardwareStackError = 714</code>	<code>/* Device encountered an error in the call stack during kernel exec */</code>
<code>cudaErrorIllegalInstruction = 715</code>	<code>/* Device encountered an illegal instruction during kernel execution */</code>
<code>cudaErrorMisalignedAddress = 716</code>	<code>/* Device encountered a load / store instr on a not aligned address */</code>
<code>cudaErrorInvalidAddressSpace = 717</code>	<code>/* A memory related device error */</code>
<code>cudaErrorInvalidPc = 718</code>	<code>/* Device encountered an invalid program counter */</code>
<code>cudaErrorLaunchFailure = 719</code>	<code>/* An exception occurred on the device while executing a kernel */</code>
<code>cudaErrorCooperativeLaunchTooLarge = 720</code>	<code>/* The block number for a cooperative kernel is not valid */</code>
<code>cudaErrorNotPermitted = 800</code>	<code>/* The attempted operation is not permitted */</code>
<code>cudaErrorNotSupported = 801</code>	<code>/* The attempted operation is not supported on the current device */</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Η λίστα σφαλμάτων που χρησιμοποιείται στο περιβάλλον της CUDA

<code>cudaErrorSystemNotReady = 802</code>	<code>/* The system is not yet ready to start any CUDA work */</code>	<code>*/</code>
<code>cudaErrorSystemDriverMismatch = 803</code>	<code>/* Mismatch between versions of the display driver and CUDA driver */</code>	<code>*/</code>
<code>cudaErrorCompatNotSupportedOnDevice = 804</code>	<code>/* The hardware detected by CUDA does not support the configuration */</code>	<code>*/</code>
<code>cudaErrorMpsConnectionFailed = 805</code>	<code>/* The MPS client failed to connect to the MPS server */</code>	<code>*/</code>
<code>cudaErrorMpsRpcFailure = 806</code>	<code>/* The RPC between the MPS server and the MPS client failed */</code>	<code>*/</code>
<code>cudaErrorMpsServerNotReady = 807</code>	<code>/* The MPS server is not ready to accept new MPS client requests */</code>	<code>*/</code>
<code>cudaErrorMpsMaxClientsReached = 808</code>	<code>/* The hardware resources to create MPS client have been exhausted */</code>	<code>*/</code>
<code>cudaErrorMpsMaxConnectionsReached = 809</code>	<code>/* The hardware resources required to dev connections are exhausted */</code>	<code>*/</code>
<code>cudaErrorMpsClientTerminated = 810</code>	<code>/* The MPS client has been terminated by the server */</code>	<code>*/</code>
<code>cudaErrorCdpNotSupported = 811</code>	<code>/* The program is using CDP, but the conf does not support it */</code>	<code>*/</code>
<code>cudaErrorCdpVersionMismatch = 812</code>	<code>/* Unsupported interaction between different versions of CDP found */</code>	<code>*/</code>
<code>cudaErrorStreamCaptureUnsupported = 900</code>	<code>/* The operation is not permitted when the stream is capturing */</code>	<code>*/</code>
<code>cudaErrorStreamCaptureInvalidated = 901</code>	<code>/* The capture stream sequence has been invalidated */</code>	<code>*/</code>
<code>cudaErrorStreamCaptureMerge = 902</code>	<code>/* The op would have merged two independent capture sequences */</code>	<code>*/</code>
<code>cudaErrorStreamCaptureUnmatched = 903</code>	<code>/* The capture was not initiated in this stream */</code>	<code>*/</code>
<code>cudaErrorStreamCaptureUnjoined = 904</code>	<code>/* The capture sequence contains a fork not joined to primary stream */</code>	<code>*/</code>
<code>cudaErrorStreamCaptureIsolation = 905</code>	<code>/* A dependency crossing the capture sequence boundary may created */</code>	<code>*/</code>
<code>cudaErrorStreamCaptureImplicit = 906</code>	<code>/* The op would have resulted in a disallowed implicit dependency */</code>	<code>*/</code>
<code>cudaErrorCapturedEvent = 907</code>	<code>/* The op is not permitted on the last recorded event in stream */</code>	<code>*/</code>
<code>cudaErrorStreamCaptureWrongThread = 908</code>	<code>/* A stream capture sequence not initiated */</code>	<code>*/</code>
<code>cudaErrorTimeout = 909</code>	<code>/* The wait operation has timed out */</code>	<code>*/</code>
<code>cudaErrorGraphExecUpdateFailure = 910</code>	<code>/* The graph update was not performed due to constraint violation */</code>	<code>*/</code>
<code>cudaErrorExternalDevice = 911</code>	<code>/* An async error has occurred in a device outside of CUDA */</code>	<code>*/</code>
<code>cudaErrorInvalidClusterSize = 912</code>	<code>/* A kernel launch error occurred due to cluster misconfiguration */</code>	<code>*/</code>
<code>cudaErrorUnknown = 999</code>	<code>/* An unknown internal error has occurred */</code>	<code>*/</code>



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Γνωστές βιβλιοθήκες της CUDA

Το περιβάλλον προγραμματισμού της **CUDA**, συνοδεύεται από ένα πλήθος **βιβλιοθηκών επέκτασης** που επιτρέπουν την **ανάπτυξη επιστημονικών εφαρμογών** που εντάσσονται σε αρκετά πεδία των θετικών επιστημών όπως είναι η **αριθμητική ανάλυση**, η **γραμμική άλγεβρα**, τα **τεχνητά νευρωνικά δίκτυα**, καθώς και άλλο πολλά. Αρκετές από αυτές παρουσιάζονται εδώ.

Η βιβλιοθήκη cuSPARSE περιέχει ένα σύνολο συναρτήσεων **γραμμικής άλγεβρας** η εκτέλεση των οποίων είναι **ταχύτατη**, αφού εκμεταλλεύεται τις **υπολογιστικές δυνατότητες** της **GPU**, για την αποτελεσματική **διαχείριση** και **χρήση αραιών πινάκων** στους οποίους η συντριπτική πλειοψηφία των κελιών τους έχει μηδενική τιμή. Η βιβλιοθήκη υποστηρίζει τις επόμενες διαδικασίες:

- **Πρόσθεση, εσωτερικό γινόμενο, συλλογή και διασπορά** ανάμεσα σε **πυκνό** και σε **αραιό** διάνυσμα
- **Πολλαπλασιασμό** ανάμεσα σε **πυκνό** πίνακα και σε **αραιό** διάνυσμα
- **Πολλαπλασιασμό** ανάμεσα σε **αραιό** πίνακα και σε **πυκνό** διάνυσμα
- **Επίλυση γραμμικού συστήματος** με πίνακα συντελεστών **αραιό** πίνακα και διάνυσμα σταθερών όρων **πυκνό** διάνυσμα για τις ειδικές περιπτώσεις κατά τις οποίες ο **αραιός** πίνακας είναι **τριγωνικός, τριδιαγώνιος και πενταδιαγώνιος**
- **Πρόσθεση και πολλαπλασιασμό** μεταξύ **αραιών** πινάκων



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Γνωστές βιβλιοθήκες της CUDA

Η βιβλιοθήκη **cuBLAS** (**B**asic **L**inear **A**lgebra **S**ubroutine) περιέχει ένα σύνολο συναρτήσεων που καλούνται μέσα από εφαρμογές σε γλώσσες **C** και **C++** και επιτρέπουν την αποδοτική υλοποίηση διαδικασιών που εμπίπτουν στο πεδίο της γραμμικής άλγεβρας.

Η βιβλιοθήκη αποτελείται από τρεις συλλογές συναρτήσεων την **cuBLAS**, την **cuBLASLt** και την **cuBLASLt** που υποστηρίζουν διαφορετικούς τύπους αλληλεπίδρασης ανάμεσα στην **KME** και στην **GPU**, όσον αφορά στην ανταλλαγή δεδομένων.

Στην πρώτη περίπτωση, η διαδικασία συνίσταται στην δέσμευση και αρχικοποίηση των κατάλληλων πινάκων και διανυσμάτων στην μνήμη της **GPU**, στην πραγματοποίηση των υπολογισμών, και στην αποστολή των αποτελεσμάτων από την **GPU** στην **KME**.

Στην δεύτερη περίπτωση τα δεδομένα μπορεί να βρίσκονται εκτός από την μνήμη της **GPU** και στην μνήμη της **KME** με τις απαιτούμενες διαδικασίες να υποστηρίζονται από την ίδια την βιβλιοθήκη.

Τέλος, οι συναρτήσεις της τρίτης συλλογής έχουν υλοποιηθεί για να υποστηρίξουν διαδικασίες τύπου **GEM** (**G**eneral **M**atrix-to-matrix **M**ultiply) στις οποίες πραγματοποιούνται πολλαπλασιασμοί πινάκων.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Γνωστές βιβλιοθήκες της CUDA

Η βιβλιοθήκη **cuFFT** περιέχει ένα σύνολο συναρτήσεων που επιτρέπουν την ταχύτατη υλοποίηση του μετασχηματισμού FFT, ενός από τους πιο γνωστούς μετασχηματισμούς που σχετίζονται με τον χώρο της ψηφιακής επεξεργασίας σήματος, με αναρίθμητες εφαρμογές σε όλους τους κλάδους της επιστήμης του μηχανικού.

Η υλοποίηση του μετασχηματισμού FFT στην CUDA, χαρακτηρίζεται από την ύπαρξη δύο ξεχωριστών βιβλιοθηκών

τη βιβλιοθήκη **cuFFT** που αποσκοπεί στην υλοποίηση υψηλής απόδοσης του μετασχηματισμού FFT σε συσκευές GPU
τη βιβλιοθήκη **cuFFTW** που λειτουργεί ως ένα βοηθητικό εργαλείο για την γρήγορη και αποδοτική χρήση της GPU

Αμφότερες οι βιβλιοθήκες επιτρέπουν στον χρήστη, να αξιοποιήσει την ισχύ της αναπαράστασης κινητής υποδιαστολής, καθώς και τον παραλληλισμό που προσφέρεται από την GPU και υποστηρίζει χαρακτηριστικά όπως

- η χρήση μισής ακρίβειας, απλής ακρίβειας και διπλής ακρίβειας
- η χρήση πραγματικών και μιγαδικών εισόδων και εξόδων
- δυνατότητα εκτέλεσης των υπολογισμών, σε περισσότερες από μία συσκευές GPU.



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργασιών

Γνωστές βιβλιοθήκες της CUDA

Η βιβλιοθήκη **cuSolver** προσφέρει συναρτήσεις γραμμικής άλγεβρας και επιτρέπουν την ταχύτατη εκτέλεση αλγορίθμων ανάλυσης πινάκων και επίλυσης γραμμικών συστημάτων, για πυκνούς και αραιούς πίνακες.

Η βιβλιοθήκη στηρίζεται στην χρήση των βιβλιοθηκών **cuBLAS** and **cuSPARSE** και αποτελείται από δύο ομάδες συναρτήσεων:

- οι συναρτήσεις της ομάδας **cuSolver** χρησιμοποιούνται για εκτέλεση σε υπολογιστικό κόμβο με μία GPU
- οι συναρτήσεις της ομάδας **cuSolverMG** χρησιμοποιούνται για την εκτέλεση σε συσκευές με πολλές GPU

Η βιβλιοθήκη **cuSolver** περιέχει ρουτίνες για την παραγοντοποίηση πινάκων και την επίλυση συστημάτων με τριγωνικό πίνακα συντελεστών, για την περίπτωση των πυκνών πινάκων, την υλοποίηση αλγορίθμων βελτιστοποίησης σε εφαρμογές στατιστικής και μηχανικής μάθησης στις οποίες εμπλέκεται η τεχνική της γραμμικής παλινδρόμησης καθώς και συναρτήσεις υπολογισμού ιδιοτιμών. Ειδικότερα, η βιβλιοθήκη **cuSolver** αποτελείται από τρεις μικρότερες βιβλιοθήκες και πιο συγκεκριμένα

- την βιβλιοθήκη **cuSolverDN** που υποστηρίζει την παραγοντοποίηση πινάκων μέσω των τεχνικών LU, QR, SVD και LDLT, καθώς και χρήσιμες συναρτήσεις για τις μεταθέσεις των στοιχείων πινάκων και διανυσμάτων
- την βιβλιοθήκη **cuSolverSP** που περιέχει συναρτήσεις που στηρίζονται στην διαδικασία παραγοντοποίησης QR στην οποία χρησιμοποιούνται αραιοί πίνακες
- την βιβλιοθήκη **cuSolverRF** που προσφέρει παρόμοιες δυνατότητες



Κεφάλαιο 6: Προγραμματισμός μαζικά παράλληλων επεξεργαστών

Γνωστές βιβλιοθήκες της CUDA

Η βιβλιοθήκη **cuRAND** επιτρέπει την απλή και αποτελεσματική δημιουργία και χρήση ψευδοτυχαίων αριθμών

Η βιβλιοθήκη αποτελείται από δύο τμήματα και πιο συγκεκριμένα, από μία βιβλιοθήκη που βρίσκεται στον υπολογιστή που περιέχει την ΚΜΕ και από ένα αρχείο επικεφαλίδας που αναφέρεται στην συσκευή GPU

Η βιβλιοθήκη χρησιμοποιείται περιλαμβάνοντας το αρχείο επικεφαλίδας `/include/curand.h` και ενσωματώνοντας την βιβλιοθήκη στο εκτελέσιμο αρχείο κατά την φάση της σύνδεσης

Η ακολουθία των τυχαίων αριθμών, μπορεί να δημιουργηθεί είτε στην ΚΜΕ είτε στην GPU και να χρησιμοποιηθεί με τον κατάλληλο σε κάθε περίπτωση τρόπο, ανάλογα με την τεχνική της δημιουργίας της

Άλλες βιβλιοθήκες της CUDA περιλαμβάνουν

- την βιβλιοθήκη **nvJPEG** που προφέρει αποδοτικούς αλγορίθμους που σχετίζονται με αποκωδικοποίηση εικόνων
- την βιβλιοθήκη **nvBlas** που υποστηρίζει την υλοποίηση αλγορίθμων που στηρίζονται σε εργαλεία από την γραμμική άλγεβρα
- την βιβλιοθήκη **cuDNN** που υποστηρίζει πρωτογενείς λειτουργίες από τον χώρο των τεχνητών νευρωνικών δικτύων

Η τελευταία βιβλιοθήκη χρησιμοποιείται από πολλές εφαρμογές όπως είναι η εφαρμογή **TensorFlow**, μία βιβλιοθήκη ανοικτού λογισμικού που σχετίζεται με μηχανική μάθηση και τεχνητή νοημοσύνη. Εκτός από τις γλώσσες C και C++ υπάρχουν εκδόσεις των βιβλιοθηκών και σε Python, όπως είναι το περιβάλλον **PyTorch** που επιτρέπει την ανάπτυξη μοντέλων μηχανικής μάθησης.





Το παρόν συνοδευτικό έργο **παρέχεται** αποκλειστικά και μόνο στους διδάσκοντες που έχουν επιλέξει το αντίστοιχο βιβλίο μέσω του συστήματος του **Ευδόξου** ως διδακτικό σύγγραμμα για τη διδασκαλία των μαθημάτων τους και την αξιολόγηση της εκμάθησης των φοιτητών και για όσο χρονικό διάστημα διατηρείται η επιλογή του συγκεκριμένου συγγράμματος. Η **διάδοση, δημοσίευση, αναπαραγωγή ή πώληση** οπουδήποτε μέρους αυτού του έργου με οποιοδήποτε μέσο καταστρέφει την ακεραιότητα του έργου **και για σκοπούς διαφορετικούς από αυτούς για τους οποίους έχει χορηγηθεί η σχετική άδεια δεν επιτρέπεται**. Το περιεχόμενο του έργου **δεν θα πρέπει να διατίθεται** στους φοιτητές παρά μόνο όταν αυτό αναφέρεται ρητά ότι μπορεί να γίνει. Η **χρήση** του παρόντος έργου γίνεται αποκλειστικά από τον διδάσκοντα στα πλαίσια των εκπαιδευτικών καθηκόντων του και με τη χρήση των μέσων που αυτός διαχειρίζεται και έχει στη διάθεσή του από τις **επίσημες υπηρεσίες του εκπαιδευτικού ιδρύματος** όπου ανήκει, διασφαλίζοντας τη μη περαιτέρω διάδοση, δημοσίευση και αναπαραγωγή του έργου προς τρίτους. Ο διδάσκων δύναται να **προσαρμόζει** μέρος του υλικού των διαφανειών σύμφωνα με τις διδακτικές του ανάγκες, αναφερόμενος όμως πάντοτε στην αρχική πηγή αναφοράς. Το παρόν έργο προστατεύεται από την ελληνική και ευρωπαϊκή νομοθεσία περί πνευματικής ιδιοκτησίας.



Επιστημονικές Εκδόσεις

T Z I O Λ Α